VOLUME 1

Core Java & OOP (Java 21)

OOP, SOLID, language internals, collections internals, functional Java, Java 9–21 features

Java 21 · Spring Boot 3 · SQL · Microservices · AI integration
Interview preparation for 12+ years experience · real scenarios from the RefinePoint platform

Contents

1. Object-Oriented Programming & SOLID

1.1 The mental model: objects are state + behaviour + identity
1.2 The four pillars, with the depth interviewers want
1.3 Class relationships
1.4 Composition over inheritance
1.5 Abstract class vs interface (Java 8 → 21)
1.6 SOLID with real examples
1.7 Other design principles you should name-drop correctly
1.8 Interview questions and answers

2. Core Language Internals

2.1 JDK, JRE, JVM — and what happens when you run a program
2.2 Primitive types and conversions
2.3 Wrapper classes, autoboxing and the Integer cache
2.4 String internals
2.5 Methods of `java.lang.Object`
2.6 Java is always pass-by-value
2.7 `static`, `final` and initialisation
2.8 Exceptions
2.9 `var`, operators and output-prediction traps
2.10 The `main` method
2.11 `BigDecimal` for money
2.12 Date and time (`java.time`)
2.13 Interview questions and answers

3. Generics, Enums, Annotations, Reflection & Nested Classes

3.1 Generics and type erasure
3.2 Enums
3.3 Annotations
3.4 Reflection
3.5 `ServiceLoader` and the SPI pattern
3.6 Nested classes and memory leaks
3.7 Java serialization
3.8 Interview questions and answers

4. Collections Internals I: Lists, HashMap, HashSet

4.1 The collections hierarchy
4.2 ArrayList
4.3 LinkedList, Vector, Stack, ArrayDeque
4.4 HashMap internals
4.5 HashSet
4.6 `equals`, `hashCode` and mutable keys
4.7 Iterators and fail-fast behaviour
4.8 `Comparable` vs `Comparator`
4.9 Immutable and fixed-size collections
4.10 Collections utility methods
4.11 Interview questions and answers

5. Collections Internals II: Sorted, Linked, Concurrent & Specialised

5.1 TreeMap and the red-black tree

5.2 LinkedHashMap and the LRU cache
5.3 IdentityHashMap, WeakHashMap and EnumMap
5.4 PriorityQueue: a binary heap in an array
5.5 The BlockingQueue family
5.6 ConcurrentHashMap internals
5.7 CopyOnWriteArrayList, ConcurrentSkipListMap and the other concurrent collections
5.8 Java 21 sequenced collections
5.9 Choosing the right collection
5.10 RefinePoint: concurrent collections in production code
5.11 Interview questions and answers

6. Functional Java: Lambdas, Streams & Optional

6.1 Functional interfaces: the types behind every lambda
6.2 Lambda internals: no class file, one `invokedynamic`
6.3 Method references: four kinds
6.4 Stream pipeline internals
6.5 Collectors in depth
6.6 flatMap, mapMulti, reduce vs collect
6.7 Parallel streams
6.8 Primitive streams and boxing cost
6.9 Stream vs loop performance
6.10 Optional
6.11 Checked exceptions in lambdas
6.12 RefinePoint patterns with streams
6.13 Interview questions and answers

7. Java 9 to 21: Every Feature an Interviewer Asks About

7.1 The release model and the LTS line
7.2 Java 9: modules (JPMS) and the small things
7.3 Java 10: `var`
7.4 Java 11: HttpClient and String methods
7.5 Switch expressions (14) and text blocks (15)
7.6 Records (16)
7.7 Sealed classes (17)
7.8 Pattern matching: instanceof, switch, record patterns
7.9 Helpful NullPointerExceptions (14)
7.10 Sequenced collections (21)
7.11 Virtual threads, generational ZGC and other runtime changes
7.12 Removed and deprecated
7.13 Preview features in Java 21
7.14 Migrating from Java 8/11 to 17/21 and Spring Boot 3
7.15 Interview questions and answers

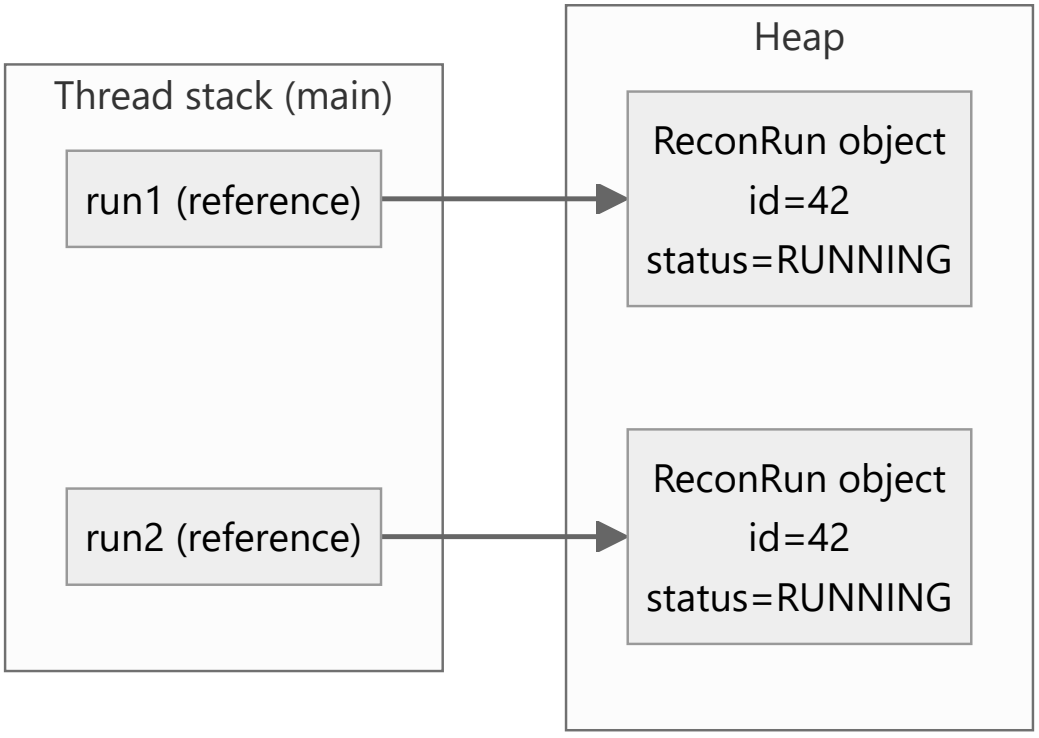
1. Object-Oriented Programming & SOLID

At 12 years of experience nobody will ask you to *define* encapsulation and stop there. They ask it to see whether you can connect the definition to design decisions: why a field is private, why an interface exists, why inheritance hurt a codebase you worked on. Every answer below gives the textbook line first (say it in one sentence), then the depth that separates a senior engineer from a junior one.

1.1 The mental model: objects are state + behaviour + identity

An object has three things:

Part	Meaning	Java mechanism
State	The data it holds right now	Instance fields
Behaviour	What it can do	Instance methods
Identity	Which one it is, independent of its state	Reference (heap address), `==`



`run1 == run2` is `false` (two identities) even though `run1.equals(run2)` may be `true` (same state, if `equals` compares `id`). Interviewers love this distinction because it leads straight into `equals/hashCode`, JPA entities and HashMap keys.

1.2 The four pillars, with the depth interviewers want

Encapsulation

One line: bundle data with the methods that operate on it and hide the data behind those methods.

Senior depth: encapsulation is not "private fields with getters and setters". A class with a public setter for every field is *not* encapsulated; it is a struct with extra steps. Real encapsulation protects **invariants** — rules that must always hold.

```java
// Not encapsulated: anyone can put the run into an impossible state
public class ReconRun {
    private String status;
    public void setStatus(String s) { this.status = s; }
}

// Encapsulated: the class owns its lifecycle rules
public class ReconRun {
    private RunStatus status = RunStatus.QUEUED;
    private Instant startedAt;

    public void start(Clock clock) {
        if (status != RunStatus.QUEUED) throw new IllegalStateException("Run already " + status);
        status = RunStatus.RUNNING;
        startedAt = clock.instant();
    }
    public void complete(int matched, int breaks) { /* only from RUNNING */ }
    public RunStatus status() { return status; }
}
```

RefinePoint: a reconciliation run moves QUEUED → RUNNING → COMPLETED/FAILED. If any service could write `status` directly, a crashed worker could leave a run "COMPLETED" without results. Putting the transition rules in the entity (and the DB claim in `FOR UPDATE SKIP LOCKED`) means the rule lives in one place.

Abstraction

One line: expose *what* something does and hide *how*.

Senior depth: abstraction is about choosing the right level of detail for the caller. Interfaces are the main tool, but a well-named method is also an abstraction. The test: can the implementation change without the caller changing?

```java
public interface LlmClient {               // the abstraction
    LlmResponse complete(LlmRequest request);
}
class AnthropicClient implements LlmClient { ... }      // detail 1
class OpenAiCompatibleClient implements LlmClient { ... } // detail 2
```

RefinePoint: the AI service talks to `LlmClient`. Switching from a local GGUF model to Claude is a config change (`ai.provider=anthropic`), and no business code (rule drafting, break explanation) changed. That is abstraction paying for itself.

Abstraction vs encapsulation (asked constantly): encapsulation hides *data* to protect invariants; abstraction hides *complexity* to simplify use. Encapsulation is implementation-level (access modifiers), abstraction is design-level (interfaces, abstract classes).

Inheritance

One line: a class acquires fields and methods of a parent class (`extends`), modelling an *is-a* relationship.

Senior depth:

- Java supports single inheritance of classes, multiple inheritance of type (interfaces).
- Inheritance creates the **tightest** coupling in Java: the subclass depends on the parent's *implementation*, not just its contract (the *fragile base class* problem).
- Prefer inheritance for genuine type hierarchies with shared invariants (e.g., `BaseEntity` holding `id`, `createdAt`), and composition for reusing behaviour.

```java
@MappedSuperclass
public abstract class BaseEntity {        // id, createdAt, updatedAt
```

```
    @Id @GeneratedValue private UUID id;
    private Instant createdAt;
}
@MappedSuperclass
public abstract class BaseOrgEntity extends BaseEntity {   // + org_id tenant column
    @Column(name = "org_id", nullable = false) private UUID orgId;
}
```

> **RefinePoint:** every tenant-scoped table extends `BaseOrgEntity`, so "every row has an org_id" is enforced by the type system instead of code review.

Fragile base class example (classic from *Effective Java*):

```
class CountingSet<E> extends HashSet<E> {
    int added = 0;
    @Override public boolean add(E e) { added++; return super.add(e); }
    @Override public boolean addAll(Collection<? extends E> c) { added += c.size(); return super.addAll(c); }
}
CountingSet<String> s = new CountingSet<>();
s.addAll(List.of("a", "b", "c"));
System.out.println(s.added); // 6, not 3 — HashSet.addAll calls add() internally
```

The subclass broke because it depended on an *undocumented implementation detail* of the parent. Fix: composition (wrap a `Set` and forward calls).

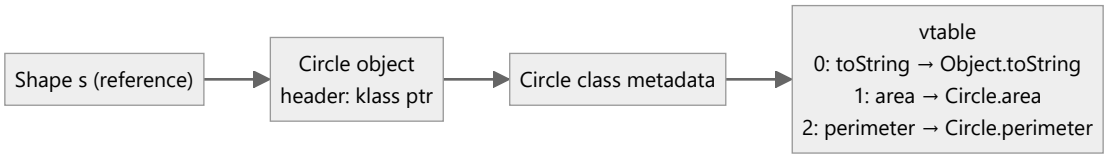
Polymorphism

One line: one interface, many forms — the same call behaves differently depending on the object.

Two kinds:

	Compile-time (static)	Runtime (dynamic)
Mechanism	Method **overloading**	Method **overriding**
Decided by	Compiler, from declared (static) types of arguments	JVM, from actual object type at runtime
Bytecode	Exact method chosen at compile time	`invokevirtual` / `invokeinterface` resolved via vtable/itable

How runtime polymorphism works internally: each class has a **virtual method table (vtable)** — an array of method pointers. A subclass copies the parent's table and replaces entries it overrides. A call `shape.area()` compiles to `invokevirtual Shape.area`; at runtime the JVM looks at the object's class pointer (in the object header), goes to that class's vtable, picks slot *n*. The JIT usually removes this lookup by **inline caching**: if a call site has only ever seen one type (monomorphic), it inlines that method directly with a cheap type guard.



Tricky overloading questions:

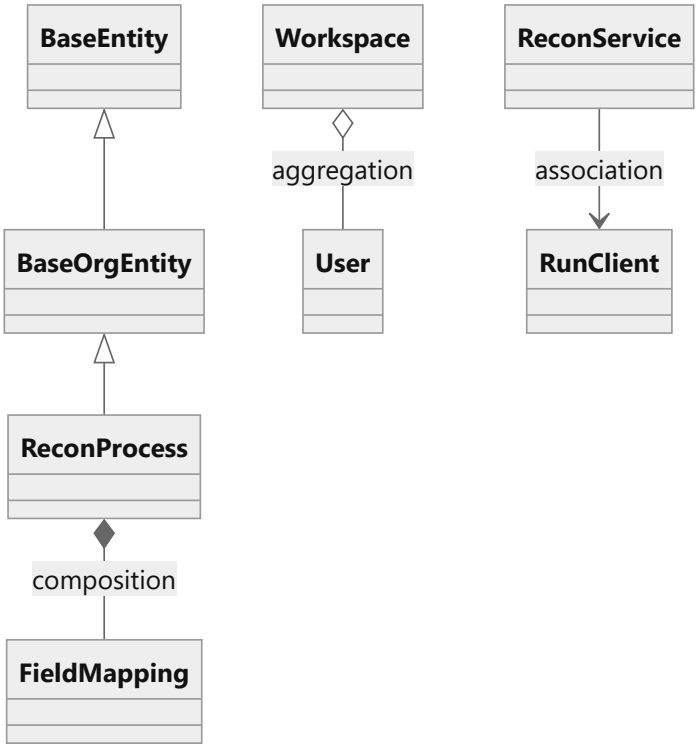
```
void print(Object o)  { System.out.println("Object"); }
void print(String s)  { System.out.println("String"); }
print(null);          // "String" — most specific applicable method wins
Object o = "hi";
print(o);             // "Object" — overloading uses the STATIC type

void m(long x) {}  void m(Integer x) {}  void m(int... x) {}
m(5);   // m(long): widening beats boxing beats varargs
```

Overload resolution phases: (1) exact/widening without boxing or varargs, (2) allow boxing/unboxing, (3) allow varargs.

1.3 Class relationships

Relationship	Meaning	Lifetime	Example
Association	Uses / knows about	Independent	`ReconService` uses `RunClient`
Aggregation	Has-a, parts can exist alone	Independent	`Workspace` has `User`s
Composition	Owns-a, part dies with whole	Dependent	`ReconProcess` owns its `FieldMapping`s
Inheritance	Is-a	—	`BaseOrgEntity` is a `BaseEntity`
Dependency	Uses temporarily (parameter)	Method call only	`export(Writer w)`



1.4 Composition over inheritance

Why prefer it:

1. Coupling only to the *interface* of the component, not its implementation.
2. Behaviour can be swapped at runtime (strategy).
3. Avoids class explosion (`LoggingCachingRetryingClient extends ...`).
4. Testability: inject a fake component.

```
// Inheritance explosion
class RetryingHttpClient extends HttpClient { }
class LoggingRetryingHttpClient extends RetryingHttpClient { }

// Composition / decorator
LlmClient client = new RetryingLlmClient(new LoggingLlmClient(new AnthropicClient(props, http)));
```

When inheritance is right: a true *is-a* that obeys Liskov, the parent is designed and documented for extension (template method), or the framework requires it (`@MappedSuperclass` , `OncePerRequestFilter`).

1.5 Abstract class vs interface (Java 8 → 21)

Feature	Abstract class	Interface
Multiple inheritance	No (single `extends`)	Yes (many `implements`)
State (instance fields)	Yes	No (only `public static final` constants)
Constructors	Yes	No
Method bodies	Yes	`default`, `static` (Java 8), `private` (Java 9)
Access modifiers on methods	Any	public (and private helpers)
Use when	Sharing state + code among closely related classes	Defining a capability/contract across unrelated classes

Diamond problem with default methods:

```
interface A { default String hello() { return "A"; } }
interface B { default String hello() { return "B"; } }
class C implements A, B {
    @Override public String hello() { return A.super.hello() + B.super.hello(); } // must resolve
}
```

Rules: (1) class methods win over interface defaults, (2) more specific interface wins (if `B extends A`, `B`'s default wins), (3) otherwise the class must override.

1.6 SOLID with real examples

S — Single Responsibility Principle

"A class should have one reason to change" — i.e., one *actor* who asks for changes.

Bad: `ReconciliationService` that loads data, matches records, writes Excel and emails the result. Finance asks for a new Excel column → you touch the matcher's class.

Good (RefinePoint shape): `ReconEngine` matches, `ExportWriter` writes files, `ExportWorker` schedules exports, `Notifier` sends mail. Each changes for one reason.

O — Open/Closed Principle

"Open for extension, closed for modification." Add new behaviour by adding code, not editing tested code.

```
public interface DataConnector {                // ryfn-plugin-api
    ConnectorDescriptor descriptor();
    ConnectorResult fetch(ConnectorContext ctx);
}
// A new source (e.g., SAP) is a new jar with META-INF/services/com.ryfnflow.plugin.connector.DataConnector
// Plugins.load(DataConnector.class) finds it via ServiceLoader — core code never changes.
```

L — Liskov Substitution Principle

Subtypes must be usable wherever the parent is expected **without breaking the caller's expectations**: no stronger preconditions, no weaker postconditions, preserve invariants, no new unchecked exceptions the caller can't expect.

Classic violation — `Square extends Rectangle`:

```
void resize(Rectangle r) { r.setWidth(5); r.setHeight(4); assert r.area() == 20; } // fails for Square
```

Java's own violation: `List.of(...)` returns a `List` whose `add` throws `UnsupportedOperationException`. The contract documents it as "optional operation", which is how the JDK legalises it — but it is a frequent source of production bugs.

I — Interface Segregation Principle

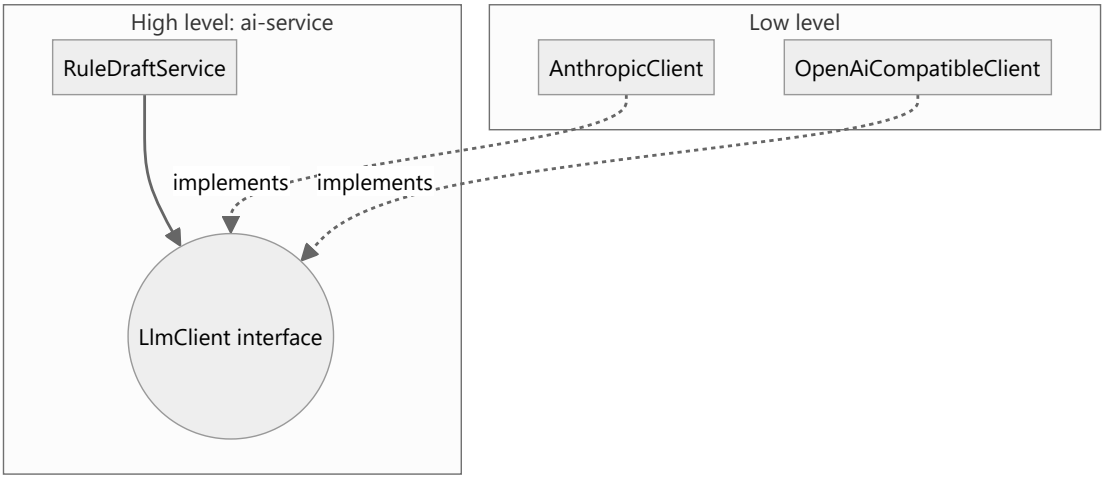
Clients should not depend on methods they do not use. Split fat interfaces into role interfaces.

```
// Fat
interface Repository<T> { T find(); void save(T t); void delete(T t); void bulkImport(Stream<T> s); void
reindex(); }
// Segregated
interface Reader<T> { Optional<T> find(UUID id); }
interface Writer<T> { void save(T t); }
```

Spring Data does this well: `Repository` (marker) → `CrudRepository` → `ListCrudRepository` → `PagingAndSortingRepository`. You can extend just `Repository` and declare only the methods you need.

D — Dependency Inversion Principle

High-level modules should not depend on low-level modules; both depend on abstractions. Abstractions are owned by the high-level policy.



DIP (principle) ≠ DI (dependency injection, a technique) ≠ IoC (inversion of control, the broader idea that a framework calls your code). Spring's container is an IoC container that performs DI, which makes DIP easy.

1.7 Other design principles you should name-drop correctly

- **DRY** — every piece of *knowledge* has one representation. Duplicated code that changes for different reasons is not a DRY violation.
- **KISS**, **YAGNI** — do not build what you do not need yet.
- **Law of Demeter** — talk to friends, not strangers: `order.getCustomer().getAddress().getCity()` is a train wreck; ask `order.shippingCity()`.
- **Tell, don't ask** — tell an object to do something rather than asking for its data and deciding outside it.
- **High cohesion, low coupling.**
- **Program to an interface, not an implementation.**

1.8 Interview questions and answers

Q1. What is OOP and why use it over procedural programming?

OOP models a program as interacting objects, each owning its data and behaviour. Benefits: invariants protected in one place (encapsulation), replaceable implementations (polymorphism), reuse through composition, and code organised around domain concepts that non-programmers recognise. Procedural code works well for small scripts or data pipelines; it degrades when many functions share and mutate the same data because any function can break any rule. In practice modern Java blends both: OOP for the domain model, functional style (streams, records, immutability) for data transformation.

Q2. Is Java 100% object-oriented?

No. Primitives (`int` , `double` , ...) are not objects, static members belong to the class not an instance, and Java 21 records/pattern matching lean toward data-oriented programming. Wrapper classes and autoboxing bridge primitives to objects. Project Valhalla (value classes) aims to close the gap.

Q3. Difference between a class and an object?

A class is a blueprint (loaded once per class loader into metaspace); an object is an instance on the heap with its own copy of instance fields. `new` allocates memory (usually in the thread's TLAB in Eden), zeroes fields, runs initialisers and the constructor, and returns a reference.

Q4. What happens step by step when you write `new ReconRun()` ?

1. If `ReconRun` is not loaded, the class loader loads, links (verify, prepare, resolve) and initialises it (static blocks run).
2. JVM allocates memory — fast path is a pointer bump in the thread-local allocation buffer (TLAB).
3. Fields are set to default values (0/null/false); object header written (mark word + class pointer).
4. Instance initialisers and field initialisers run in textual order, after the `super(...)` call.
5. Constructor body runs.
6. Reference returned. (With escape analysis the JIT may skip heap allocation entirely — scalar replacement.)

Q5. Can a constructor be private? Why?

Yes: singletons, static factory methods (`List.of`), utility classes (`private Util() {}`), and builders. Private constructors also prevent subclassing (subclass constructors must call a super constructor).

Q6. Can constructors be inherited, overridden, `final` , `static` , or `abstract` ?

None of those. Constructors are not members, so they are not inherited or overridden. The first statement is always `this(...)` or `super(...)` (implicit `super()` if omitted). Java 22+ previews statements before `super(...)` , but in Java 21 the call must be first.

Q7. Order of initialisation in an inheritance chain?

```
class P { static { print("P static"); } { print("P instance"); } P() { print("P ctor"); } }
class C extends P { static { print("C static"); } { print("C instance"); } C() { print("C ctor"); } }
new C(); new C();
```

Output: `P static, C static, P instance, P ctor, C instance, C ctor, P instance, P ctor, C instance, C ctor` . Static blocks run once at class initialisation, parent first; instance initialisers run right before each constructor body, parent chain first.

Q8. What happens if you call an overridable method from a constructor?

The subclass override runs **before the subclass's fields are initialised**:

```
class Base { Base() { init(); } void init() {} }
class Child extends Base { private final List<String> items = new ArrayList<>(); @Override void init() {
items.add("x"); } }
new Child(); // NullPointerException: items is still null
```

Rule: never call overridable methods from constructors (Effective Java item 19).

Q9. Method overloading vs overriding — every rule.

Overloading: same name, different parameter list (type/number/order), in the same class or subclass; return type alone is not enough; resolved at compile time. Overriding: same signature in subclass; return type same or **covariant** (subtype); access cannot be more restrictive; cannot throw new or broader *checked* exceptions; `static`, `private`, `final` methods cannot be overridden; resolved at runtime. Use `@Override` so the compiler catches signature mistakes.

Q10. Can we override a static method? What is method hiding?

No. A static method with the same signature in a subclass **hides** the parent's. Which one runs depends on the *reference type*, not the object:

```
class A { static String who() { return "A"; } }
class B extends A { static String who() { return "B"; } }
A ref = new B(); ref.who(); // "A"
```

Q11. Can private methods be overridden? What about `final`?

Private methods are not visible to subclasses, so a same-named method in the subclass is a new method. `final` methods cannot be overridden (compile error). The JIT can inline final and private methods without guards, but modern JITs inline virtual methods too, so `final` is about design intent, not speed.

Q12. What is covariant return type?

An override may return a subtype of the parent's return type. `Object.clone()` returns `Object`, but `MyClass.clone()` can return `MyClass`. The compiler generates a **bridge method** with the original signature that delegates to the new one.

Q13. What is dynamic method dispatch?

Choosing the overriding method at runtime based on the actual object, via `invokevirtual` / `invokeinterface` and vtables/itables. Fields are **not** polymorphic: `Parent p = new Child(); p.name` reads `Parent.name` if both declare `name`.

Q14. Abstract class with no abstract methods — useful?

Yes — prevents direct instantiation while sharing code. `HttpServlet` is abstract with no abstract methods; you are meant to subclass and override what you need.

Q15. Can an abstract class have a constructor? Can an interface?

Abstract class: yes (called via `super()` from subclasses, used to initialise shared state). Interface: no.

Q16. When choose abstract class vs interface in a real system?

Interface for a capability many unrelated types implement or for something you want to mock/swap (`LlmClient` , `DataConnector` , `Notifier`). Abstract class when subclasses share state and a fixed algorithm skeleton (template method), e.g., an `AbstractImporter` with `read() → validate() → persist()` where only `read()` varies. Often both: an interface for the contract plus an abstract skeletal implementation (`AbstractList` for `List`).

Q17. What is the diamond problem and how does Java handle it?

Ambiguity when a class inherits the same member from two paths. Java avoids it for state (no multiple class inheritance) and resolves it for default methods with explicit rules (class wins, more specific interface wins, else must override and may call `X.super.m()`).

Q18. Explain `this` and `super` .

`this` = current instance (disambiguate fields, call another constructor `this(...)` , pass self). `super` = parent's members (`super.method()` , `super(...)`). Neither can be used in static context.

Q19. What is a marker interface? Alternatives?

An interface with no methods that tags a type: `Serializable` , `Cloneable` , `RandomAccess` . The JVM/libraries check `instanceof` . Annotations are the modern alternative, but marker interfaces have one advantage: they define a *type* the compiler can check (`void write(Serializable s)`).

Q20. Explain `Cloneable` and why it is considered broken.

`Object.clone()` is protected and does a shallow field copy only if the class implements `Cloneable` (else throws `CloneNotSupportedException`). It bypasses constructors, does not work with final fields that need deep copy, and the interface does not even declare `clone()` . Prefer copy constructors or static factories: `new ArrayList<>(other)` , `Map.copyOf(map)` , record `with` -style methods.

Q21. Shallow vs deep copy?

Shallow copy duplicates the top-level object but shares referenced objects; deep copy recursively copies the graph. A shallow copy of a `ReconDefinition` that holds a mutable `List<Rule>` lets two definitions share — and corrupt — one rule list. Use immutable collections to make shallow copies safe.

Q22. What is an immutable class and how do you create one?

State cannot change after construction. Steps: class `final` (or private constructor + factories), all fields `private final` , no setters, defensive copies of mutable inputs *and* outputs, do not leak `this` during construction. Records give you most of it, but you still need defensive copies of mutable components:

```
public record MatchRule(String name, List<String> keys) {
    public MatchRule { keys = List.copyOf(keys); }   // compact constructor: defensive, unmodifiable
}
```

Benefits: thread-safe without locks, safe as map keys, cacheable, simpler reasoning.

Q23. Why is `String` immutable?

String pool safety (shared literals), security (class names, file paths, URLs cannot change after validation), thread safety, cached `hashCode` (great as HashMap keys), and class-loading correctness. (Details in chapter 2.)

Q24. Explain association, aggregation and composition with an example from your work.

In RefinePoint: `ReconService` *associates* with `RunClient` (calls it); a `Workspace` *aggregates* users (a user can belong elsewhere and survives the workspace); a `ReconProcess` is *composed* of field mappings and rules (deleting the process deletes them — `orphanRemoval = true`, `CascadeType.ALL`).

Q25. What is coupling and cohesion?

Coupling — how much one module depends on another's internals (aim low). Cohesion — how related the responsibilities inside a module are (aim high). Microservices boundaries are the same idea at deployment scale: high cohesion inside a service, low coupling (async events, versioned APIs) between them.

Q26. Explain each SOLID principle with a violation you fixed.

Have one story per letter. Example DIP story: "Our AI features called the OpenAI SDK directly from five services. When we wanted a local model for privacy-sensitive customers, every call site had to change. I introduced an `LlmClient` interface owned by the AI module with two adapters, chosen by configuration. Switching provider became a property change; tests use a stub client."

Q27. Does SOLID conflict with simplicity (KISS/YAGNI)?

It can. An interface with exactly one implementation that will never change is ceremony. Apply OCP/DIP at *volatile* boundaries (external providers, I/O, plug-in points), not everywhere. Senior answer: principles are heuristics to manage change cost; apply them where change is likely.

Q28. What is the Law of Demeter? Does a fluent builder/stream violate it?

"Only talk to your immediate friends." Builders and streams return the same object or a value of the same abstraction, so chaining is fine; the law targets reaching through *different* objects' internals (`a.getB().getC().doX()`).

Q29. What is an object's `hashCode` by default? Is it the memory address?

Not necessarily. HotSpot's default identity hash is a random/thread-local xorshift number generated on first call and stored in the object header (mark word). Objects move during GC, so it cannot be the address. Once computed, it is fixed for the object's lifetime. (Calling it on a lock-biased object historically revoked the bias; biased locking was removed in JDK 18.)

Q30. `equals` and `hashCode` contract — and what breaks if you violate it?

`equals` must be reflexive, symmetric, transitive, consistent, and `x.equals(null) == false`. If `a.equals(b)` then `a.hashCode() == b.hashCode()` (the reverse is not required). Violate it and `HashMap` / `HashSet` lose entries: an object put in a set is not found because its hash lands in a different bucket.

```
Set<Point> s = new HashSet<>();
s.add(new Point(1, 2));
s.contains(new Point(1, 2)); // false if equals overridden without hashCode
```

Mutable fields in `hashCode` are equally dangerous: mutate a key after insertion and it is "lost" in the wrong bucket.

Q31. How should JPA entities implement equals/hashCode?

Do not use all fields (lazy proxies, collections). Do not use a generated ID naïvely: before persist it is null, after persist it changes, so a `HashSet` containing the new entity breaks. Options: (a) business key that never changes, (b) ID-based equals with a constant `hashCode` (`getClass().hashCode()`), (c) assign UUIDs in the constructor (RefinePoint uses UUID ids) so the id

exists before persisting. Also compare with `Hibernate.getClass()` / `instanceof` rather than `getClass()` because of proxy subclasses.

Q32. `instanceof` vs `getClass()` in equals?

`getClass()` keeps symmetry strict (a subclass is never equal to a parent), `instanceof` allows subclass equality but risks symmetry/transitivity violations if subclasses add state. With Hibernate proxies, `getClass()` fails because the proxy is a subclass. Records use `getClass`-like semantics automatically and are final, so the problem disappears.

Q33. What is the difference between `==` and `equals()`?

`==` compares references for objects (identity) and values for primitives. `equals()` compares logical equality as defined by the class (defaults to `==` in `Object`). Watch for `Integer` cache: `Integer a = 127, b = 127; a == b` is true, but for 128 it is false (cache is -128..127 by default, tunable with `-XX:AutoBoxCacheMax`).

Q34. What is an inner class, static nested class, local and anonymous class?

Static nested: no reference to the outer instance (preferred, like `Map.Entry`). Inner (non-static): holds an implicit `Outer.this` reference — can cause memory leaks (e.g., an inner `Runnable` keeps a large outer object alive). Local: declared in a method. Anonymous: inline subclass/implementation, largely replaced by lambdas. Details in chapter 3.

Q35. What is the difference between `final`, `finally`, and `finalize`?

`final`: constant variable / non-overridable method / non-extendable class. `finally`: block that always runs after try (except `System.exit`, JVM crash, or infinite loop in try). `finalize`: deprecated (Java 9) and *removed for removal* (JEP 421, Java 18) GC callback; unpredictable, slow, can resurrect objects. Use `try-with-resources` and `java.lang.ref.Cleaner` instead.

Q36. What is "programming to an interface"?

Declare variables, parameters and return types with the most general type that provides the needed operations (`List<Run> runs = new ArrayList<>()`, `Map` not `HashMap`). The implementation can change without callers changing, and tests can supply doubles.

Q37. How does polymorphism help in Spring?

Spring injects by interface; at runtime the bean may be your class, a JDK dynamic proxy or a CGLIB subclass that adds transactions, security or caching. Your code calls `service.save()` polymorphically and gets cross-cutting behaviour for free. Collections of implementations can be injected (`List<DataConnector>`, `Map<String, MatchStrategy>`), giving strategy dispatch with zero `if/else`.

Q38. Replace a large `switch` on a type with polymorphism — and when not to.

Classic refactor: move each branch into an implementation of a common interface (Replace Conditional with Polymorphism). But Java 21 sealed interfaces + pattern-matching `switch` make the switch exhaustive and compiler-checked, which is better when the *set of types is closed* and *operations are many* (data-oriented programming). Rule of thumb: open set of types → polymorphism; closed set of types with many operations → sealed + switch (the expression problem).

Q39. What is the expression problem?

OOP makes adding new *types* easy (new subclass) but adding new *operations* hard (touch every class). Functional/sealed-switch style makes new operations easy (new function) but new types hard (touch every switch). Knowing this shows architectural maturity.

Q40. How do you design a class for extension vs prohibit it?

Either document self-use and provide protected hooks (template method), or make the class `final`/sealed. Effective Java: "Design and document for inheritance or else prohibit it." Java 17 `sealed` lets you allow a specific list of subclasses: `sealed interface Result permits Matched, Break, Ignored`.

Q41. What are access modifiers and their scopes?

Modifier	Class	Package	Subclass (other pkg)	World
private	✓			
(default)	✓	✓		
protected	✓	✓	✓ (through inheritance)	
public	✓	✓	✓	✓
Top-level classes can only be public or package-private. Modules (Java 9) add another layer: `public` is only visible outside the module if the package is `exports`ed.				

Q42. Why is multiple inheritance of classes not allowed?

To avoid ambiguity in state and method resolution (diamond) and complexity in constructors/initialisation order. Java chose single inheritance of implementation plus multiple inheritance of type (interfaces) — and since Java 8, limited inheritance of behaviour via defaults, without state.

Q43. How would you explain OOP design to a junior using a real feature?

Pick a feature end to end: "Matching strategies — `MatchStrategy` interface (abstraction), `ExactMatch`, `ToleranceMatch`, `FuzzyNameMatch` implementations (polymorphism), each immutable and configured from `MatchRuleConfig` (encapsulation), engine composed with a list of strategies (composition), new strategy = new class (OCP)."

Q44. What is object slicing? Does Java have it?

In C++, assigning a derived object to a base *value* copies only the base part. Java has no value-type objects (references only), so no slicing — a `Parent p = child` still refers to the full `Child` object.

Q45. What is a God class and how do you break it?

A class that knows/does too much (thousands of lines, dozens of dependencies). Break it by identifying responsibilities (look at which fields each method uses — clusters reveal hidden classes), extract classes, introduce facades for callers, and move behaviour next to the data it uses. Do it incrementally behind tests.

Q46. Is getter/setter on every field good OOP?

Not by default. It exposes internals and invites anaemic models. Expose behaviour (`run.start()`), keep getters for read models/DTOs, and use records for pure data carriers. JPA needs a no-arg constructor and field access, not setters.

Q47. Anaemic domain model vs rich domain model?

Anaemic: entities are data bags, all logic in services (procedural code in OO clothing). Rich: entities hold behaviour and invariants. In large Spring apps a pragmatic middle ground is common: entities guard their own state transitions, services orchestrate across aggregates, transactions and I/O.

Q48. What are value objects?

Small immutable objects defined by their value, not identity: `Money(amount, currency)`, `OrgCode`, `DateRange`. They validate on construction, so invalid values cannot exist. Records are perfect value objects. Using `BigDecimal amount` + `String currency` everywhere instead of `Money` is "primitive obsession".

Q49. Explain the "is-a" vs "has-a" test with a mistake you've seen.

`Stack extends Vector` in the JDK — a stack is *not* a vector, yet it inherits `add(index, e)` which breaks LIFO. Also `Properties extends Hashtable` allowing non-String values. Both should have used composition.

Q50. What does "favour immutability" buy you in a multi-threaded Spring app?

Singleton beans are shared across request threads. If a bean holds mutable instance state (e.g., a `SimpleDateFormat` field or a per-request list), you get race conditions. Immutable beans and immutable DTOs/records eliminate a whole class of bugs without synchronisation.

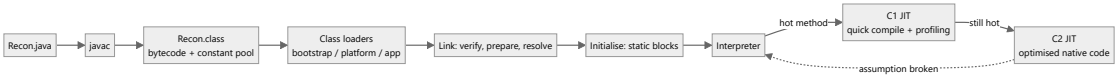
2. Core Language Internals

Core-language rounds are where senior candidates lose points they should never lose. The questions sound basic ("is Java pass-by-reference?", "why is String immutable?"), but the interviewer is listening for the *mechanism*: the string pool and `invokedynamic`, the Integer cache, how `finally` interacts with `return`, why `new BigDecimal(0.1)` is wrong. This chapter walks through each mechanism once with a picture, then drills it with output-prediction and production questions.

2.1 JDK, JRE, JVM — and what happens when you run a program

Term	What it is	Contains
JVM	A specification + an implementation (HotSpot, OpenJ9) that executes bytecode	Class loader subsystem, runtime data areas (heap, stacks, metaspace), execution engine (interpreter, JIT, GC)
JRE	Runtime environment to *run* Java programs	JVM + core class libraries (`java.base` , …)
JDK	Development kit to *build* Java programs	JRE + tools: `javac` , `jar` , `javadoc` , `jshell` , `jlink` , `jcmd` , `jfr` , `jpackage`

Since Java 11 Oracle no longer ships a separate JRE download. You either install a JDK or build a trimmed runtime with `jlink` containing only the modules your app needs (common in Docker images).



Execution engine in one paragraph: the JVM starts interpreting bytecode. Methods that are called often (invocation and back-edge counters) are compiled by C1 with profiling, then by C2 using that profile (inlining, escape analysis, loop unrolling). If a speculative assumption breaks — a new subclass is loaded and a call site is no longer monomorphic — the code is **deoptimised** back to the interpreter. This is *tiered compilation* and is why Java services need a warm-up period and why microbenchmarks need JMH.

Write once, run anywhere works because `.class` files contain platform-neutral bytecode; only the JVM is platform-specific.

2.2 Primitive types and conversions

Type	Size	Default (field)	Range / notes
`byte`	8 bit	0	-128 … 127
`short`	16 bit	0	-32,768 … 32,767
`char`	16 bit unsigned	`'\u0000'`	0 … 65,535 (a UTF-16 code unit, not a "character")
`int`	32 bit	0	±2.1 billion
`long`	64 bit	0L	±9.2 × 10^18
`float`	32 bit IEEE 754	0.0f	~7 decimal digits
`double`	64 bit IEEE 754	0.0d	~15–16 decimal digits
`boolean`	JVM-dependent (int on stack, byte in arrays)	false	true/false

Local variables have **no default** — the compiler refuses to read an unassigned local ("definite assignment").

Widening (safe, implicit): `byte → short → int → long → float → double` , and `char → int` . Note `long → float` is "widening" yet can lose precision: `(float) 123456789L` is `1.23456792E8` .

Narrowing (explicit cast): keeps the low-order bits. `(byte) 200` is `-56`; `(int) 3.99` is `3` (truncation toward zero); `(int) Double.NaN` is `0`; `(int) 1e20` is `Integer.MAX_VALUE` (floating-to-int saturates).

Arithmetic promotion: any arithmetic on `byte`, `short`, `char` is done in `int`.

```
byte a = 10, b = 20;
byte c = a + b;        // compile error: int cannot be converted to byte
byte d = (byte)(a + b);
a += b;                // compiles: compound assignment hides a cast — (byte)(a + b)
char ch = 'A' + 1;     // compiles: constant expression fits in char → 'B'
```

Overflow is silent for integers: `Integer.MAX_VALUE + 1 == Integer.MIN_VALUE`. Use `Math.addExact`, `Math.multiplyExact`, `Math.toIntExact` when overflow must fail loudly (counting rows, computing offsets). `Math.abs(Integer.MIN_VALUE)` is still negative.

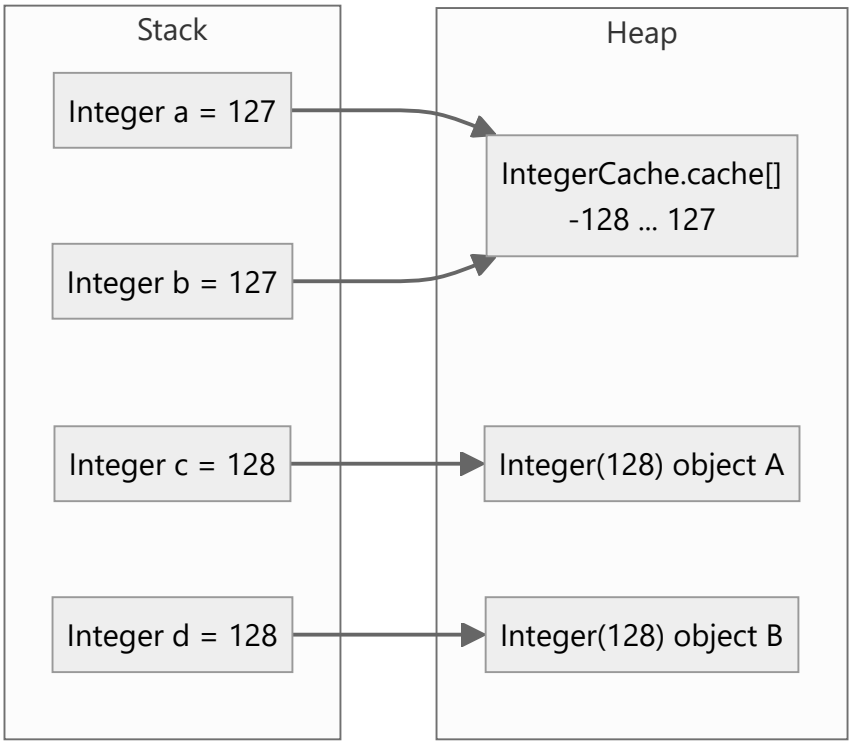
Floating point: `0.1 + 0.2 == 0.30000000000000004`; `0.0 / 0.0` is `NaN` and `NaN != NaN`; `1.0 / 0` is `Infinity` but `1 / 0` throws `ArithmeticException`; `-0.0 == 0.0` is `true` but `Double.valueOf(-0.0).equals(0.0)` is `false`.

2.3 Wrapper classes, autoboxing and the Integer cache

Every primitive has a wrapper (`Integer`, `Long`, `Character`, …). The compiler inserts conversions:

```
Integer x = 5;         // compiles to Integer.valueOf(5)   — boxing
int y = x;             // compiles to x.intValue()         — unboxing
```

`Integer.valueOf(int)` returns a **cached instance** for -128…127 (`Integer.IntegerCache`, built at class initialisation). The upper bound is tunable with `-XX:AutoBoxCacheMax=<n>` (or `java.lang.Integer.IntegerCache.high`).



Wrapper	Cached by `valueOf`
`Byte`, `Short`, `Long`	-128 … 127
`Integer`	-128 … 127 (upper bound configurable)
`Character`	0 … 127
`Boolean`	`TRUE`, `FALSE`
`Float`, `Double`	none

Autoboxing traps you must be able to explain:

```
Integer a = 1000, b = 1000;
a == b;                          // false — two objects
a.equals(b);                     // true

Long total = 0L;
for (long i = 0; i < 1_000_000; i++) total += i;   // boxes a new Long every iteration

Map<String, Integer> counts = new HashMap<>();
int n = counts.get("missing");  // NullPointerException on unboxing null

Integer i = null;
int r = flag ? i : 0;           // NPE: mixed Integer/int ternary unboxes i

List<Integer> list = new ArrayList<>(List.of(1, 2, 3));
list.remove(1);                 // removes INDEX 1 → [1, 3]
list.remove(Integer.valueOf(1));// removes VALUE 1
```

Constructors like `new Integer(5)` are deprecated for removal (Java 9/16) because they defeat the cache and will clash with value classes (Project Valhalla). Use `valueOf` or rely on autoboxing.

2.4 String internals

Layout and immutability

```
public final class String implements Serializable, Comparable<String>, CharSequence {
    @Stable private final byte[] value;   // Java 9+: bytes, not char[]
    private final byte coder;             // LATIN1 = 0, UTF16 = 1
    private int hash;                     // cached hashCode, 0 until computed
    private boolean hashIsZero;           // Java 13+: distinguishes "computed as 0"
}
```

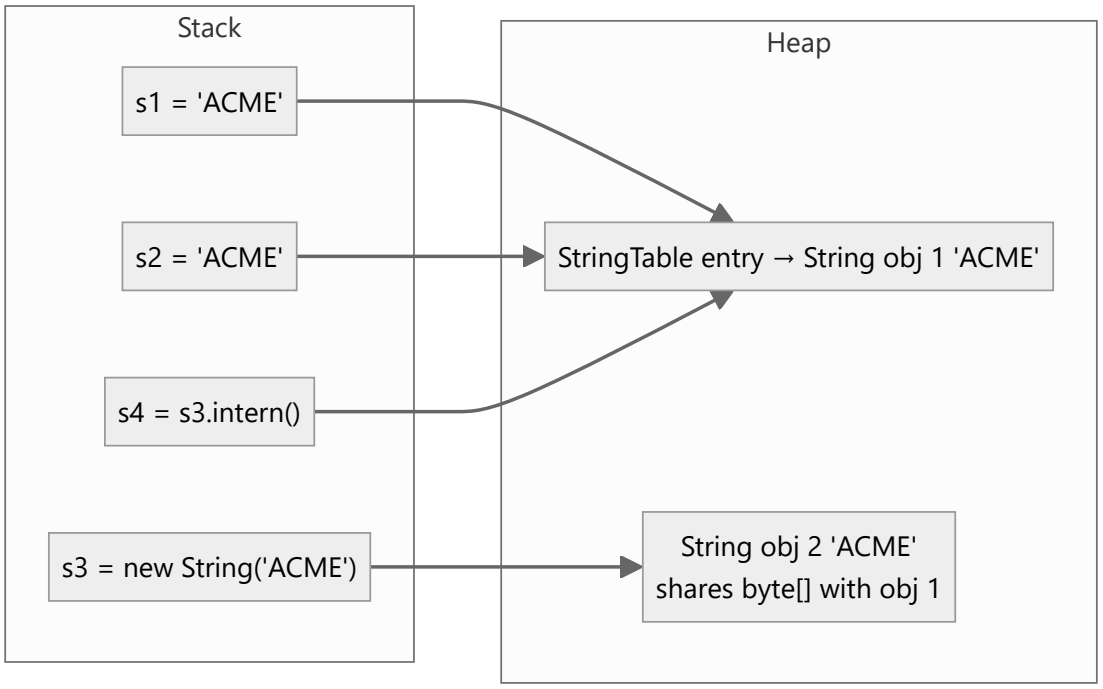
Compact strings (JEP 254, Java 9): if every character fits in Latin-1, the string stores one byte per char; otherwise two bytes (UTF-16). Most business strings (codes, IDs, English text) halve their memory. Turn it off with `-XX:-CompactStrings` (rarely useful).

Why immutable (have all five reasons ready):

1. **String pool** — literals are shared; if one caller could mutate `"USD"`, every other user would see it.
2. **Security** — file paths, class names, URLs, SQL validated once cannot be changed after the check (time-of-check/time-of-use).
3. **Thread safety** — share freely without locks.
4. **Cached hash** — computed once, perfect as `HashMap` keys.
5. **Class loading** — class names passed to loaders must not change.

The string pool

The pool (`StringTable`) is a native hash table of references to `String` objects that live in the **heap** (moved from PermGen to heap in Java 7, so pooled strings are garbage-collectable). Its size is `-XX:StringTableSize` (default 65,536 buckets in recent JDKs).



- Literals are resolved through the constant pool to the **same** pooled instance (`ldc` instruction), so `s1 == s2` .
- `new String("ACME")` always creates a new object (the literal is still pooled) — `s1 != s3` .
- `intern()` returns the pooled instance, adding this one if absent — `s1 == s4` .
- **Compile-time constants are folded:** `"AC" + "ME"` is the literal `"ACME"` . A `final String a = "AC"` local is also a constant variable, so `a + "ME" == "ACME"` . A non-final variable is not, so the result is a new object.

Interning in production: useful to dedupe millions of repeated values (currency codes, country codes) parsed from files — but each `intern()` call is a native hash lookup, and a huge pool slows GC root scanning. G1's `-XX:+UseStringDeduplication` deduplicates the *backing arrays* of long-lived strings in the background without changing identity, which is usually the safer choice.

> **RefinePoint:** a large bank statement can repeat five currency codes across millions of rows. Held as Java objects, that is millions of equal `String` s, which is one reason large files are processed in DuckDB rather than on the heap. For repeated values that do stay on the heap, a small `Map<String, String>` canonicaliser keeps one instance per value without touching the global pool.

Concatenation: `+` , `StringBuilder` , `invokedynamic`

Before Java 9, `a + b + c` compiled to `new StringBuilder().append(a).append(b).append(c).toString()` . Since Java 9 (JEP 280) it compiles to a single `invokedynamic` call bootstrapped by `StringConcatFactory.makeConcatWithConstants` . At first execution the JVM builds an optimised method handle that pre-computes the exact length and coder, allocates the `byte[]` once and copies parts in — fewer allocations, and the strategy can improve in future JDKs without recompiling your code.

But inside a **loop**, `+=` still creates a new String on every iteration ($O(n^2)$ copying). Use one `StringBuilder` (or `String.join` , `Collectors.joining`).

```
String csv = "";
for (String s : values) csv += s + ",";          // O(n^2): copies the whole string every time
```

```
var sb = new StringBuilder(values.size() * 16);    // presize to avoid array growth
for (String s : values) sb.append(s).append(',');
```

	`String`	`StringBuilder`	`StringBuffer`
Mutability	Immutable	Mutable	Mutable
Thread-safe	Yes (immutable)	No	Yes (`synchronized` methods)
Speed	New object per change	Fastest	Slower (lock per call)
Since	1.0	1.5	1.0
Use	Values, keys	Building strings in one thread (99% of cases)	Legacy APIs only

`StringBuilder` starts with capacity 16 and grows to `(old * 2) + 2` when full, copying the array. `StringBuffer` 's locking is almost never useful: sharing a builder between threads is a design smell.

Other String facts worth knowing

- `substring` copies the range (since 7u6). Before that, it shared the parent array, causing leaks when a small substring kept a huge string alive.
- `strip()` (Java 11) is Unicode-aware; `trim()` removes only chars ≤ `' '`. `isBlank()`, `lines()`, `repeat()`, `indent()`, `formatted()` and text blocks (`"""`) are Java 11–15 additions.
- `switch` on String uses `hashCode()` then `equals()` — compiled to a `lookupswitch` on the hash.
- `String.valueOf((char[]) null)` throws `NullPointerException`, but `String.valueOf((Object) null)` returns `"null"`. A bare `String.valueOf(null)` picks the `char[]` overload (most specific) and throws.
- Store passwords in `char[]`, not `String`: a char array can be wiped; a String stays in memory (and maybe the pool) until GC.

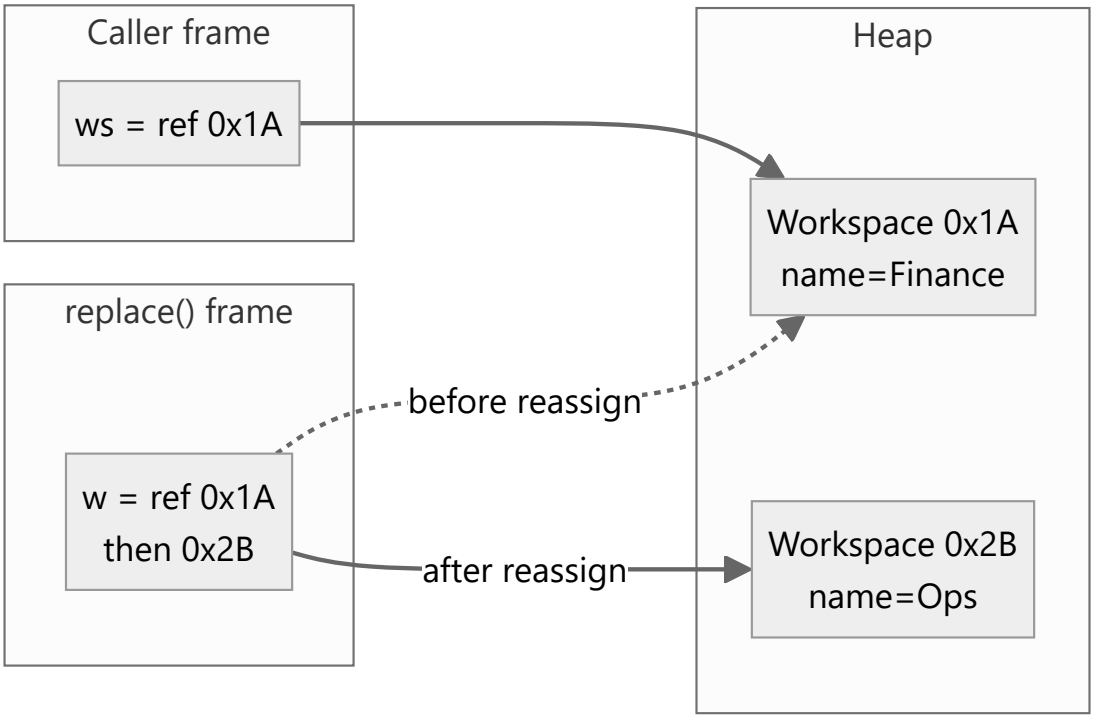
2.5 Methods of `java.lang.Object`

Method	Purpose / key fact
`equals(Object)`	Logical equality; default is `==`
`hashCode()`	Must agree with `equals`; default is an identity hash stored in the object header
`toString()`	Default `ClassName@hexIdentityHash`
`getClass()`	Runtime `Class` object; `final`
`clone()`	`protected`, shallow copy, needs `Cloneable`
`wait()`, `notify()`, `notifyAll()`	Monitor condition methods; must hold the object's monitor (`synchronized`) or get `IllegalMonitorStateException`
`finalize()`	Deprecated for removal (JEP 421); use `Cleaner` / try-with-resources

2.6 Java is always pass-by-value

Java passes a **copy of the value** in the variable. For a primitive the value is the number; for an object the value is the **reference** (a pointer). The method gets its own copy of the pointer: it can mutate the object the pointer refers to, but reassigning the parameter does not affect the caller.

```
void rename(Workspace w) { w.setName("Finance"); }     // visible to caller (same object)
void replace(Workspace w) { w = new Workspace("Ops"); } // invisible to caller (local copy reassigned)
void swap(Integer a, Integer b) { Integer t = a; a = b; b = t; } // swaps nothing
```



2.7 `static`, `final` and initialisation

static fields belong to the class. Since Java 8 they live in the heap inside the `java.lang.Class` mirror object (not PermGen/metaspace). One copy per class **per class loader** — two web apps in one Tomcat each get their own "singleton".

A class is initialised (static blocks run, in textual order with static field initialisers) on its first *active use*: `new`, static method call, non-constant static field access, `Class.forName(name)`, reflection, or initialising a subclass. Accessing a `static final` **compile-time constant** does *not* initialise the class because the value is inlined at the call site.

Lazy holder idiom — thread-safe lazy singleton using class-initialisation guarantees (the JVM holds an init lock):

```
public final class FxRates {
    private FxRates() {}
    private static final class Holder { static final FxRates INSTANCE = load(); }
    public static FxRates get() { return Holder.INSTANCE; }  // Holder initialised on first call
}
```

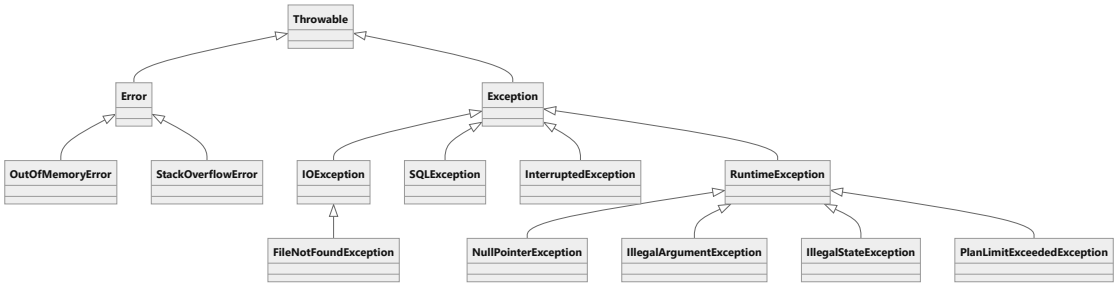
final meanings:

- `final` variable: assigned once. A **blank final** field must be assigned in every constructor path.
- `final` reference ≠ immutable object: `final List<String> l = new ArrayList<>(); l.add("x");` is legal.
- `final` method: cannot be overridden. `final` class: cannot be extended (`String`, wrappers, records).
- **Effectively final:** a local never reassigned; lambdas and inner classes may capture only effectively final locals because they capture a *copy* of the value.
- **JMM guarantee:** fields that are `final` are safely published after the constructor finishes (if `this` didn't escape), so other threads never see their default values.

> **Trap:** `public static final int MAX_ROWS = 500_000;` in a shared library is **inlined** into every caller's bytecode. Change it to 1,000,000 and redeploy only the library: services compiled against the old jar still use 500,000. Expose values that may change through a static method or a configuration property. In RefinePoint, a change to `ryfn-platform` is followed by `mvn clean package` on every service, so no service keeps a stale platform jar or a stale inlined constant.

2.8 Exceptions

Hierarchy



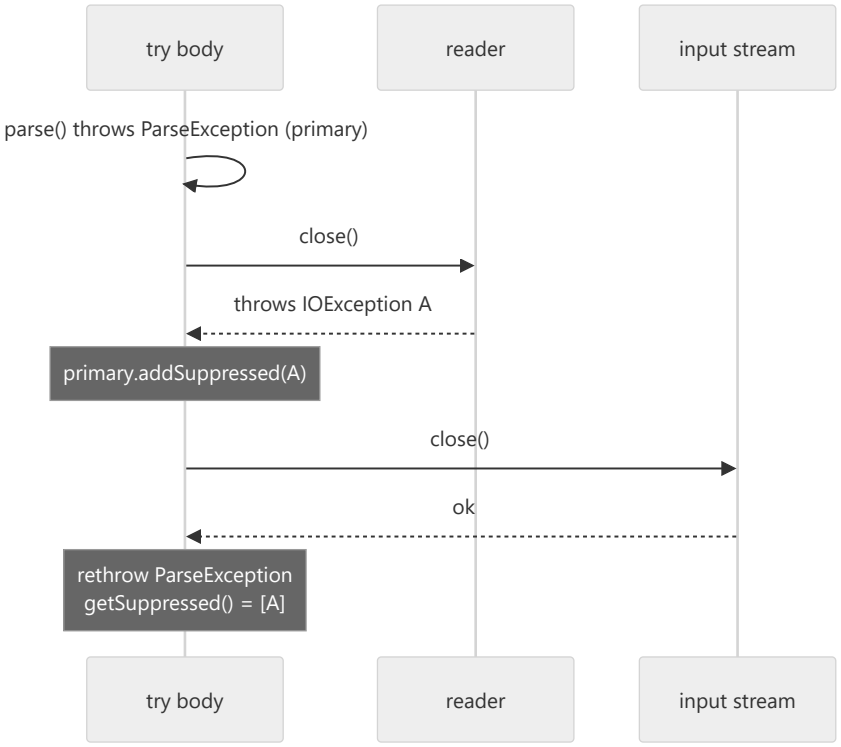
	Checked	Unchecked
Types	`Exception` and subclasses except `RuntimeException`	`RuntimeException`, `Error` and subclasses
Compiler	Must be caught or declared in `throws`	No requirement
Meaning	Recoverable condition outside the program's control (I/O, network)	Programming bugs or unrecoverable state
JVM level	No difference — checking is purely a `javac` feature	Same

Checked exceptions do not compose with lambdas (`Function` cannot throw `IOException`), which is a big reason modern frameworks (Spring, JPA, Jackson's newer APIs) use unchecked exceptions. Spring translates `SQLException` into the unchecked `DataAccessException` hierarchy.

try-with-resources and suppressed exceptions

```
try (var in = Files.newInputStream(path);
     var reader = new BufferedReader(new InputStreamReader(in))) {
    return parse(reader);
}   // reader.close() then in.close() — reverse order of declaration
```

The compiler generates the `finally` logic: resources (anything `AutoCloseable`) are closed in **reverse order**, even if the body throws. If the body throws *and* `close()` throws, the close exception is attached to the primary one via `addSuppressed()` instead of replacing it (the pre-Java-7 `finally { close(); }` pattern lost the original error).



Java 9 allows effectively final variables in the header: `try (conn) { ... }`.

`finally` edge cases

```
int f() { try { return 1; } finally { return 2; } }        // returns 2, and swallows any exception
int g() { int x = 1; try { return x; } finally { x = 99; } } // returns 1: value saved before finally runs
void h() { try { System.exit(0); } finally { log("never"); } } // finally does not run
```

`finally` does not run on `System.exit`, `Runtime.halt`, JVM crash, the thread being killed by the OS, or an infinite loop/deadlock inside `try`. A `return` or `throw` in `finally` discards the pending exception — never do it.

Custom exceptions, translation and chaining

```
public class CreditLimitExceededException extends RuntimeException {
    private final String limit;
    public CreditLimitExceededException(String limit, long allowed) {
        super("Credit limit reached: " + limit + " (allowed " + allowed + ")");
        this.limit = limit;
    }
    public String limit() { return limit; }
}

try {
    return sftp.download(path);
} catch (SftpException e) {
    throw new IngestFailedException("Cannot fetch " + path, e);   // translate + chain the cause
}
```

Guidelines: extend `RuntimeException` for domain errors; carry structured fields (code, limit) not just text; always pass the cause; translate low-level exceptions at layer boundaries so the controller does not know about SFTP or JDBC.

> **RefinePoint:** the shared `GlobalExceptionHandler` in `ryfn-platform` (`@RestControllerAdvice`) maps domain exceptions to the `ApiResponse` error envelope: `PlanLimitExceededException` becomes HTTP 402, `ConflictException` a conflict, `RateLimitExceededException` a 429. Services throw meaningful exceptions, controllers have no try/catch boilerplate, and every service returns errors in the same shape.

The cost of exceptions

Creating an exception is expensive mostly because the constructor calls `fillInStackTrace()`, which walks the stack. Throwing and catching within the same compiled method is cheap. For high-frequency, expected control flow (validation of millions of rows) either avoid exceptions (return a result object) or create exceptions without stack traces:

```
public class RowRejected extends RuntimeException {
    public RowRejected(String msg) { super(msg, null, false, false); } // no suppression, no stack trace
}
```

HotSpot's `-XX:+OmitStackTraceInFastThrow` (on by default) replaces repeatedly thrown implicit exceptions (NPE, `ArithmeticException`, `ArrayIndexOutOfBounds`) in hot compiled code with a preallocated exception **without a stack trace** — the reason production logs sometimes show `java.lang.NullPointerException` with no lines. Look further back in the logs for the first occurrence, or run with `-XX:-OmitStackTraceInFastThrow`.

2.9 `var`, operators and output-prediction traps

`var` **(Java 10)** is local-variable type inference — static typing still applies; the type is fixed at compile time. Allowed: locals with initialisers, `for` loops, try-with-resources, lambda parameters (Java 11, so you can annotate them). Not allowed: fields, method parameters, return types, `var x;`, `var x = null;`, array initialisers `var a = {1,2}`, lambdas without target type. `var list = new ArrayList<>();` infers `ArrayList<Object>` — a trap. `var` is a reserved type name, not a keyword, so a variable named `var` still compiles.

Operator facts that output questions exploit:

```
int i = 5; i = i++;           // 5: old value saved, i incremented, then old value assigned
int j = 5; j = j++ + ++j;     // 5 + 7 = 12
System.out.println(1 + 2 + "3");   // "33": left to right, numeric add first
System.out.println("1" + 2 + 3);   // "123"
System.out.println('a' + 'b');     // 195: char + char is int
System.out.println("" + 'a' + 'b');// "ab"
System.out.println(-7 % 3);        // -1: sign follows the dividend
System.out.println(-8 >> 1);       // -4: arithmetic shift keeps sign
System.out.println(-8 >>> 28);     // 15: logical shift fills zeros
System.out.println(Math.round(-2.5)); // -2: rounds half up (floor(x + 0.5))
System.out.println(Math.round(2.5));  // 3
System.out.println(1.0 / 0);       // Infinity
System.out.println(0.1f == 0.1);   // false: float widened to double differs
Object o = true ? 1 : "s";         // Integer
System.out.println(true ? 1 : 2.0);// 1.0: binary numeric promotion in ternary
```

Short-circuit: `&&` / `||` skip the right side; `&` / `|` on booleans evaluate both. Precedence trap: `a + b * c`, `x == 1 || y == 2 && z == 3` (`&&` binds tighter).

2.10 The `main` method

`public static void main(String[] args)` — `public` so the launcher can call it from outside the package, `static` so no instance is needed, `void` because the exit status is set with `System.exit(int)`, `String[]` for command-line arguments. `String... args` and `final String[] args` are also valid; overloaded `main` methods are allowed but only the standard signature is the entry point.

Newer launchers:

- Java 11: run a single source file directly — `java Hello.java` (compiles in memory).
- Java 21: unnamed classes and **instance main methods** in preview (JEP 445): `void main() { System.out.println("hi"); }` with `--enable-preview`. They became final in Java 25.

In Spring Boot, `main` calls `SpringApplication.run(App.class, args)`; the fat jar's real `Main-Class` is Spring Boot's `JarLauncher`, which sets up a class loader over `BOOT-INF/lib` and then calls your `main` via the `Start-Class` manifest attribute.

2.11 `BigDecimal` for money

`double` is binary floating point: 0.1 has no exact representation, so sums drift (`0.1 + 0.2 != 0.3`). Money must be exact. `BigDecimal` = **unscaled `BigInteger` value × 10^-scale**: `123.45` is unscaled `12345`, scale `2`.

```
new BigDecimal(0.1);          // 0.1000000000000000055511151231257827021181583404541015625
new BigDecimal("0.1");        // 0.1  — use the String constructor
BigDecimal.valueOf(0.1);      // 0.1  — uses Double.toString, fine for literals

new BigDecimal("2.0").equals(new BigDecimal("2.00"));    // false: scale differs
new BigDecimal("2.0").compareTo(new BigDecimal("2.00")); // 0: numerically equal

BigDecimal.ONE.divide(BigDecimal.valueOf(3));            // ArithmeticException: non-terminating
BigDecimal.ONE.divide(BigDecimal.valueOf(3), 10, RoundingMode.HALF_EVEN); // 0.3333333333
amount.setScale(2, RoundingMode.HALF_EVEN);              // banker's rounding
```

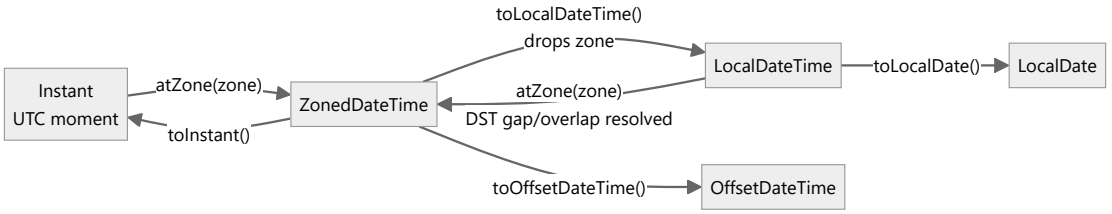
Rules: construct from `String` or `valueOf`; compare with `compareTo` (and beware `HashSet<BigDecimal>` treats `2.0` and `2.00` as different — normalise with `stripTrailingZeros()` first); always give a scale and `RoundingMode` for division; keep the currency next to the amount (a `Money` record); map to `NUMERIC(19,4)` in PostgreSQL. `BigDecimal` is immutable — `amount.add(x)` without assigning the result does nothing.

> **RefinePoint:** tolerance checks in `ryfn-engine` compare exactly, as in the balance control `expected.subtract(b).abs().compareTo(tol) > 0`, never with `double`. `FxRates` holds rates as `BigDecimal`, and when only the opposite direction is quoted it derives the rate with `BigDecimal.ONE.divide(rate, 10, RoundingMode.HALF_UP)`: an explicit scale and rounding mode, because a bare `divide` would throw on a non-terminating result. A reconciliation that says "matched" because two doubles were "close enough" would be a correctness bug in a finance product.

2.12 Date and time (`java.time`)

The legacy `java.util.Date` (a mutable instant with a misleading `toString`), `Calendar` (months from 0) and `SimpleDateFormat` (**not thread-safe** — a shared static formatter corrupts dates under load) were replaced in Java 8 by `java.time` (JSR-310): immutable, thread-safe, clearly separated concepts.

Type	Represents	Example use
`Instant`	A point on the UTC timeline (seconds + nanos since epoch)	`createdAt`, event timestamps, audit logs
`LocalDate`	A calendar date, no time, no zone	Value date, statement date, month-end
`LocalDateTime`	Date + time **without** zone — not a moment	"Run every day at 02:00" in some zone
`ZonedDateTime`	Date + time + `ZoneId` (DST rules)	Scheduling in a user's zone
`OffsetDateTime`	Date + time + fixed offset (`+05:30`)	APIs, JDBC `timestamptz` mapping
`Duration` / `Period`	Time-based (seconds) / date-based (days, months) amount	Timeouts / "3 months"
`ZoneId` / `ZoneOffset`	`Asia/Kolkata` (rules) / `+05:30` (fixed)	
`Clock`	Injectable source of "now"	Testable code



Rules for backend systems:

- Store moments as `Instant` / `timestamptz` in UTC. Convert to a zone only for display or for business rules that depend on a local calendar.
- `LocalDateTime` is not a moment — "2026-03-29 01:30" in Europe/London does not exist (DST gap) and "2026-10-25 01:30" happens twice.
- `Period.ofMonths(1)` added to Jan 31 gives Feb 28 (clamped); `Duration.ofDays(1)` across a DST change is 24 hours, `Period.ofDays(1)` is one calendar day (23 or 25 hours).
- Inject `Clock` instead of calling `Instant.now()` directly so tests can freeze time.
- Format with `DateTimeFormatter` (immutable and thread-safe); `ISO_INSTANT` for APIs.
- Jackson needs `JavaTimeModule` (Spring Boot registers it) and `WRITE_DATES_AS_TIMESTAMPS=false` for ISO strings.

> **RefinePoint:** FX quotes are `LocalDate` -keyed because a rate belongs to a calendar date, while execution times are `Instant` s. In job-service each job stores an IANA `time_zone` ; `Schedule` computes the next due time in that `ZoneId` , and it is stored as `next_run_at` , an absolute moment that the claim query compares with `now()` . A 02:00 job in New York and one in Kolkata both fire at the right moment, and the zone rules handle DST rather than our code.

2.13 Interview questions and answers

Q1. What is the difference between JDK, JRE and JVM?

The JVM executes bytecode, the JRE is the JVM plus the standard libraries needed to run programs, and the JDK is the JRE plus development tools (`javac` , `jar` , `jlink` , `jcmd` , `jfr`). The JVM is platform-specific; bytecode is not, which gives "write once, run anywhere". Since Java 11 there is no separate JRE download — production images either use a full JDK base image or a `jlink` -built minimal runtime.

Q2. Is the JVM an interpreter or a compiler?

Both. HotSpot starts by interpreting bytecode, profiles it, and compiles hot methods to native code with the C1 then C2 JIT compilers (tiered compilation). Optimisations are speculative (based on observed types), so code can be deoptimised when

assumptions break. GraalVM can additionally compile ahead of time into a native image (Spring Boot supports it with AOT processing).

Q3. What are the class loaders and what is parent delegation?

Bootstrap (native, loads `java.base`), Platform (other JDK modules), and Application/System (your classpath). A loader first asks its parent; only if the parent cannot find the class does it try itself. This prevents application code from replacing `java.lang.String` and ensures one copy of core classes. A class's identity is *(class name + defining loader)*, so the same class loaded by two loaders gives `ClassCastException: X cannot be cast to X`. Spring Boot DevTools uses a restart class loader for your code, which is why that error appears there sometimes.

Q4. Why is `char` 2 bytes in Java?

Java was designed around UCS-2/UTF-16, so `char` is a 16-bit UTF-16 code unit. Characters outside the Basic Multilingual Plane (emoji, some CJK) need two chars (a surrogate pair), so an emoji string has `length()` 2. Use `codePointCount` / `codePoints()` when you need real character counts. Compact strings changed how `String` *stores* text, not the size of `char`.

Q5. What is the output: `byte b = (byte) 130; System.out.println(b);` ?

`-126`. Narrowing keeps the low 8 bits: 130 is `1000 0010`, which in two's complement is -126.

Q6. Why does `short s = 1; s = s + 1;` fail but `s += 1;` compile?

`s + 1` is an `int` expression (binary numeric promotion), and assigning `int` to `short` needs an explicit cast. Compound assignment operators include an implicit cast: `s += 1` is `s = (short)(s + 1)`. That hidden cast can also hide overflow.

Q7. Is `long → float` widening? Can it lose data?

Yes, it is a widening primitive conversion and yes, it can lose precision: a `float` has only 24 bits of mantissa, so large `long` values are rounded. Widening guarantees no loss of *magnitude*, not of precision.

Q8. What is the Integer cache and why does `Integer a = 128, b = 128; a == b` print false?

`Integer.valueOf` (used by autoboxing) returns cached objects for -128…127; outside that range each boxing creates a new object, so `==` compares two different references. Always compare wrappers with `equals` (or unbox). The high bound can be raised with `-XX:AutoBoxCacheMax`, which is why code relying on `==` may pass on one machine and fail on another.

Q9. What happens when you unbox `null`?

`NullPointerException`, thrown from the compiler-inserted `intValue()` call. Common sources: `int x = map.get(key)`, `if (entity.getCount() > 0)` when the column is nullable, and mixed ternaries `cond ? wrapper : 0`. Use primitives for required values and `Optional` or explicit null checks for nullable ones.

Q10. Explain the ternary NPE: `Integer i = null; Integer r = true ? i : 0;`

When one operand is `Integer` and the other is `int`, the conditional expression's type is `int` (binary numeric promotion), so `i` is unboxed — NPE — and then reboxed. Fix: make both operands `Integer` (`Integer.valueOf(0)`) or use an `if`.

Q11. `list.remove(1)` on a `List<Integer>` — what is removed?

The element at **index** 1, because `remove(int)` is an exact match and overload resolution prefers it over boxing to `remove(Object)`. Use `list.remove(Integer.valueOf(1))` to remove the value.

Q12. Why is `String` immutable? Give at least four reasons.

Pool sharing (literals are shared safely), security (validated paths, URLs, class names cannot change after checks), thread safety without locks, cached hash code for fast `HashMap` lookups, and safe use in class loading. Immutability also makes strings safe keys and safe to pass across API boundaries without defensive copies.

Q13. How many objects are created by `String s = new String("abc");`?

Up to two: the literal `"abc"` is placed in the string pool when the class's constant is resolved (if not already there), and `new` creates a second `String` on the heap. In modern JDKs both share the same `byte[]`, so the second object costs only its header and fields. If `"abc"` was already pooled, only one new object is created by this statement.

Q14. What does `intern()` do and where is the string pool?

`intern()` returns the canonical pooled instance for the string's content, adding it if absent. The pool is a native hash table (`StringTable`) whose entries point at `String` objects in the **heap** (since Java 7; earlier it was in PermGen and could cause `OutOfMemoryError: PermGen space`). Pooled strings that are unreachable are garbage-collected.

Q15. Predict the output.

```
String a = "hello";
String b = "hel" + "lo";
String c = "hel";
String d = c + "lo";
final String e = "hel";
String f = e + "lo";
System.out.println(a == b);           // true  – constant folded at compile time
System.out.println(a == d);           // false – runtime concat creates a new String
System.out.println(a == f);           // true  – e is a constant variable
System.out.println(a == d.intern());  // true
```

Q16. How is string concatenation compiled since Java 9?

To a single `invokedynamic` instruction bootstrapped by `StringConcatFactory` (JEP 280). On first call the JVM links a method handle that computes the final length and coder, allocates once, and copies parts. It is faster and produces less garbage than the old `StringBuilder` chain, and the JDK can improve the strategy without you recompiling. It does not fix concatenation in loops, which still creates a new string per iteration.

Q17. `StringBuilder` vs `StringBuffer`?

Same API; `StringBuffer` synchronises every method, `StringBuilder` does not. Use `StringBuilder` — builders are almost always method-local, so locking is pure overhead. `StringBuffer` remains only for legacy APIs.

Q18. What are compact strings?

Since Java 9, `String` stores a `byte[]` plus a `coder` flag: Latin-1 strings use 1 byte per char, others UTF-16 (2 bytes). For typical business data this halves string memory, and strings are often 25–40% of a Java heap. Methods check the coder and use specialised intrinsics.

Q19. Why prefer `char[]` over `String` for passwords?

A `char[]` can be overwritten (`Arrays.fill(pw, '\0')`) as soon as you are done; a `String` is immutable, may be interned, and stays readable in a heap dump until garbage-collected. It also won't accidentally appear in logs via `toString()`. `Console.readPassword()` and `KeyStore` APIs use `char[]` for this reason.

Q20. `==` vs `equals()` vs `equalsIgnoreCase` vs `compareTo` for strings?

`==` compares references; `equals` compares content; `equalsIgnoreCase` compares content case-insensitively per char; `compareTo` is lexicographic by UTF-16 code unit and returns negative/zero/positive. For user-facing sorting in a locale, use `Collator`. Put the literal first (`"USD".equals(code)`) or use `Objects.equals` to be null-safe.

Q21. Is Java pass-by-value or pass-by-reference?

Always pass-by-value. For objects, the value copied is the reference, so the method can mutate the shared object but cannot make the caller's variable point to a different object. A `swap(a, b)` method therefore cannot swap the caller's variables.

Q22. What does this print?

```
static void change(StringBuilder sb, String s) { sb.append("!"); s = s + "!"; sb = new StringBuilder("new"); }
var sb = new StringBuilder("hi"); var s = "hi";
change(sb, s);
System.out.println(sb + " " + s);  // "hi! hi"
```

The append mutated the shared builder before the parameter was reassigned; the String reassignment and builder reassignment only changed local copies.

Q23. List the methods of `Object`.

`equals`, `hashCode`, `toString`, `getClass`, `clone` (protected), `wait()` (three overloads), `notify`, `notifyAll`, and the deprecated `finalize`. `getClass`, `wait`, `notify`, `notifyAll` are `final`.

Q24. Why are `wait` / `notify` in `Object` and not in `Thread`?

Because every object can be a monitor: the condition waited on belongs to the shared object, not to the thread. A thread calls `wait()` on the object whose monitor it holds; it releases that monitor and parks in the object's wait set. Today prefer `java.util.concurrent` (locks with `Condition`, queues, latches).

Q25. When is a class initialised? Does referencing a `static final` constant trigger it?

On first active use: instance creation, static method call, assignment or read of a non-constant static field, reflection/ `Class.forName`, subclass initialisation, or being the main class. Reading a compile-time constant (`static final` primitive or String with a constant initialiser) does **not** initialise the class because the value is inlined. `Class.forName(name, false, loader)` loads without initialising; `ClassName.class` also does not initialise.

Q26. What is the order of static blocks, instance blocks and constructors?

Static field initialisers and static blocks run once, in textual order, when the class is initialised (parent class first). For each `new`: the parent constructor chain runs first; then the class's instance field initialisers and instance blocks in textual order; then the constructor body.

Q27. Can a static method access instance variables? Can it be overridden?

It cannot access instance members directly (there is no `this`); it needs an object reference. It cannot be overridden, only hidden — the call is bound at compile time to the declared type.

Q28. What does `final` mean on a variable, method, class and parameter?

Variable: assigned exactly once (a reference stays pointed at the same object, but the object can still change). Method: cannot be overridden. Class: cannot be subclassed. Parameter: cannot be reassigned within the method. A `final` field also gets JMM safe-publication guarantees.

Q29. What is "effectively final" and why do lambdas require it?

A local variable that is never reassigned after initialisation. Lambdas and inner classes capture the *value* of locals (copied into the lambda object), not the variable itself; allowing later reassignment would create two diverging copies and invite data races. To accumulate inside a lambda, use an `AtomicInteger` or, better, a stream reduction.

Q30. Checked vs unchecked exceptions — which do you use and why?

Checked: subclasses of `Exception` except `RuntimeException`; the compiler forces handling. Unchecked: `RuntimeException` / `Error`. I use unchecked for domain and infrastructure errors because callers usually cannot recover locally, checked exceptions leak implementation details through signatures, and they do not work with lambdas/streams. Checked exceptions are reasonable for low-level APIs where the immediate caller can genuinely recover. Spring's `@Transactional` rolls back on unchecked exceptions by default, but **not** on checked ones unless you set `rollbackFor`.

Q31. `Error` vs `Exception` — should you catch `OutOfMemoryError`?

`Error` signals a serious problem the application should not try to handle (OOM, `StackOverflowError`, `LinkageError`). Catching OOM to log and exit can be acceptable, but continuing is dangerous because invariants may be broken mid-operation. In production set `-XX:+HeapDumpOnOutOfMemoryError` and `-XX:+ExitOnOutOfMemoryError` so the container restarts cleanly and you get a dump to analyse.

Q32. What does this method return?

```
static int test() {
    try { throw new RuntimeException("boom"); }
    catch (RuntimeException e) { return 1; }
    finally { return 2; }
}
```

`2`. The `finally` return replaces the catch's return. If the catch had thrown instead, the exception would be discarded too. Compilers warn about this ("finally block does not complete normally").

Q33. Does `finally` always execute?

It runs after `try` / `catch` on normal completion, `return`, `break`, `continue` and exceptions. It does not run if the JVM exits (`System.exit`, `Runtime.halt`), the process is killed (`kill -9`, container OOM-kill), the JVM crashes, or the `try` never finishes (infinite loop, deadlock). For a daemon thread, `finally` may not run when the JVM shuts down.

Q34. What are suppressed exceptions?

Exceptions thrown while closing resources after a primary exception. try-with-resources attaches them with `Throwable.addSuppressed`, so the primary cause is preserved and the close failures are still visible via `getSuppressed()` and in the printed stack trace ("Suppressed: ..."). You can call `addSuppressed` yourself when cleaning up multiple resources manually.

Q35. In what order are resources closed in try-with-resources, and what if a constructor fails?

Reverse order of declaration. If the second resource's initialisation throws, the first (already opened) resource is still closed. Resources whose initialiser returned `null` are skipped. Watch wrapping: `new BufferedReader(new FileReader(f))` declared as one resource leaks the `FileReader` if the `BufferedReader` constructor throws — declare each as its own resource.

Q36. How do you write a good custom exception?

Extend `RuntimeException` (or a domain base exception), provide constructors with message and cause, carry structured data (error code, entity id, limit) as fields, keep it immutable, and document when it is thrown. Name it after the problem (`PlanLimitExceededException`, `RunAlreadyStartedException`), not the layer. Map it centrally to an HTTP status in a `@RestControllerAdvice`.

Q37. What is exception chaining and translation?

Translation: catching a low-level exception and throwing one meaningful at the current abstraction level (`SftpException` → `IngestFailedException`). Chaining: passing the original as the `cause` so the full trail ("Caused by: ...") survives. Losing the cause (`throw new X(e.getMessage())`) is one of the most common production debugging blockers.

Q38. Why are exceptions expensive, and how do you make them cheaper?

The cost is mostly `fillInStackTrace()` walking the thread's stack at construction time, plus the stack-trace array. Use exceptions for exceptional cases; for expected validation failures in hot loops return a result type. If you must throw frequently, use the 4-argument constructor with `writableStackTrace=false` or reuse a preallocated instance (only for exceptions with no varying state).

Q39. Why does a production log show `NullPointerException` with no stack trace?

HotSpot's `OmitStackTraceInFastThrow` optimisation: after an implicit exception is thrown many times from the same compiled code, the JIT throws a preallocated instance without a stack trace. Search earlier in the logs for the first occurrences (which have traces) or restart with `-XX:-OmitStackTraceInFastThrow`. Java 14+ helpful NPE messages (JEP 358) tell you exactly which reference was null (`because "run.getOwner()" is null`) when the trace is present.

Q40. Can you catch multiple exceptions in one block? Any restriction?

Yes, `catch (IOException | SQLException e)`. The types must not be subclasses of each other (compile error), and `e` is implicitly `final`. Java 7 also added precise rethrow: rethrowing a caught `Exception e` from a block whose try only throws `IOException` lets the method declare just `throws IOException`.

Q41. Can a `try` block exist without `catch`?

Yes, with `finally` (`try { } finally { }`) or as try-with-resources (`try (r) { }`). A plain `try { }` with neither is a compile error.

Q42. What happens if an exception is thrown in a static initialiser?

Class initialisation fails with `ExceptionInInitializerError` (wrapping the cause). Any later attempt to use the class throws `NoClassDefFoundError: Could not initialize class X` — which confuses people because the class file exists. Look for the first `ExceptionInInitializerError` in the logs.

Q43. `ClassNotFoundException` vs `NoClassDefFoundError`?

`ClassNotFoundException` is a checked exception from explicit loading by name (`Class.forName`, `loadClass`) when the class is not on the classpath. `NoClassDefFoundError` is an `Error` when a class that was present at compile time cannot be found or initialised at runtime (missing jar, failed static init). In Spring Boot it often means a dependency version conflict — inspect with `mvn dependency:tree`.

Q44. What are the rules and pitfalls of `var`?

Only for local variables with an initialiser (plus loop/try-with-resources variables and lambda parameters). The type is inferred at compile time and fixed. Pitfalls: `var list = new ArrayList<>()` becomes `ArrayList<Object>`; `var x = 1` is `int` when you wanted `long`; it hides types when the right side is a method call, hurting readability. Use it when the type is obvious from the right side.

Q45. Predict: `int i = 0; i = i++ + i++; System.out.println(i);`

`1`. The first `i++` yields 0 (i becomes 1), the second yields 1 (i becomes 2), and the sum 0 + 1 = 1 is assigned to i, overwriting 2.

Q46. Predict: `10 == 10.0, 'a' + 1, (char)('a' + 1).`

`true` (int promoted to double), `98` (char promoted to int), `b`.

Q47. Why does `0.1 + 0.2 != 0.3`, and how do you compare doubles?

0.1 and 0.2 cannot be represented exactly in binary floating point; their sum is `0.30000000000000004`. Compare with an epsilon (`Math.abs(a - b) < 1e-9`) for scientific values, or use `BigDecimal` for money. `Double.compare` handles `NaN` and `-0.0` consistently, unlike `==`.

Q48. Why not use `double` for money? How do you use `BigDecimal` correctly?

Binary floating point cannot represent most decimal fractions, so totals drift and rounding is uncontrolled. Use `BigDecimal` built from `String` / `valueOf`, compare with `compareTo`, specify scale and `RoundingMode` (typically `HALF_EVEN`) for division and final rounding, reassign results (it is immutable), and store as `NUMERIC` in the DB. Alternatively store minor units in a `long` (cents) when all currencies' scales are known and no fractional rates apply.

Q49. Why is `new BigDecimal("2.0").equals(new BigDecimal("2.00"))` false?

`equals` compares value **and scale**; `compareTo` compares only the numeric value. This bites `HashSet` / `HashMap` keys and assertions in tests. Normalise with `stripTrailingZeros()` or use `compareTo`.

Q50. What is `RoundingMode.HALF_EVEN` and why do banks use it?

"Banker's rounding": round to the nearest neighbour, and if exactly halfway, round to the even digit (2.5 → 2, 3.5 → 4). Over many operations the rounding errors cancel out instead of biasing upward as `HALF_UP` does. Use what the business specifies — some tax rules mandate `HALF_UP`.

Q51. `Instant` vs `LocalDateTime` vs `ZonedDateTime` — when to use each?

`Instant` is an absolute moment in UTC — use it for timestamps you store and compare. `LocalDateTime` is a wall-clock date-time with no zone, not a moment — use it for user-entered schedules where the zone is stored separately. `ZonedDateTime`

combines local date-time with zone rules — use it for calendar arithmetic that must respect DST, then convert to `Instant` to store. In PostgreSQL map `Instant` / `OffsetDateTime` to `timestamptz`.

Q52. Why was `SimpleDateFormat` a production problem?

It is mutable and not thread-safe: it stores the `Calendar` being parsed in a field. A `static final SimpleDateFormat` shared by request threads produces wrong dates or `NumberFormatException` under concurrency — intermittent and hard to reproduce. Fixes then were `ThreadLocal` or per-call instances; today use the immutable, thread-safe `DateTimeFormatter`.

Q53. How do you make time-dependent code testable?

Inject a `java.time.Clock` (a Spring bean `Clock.systemUTC()`) and call `Instant.now(clock)` / `LocalDate.now(clock)`. Tests use `Clock.fixed(...)` or a mutable test clock to simulate month-end, expiry, or DST transitions without sleeping.

Q54. `Duration` vs `Period`?

`Duration` is an exact amount of time in seconds and nanoseconds (timeouts, SLAs). `Period` is a calendar amount in years, months and days (subscription length, "due in 30 days"). Adding `Period.ofDays(1)` to a `ZonedDateTime` keeps the same wall-clock time across a DST change; adding `Duration.ofHours(24)` keeps the exact elapsed time.

Q55. How do you handle time zones in a multi-tenant SaaS?

Store all instants in UTC (`timestamptz`), store each tenant's `ZoneId` as an IANA name (`Europe/London`, never a fixed offset), convert at the edges (API formatting, reports, scheduling), and compute business dates such as month-end in the tenant's zone. Keep the JVM default zone at UTC (`-Duser.timezone=UTC`) so nothing silently depends on the server's location.

Q56. Describe a real bug caused by a core-language detail.

Tell one that really happened to you, in the order symptom, mechanism, fix, prevention. The shape interviewers want: "Totals built from `33.37 + 33.37 + 33.36` failed to equal a ledger's `100.10`, because the amounts had passed through `double`. We parsed from the original string into `BigDecimal`, compared with `compareTo` against a tolerance, and made it a rule that amounts never pass through `double`, including in JSON (Jackson `USE_BIG_DECIMAL_FOR_FLOATS`)." Other good candidates are `==` on `Long` ids that worked in tests only because the values were under 128, and a shared `SimpleDateFormat`.

Q57. What is the `main` method signature and can you overload it or omit `static`?

`public static void main(String[] args)` (varargs allowed). You may overload `main`, but only that signature is the launcher's entry point. In the Java 21 preview (final in Java 25), instance `void main()` methods are allowed for simple programs; production apps still use the classic form.

Q58. What does `System.exit(0)` do inside a Spring Boot app, and is it safe?

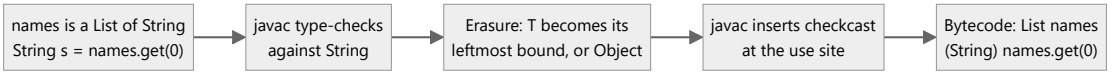
It starts JVM shutdown: shutdown hooks run (Spring's hook closes the context, calling `@PreDestroy` and graceful shutdown), then the JVM halts; `finally` blocks of the calling thread do not run. In CLI-style apps prefer `SpringApplication.exit(context, () -> code)` and then `System.exit`; in services let the orchestrator stop the process with SIGTERM so graceful shutdown drains in-flight requests.

3. Generics, Enums, Annotations, Reflection & Nested Classes

This chapter covers the language features that frameworks are built from. Spring's `@Transactional` is an annotation, read by reflection, applied by a generated proxy. Jackson binds JSON through generic type tokens because erasure removes the type arguments. Interviewers use these topics to separate people who use frameworks from people who can debug them. For each feature, know what the compiler does, what survives into the class file, and what happens at runtime.

3.1 Generics and type erasure

Generics give compile-time type safety without casts. They are implemented by **erasure**: the compiler checks the types, then removes them. The JVM sees no type arguments in ordinary objects, and `List<String>` and `List<Integer>` share the single class `List`.



The erasure rules are:

1. Replace each type parameter with its leftmost bound: `T` becomes `Object`, and `T extends Comparable<T>` becomes `Comparable`.
2. Insert casts where a value of the erased type is used as a more specific type.
3. Generate **bridge methods** to preserve overriding after erasure.

```
// Source
public class Box<T extends Comparable<T>> {
    private T value;
    public T get() { return value; }
}
String s = new Box<String>().get();

// After erasure, conceptually
public class Box {
    private Comparable value;
    public Comparable get() { return value; }
}
String s = (String) new Box().get();    // the cast is added here
```

Why erasure? Migration compatibility. Java 5 had to run existing non-generic code and let it interoperate with generic libraries, which required identical class files. The cost is a list of things generics cannot do:

Not allowed	Why	Workaround
`new T()`	`T` unknown at runtime	Pass a `Supplier<T>` or `Class<T>`
`new T[10]`, `new List<String>[10]`	Arrays check element types at runtime and need a reifiable type	`(T[]) new Object[n]`, or use a `List`
`x instanceof List<String>`	Type argument is erased	`x instanceof List<?>`; Java 16+ allows it when the compiler can prove the cast
`List<int>`	Type arguments must be references	`List<Integer>`, `IntStream`; Valhalla aims to change this
`static T field`	Static belongs to the erased class	Generic static method
`catch (T e)`, `class X<T> extends Exception`	Exception dispatch happens at runtime	Pass a class token
Overloads `f(List<String>)` and `f(List<Integer>)`	Same erasure: "name clash"	Rename the methods

What does survive erasure? Generic *declarations* are kept in the `Signature` attribute of the class file: a field's declared type, a method's signature, and a class's generic superclass. Erased are the type arguments of an object instance. That gap is what makes the **super type token** work:

```
// new X<...>(){} creates an anonymous subclass whose generic superclass is recorded
List<ReconDraftSpec> specs = mapper.readValue(json, new TypeReference<List<ReconDraftSpec>>() {});

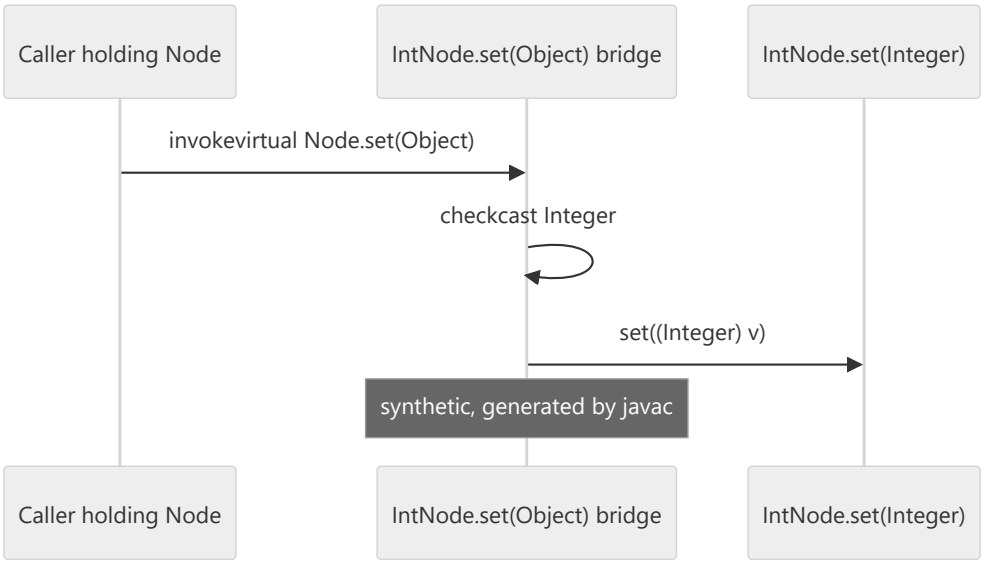
Type t = new TypeReference<List<ReconDraftSpec>>() {}.getType();  // List<ReconDraftSpec>, intact
```

Spring's `ParameterizedTypeReference` and `ResolvableType` work the same way, which is how Spring can autowire `Repository<Order>` differently from `Repository<User>`.

Bridge methods

```
class Node<T> { void set(T v) { } }
class IntNode extends Node<Integer> {
    @Override void set(Integer v) { }
}
```

After erasure, `Node` has `set(Object)` while `IntNode` has `set(Integer)`, which is an overload and not an override, so polymorphism would break. The compiler adds a synthetic bridge:



Bridges also implement covariant return types. You see them as extra frames in stack traces and as `Method.isBridge()` in reflection, and passing a `String` through a raw `Node` reference fails with `ClassCastException` at the bridge's cast.

Bounds, wildcards and PECS

Generics are **invariant**: `List<Integer>` is not a `List<Number>`. If it were, you could add a `Double` to a list of integers. Arrays are covariant, and pay for it at runtime:

```
Object[] objs = new String[1];
objs[0] = 1;                    // compiles, throws ArrayStoreException at runtime

List<Object> list = new ArrayList<String>();   // compile error: the safer design
```

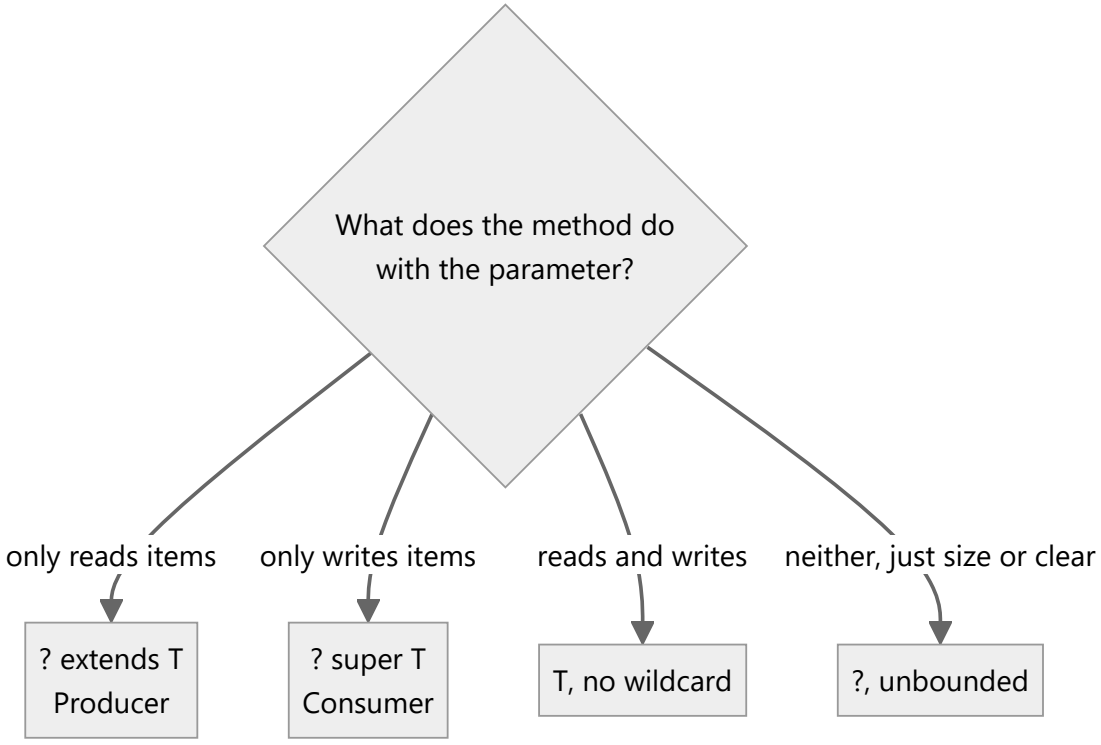
Syntax	Meaning	Read as	Write
`List<Number>`	Exactly Number	`Number`	`Number` and subtypes
`List<? extends Number>`	Some unknown subtype of Number	`Number`	Nothing except `null`
`List<? super Integer>`	Some unknown supertype of Integer	`Object`	`Integer` and subtypes
`List<?>`	Some unknown type	`Object`	Nothing except `null`
`List` (raw)	Legacy, generics switched off	`Object`	Anything, unchecked

PECS, Producer Extends, Consumer Super: if a parameter *produces* values you read, use `extends`; if it *consumes* values you write, use `super`.

```
public static <T> void copy(List<? super T> dest, List<? extends T> src) {
    for (T item : src) dest.add(item);       // src produces, dest consumes
}
List<Number> nums = new ArrayList<>();
copy(nums, List.of(1, 2));                   // T inferred, works
copy(nums, List.of(1.5));

public static <T extends Comparable<? super T>> T max(Collection<? extends T> c) { ... }
```

`Comparable<? super T>` in that last signature lets `max` work when the `compareTo` is defined on a parent class (`java.sql.Timestamp` inherits `Comparable<Date>`).



Generic method syntax: the type parameter precedes the return type, `public static <T> List<T> listOf(T a)`, and the compiler infers it from arguments and target type. Explicit form: `Util.<String>listOf("a")`. Multiple bounds use `&`: `<T extends Number & Comparable<T>>`, with the class first; `T` erases to the first bound. Recursive bounds such as `Enum<E extends Enum<E>>` let the base class refer to its own subclass type.

Heap pollution and reifiable types

A type is **reifiable** when its full type is available at runtime: primitives, non-generic classes, raw types, unbounded wildcards (`List<?>`), and arrays of those. `List<String>` is not reifiable. **Heap pollution** happens when a variable of a parameterised type refers to an object that is not of that type, and it surfaces later as a `ClassCastException` at a line with no visible cast.

```
List raw = new ArrayList<String>();
raw.add(42);                          // unchecked warning
List<String> names = raw;             // heap pollution
String s = names.get(0);              // ClassCastException here, far from the cause
```

Generic varargs are the classic source. `void f(List<String>... lists)` creates a `List[]` at runtime, so the method can write the wrong thing into it. `@SafeVarargs` is your promise that the method does not store into the array or expose it. `Collections.checkedList` adds runtime checks when you need to catch pollution at the point of insertion.

3.2 Enums

An enum compiles to a `final` class extending `java.lang.Enum`, with one `public static final` field per constant, created in the static initialiser.

```
enum Status { QUEUED, RUNNING }
// compiles, conceptually, to
final class Status extends Enum<Status> {
    public static final Status QUEUED = new Status("QUEUED", 0);
    public static final Status RUNNING = new Status("RUNNING", 1);
    private static final Status[] $VALUES = { QUEUED, RUNNING };
    public static Status[] values() { return $VALUES.clone(); }
    public static Status valueOf(String n) { return Enum.valueOf(Status.class, n); }
```

```
    private Status(String name, int ordinal) { super(name, ordinal); }
}
```

Consequences follow directly from that shape:

- Constants are singletons, so `==` is safe, null-safe and fast. Class initialisation makes creation thread-safe.
- `values()` clones an array on each call, so cache it on a hot path.
- Enums cannot extend a class (they already extend `Enum`) but can implement interfaces.
- A constant with a body compiles to an anonymous subclass (`Status$1`). In that case `getClass()` differs from `getDeclaringClass()`.
- A non-constant static field cannot be read from an enum constructor, since constants are built before other statics are initialised.
- Serialization writes only the constant's name and resolves it back through `valueOf`, and reflection refuses to instantiate enums, which makes them the safest singleton.

```
public enum Registry {                  // enum singleton
    INSTANCE;
    private final Map<String, Plugin> plugins = new ConcurrentHashMap<>();
    public void add(String k, Plugin p) { plugins.put(k, p); }
}
```

Strategy enum, behaviour per constant:

```
public enum Rounding {
    NONE { @Override BigDecimal apply(BigDecimal v) { return v; } },
    CENTS { @Override BigDecimal apply(BigDecimal v) { return v.setScale(2, RoundingMode.HALF_EVEN); } };
    abstract BigDecimal apply(BigDecimal v);
}
```

Switch on enums. The compiler builds a synthetic `$SwitchMap$...` int array in a helper class, indexed by `ordinal()`, so reordering constants does not break already compiled switches. A Java 21 switch *expression* over an enum must be exhaustive, and the compiler adds a default that throws `MatchException` (`IncompatibleClassChangeError` in code compiled for releases before 21) if a new constant appears at runtime without recompiling.

```
String label = switch (status) {
    case QUEUED -> "Waiting";
    case RUNNING -> "In progress";
};  // no default: adding a constant becomes a compile error here
```

`EnumMap` and `EnumSet` are the right collections for enum keys, and are much faster and smaller than the hash versions:

```
EnumMap<Status, V>                EnumSet<Status> (RegularEnumSet, up to 64 constants)
keyUniverse = Status.values()     long elements = 0b...0101
vals[]  index = ordinal           bit n set = constant with ordinal n present
[ v0 | null | v2 | ... ]          add = elements |= 1L << ordinal
No hashing, no entry objects      Over 64 constants: JumboEnumSet with long[]
```

Trap: never persist `ordinal()`. With JPA, `@Enumerated` defaults to `ORDINAL`, so inserting a new constant in the middle silently re-maps existing rows. Use `@Enumerated(EnumType.STRING)`, or a stable code column with an `AttributeConverter`.

RefinePoint: `MatchStrategy` is a plain enum (`EXACT`, `NUMERIC_TOLERANCE`, `JARO_WINKLER`, `LEVENSHTEIN`, ...), and `ReconDefinition` declares `Cardinality`, `FieldRole` and `ControlType` as nested enums. A process's definition is stored as JSON in the `definition_json` `jsonb` column and read back with Jackson, so renaming a constant would break saved reconciliations. Constant names are part of the persisted format.

3.3 Annotations

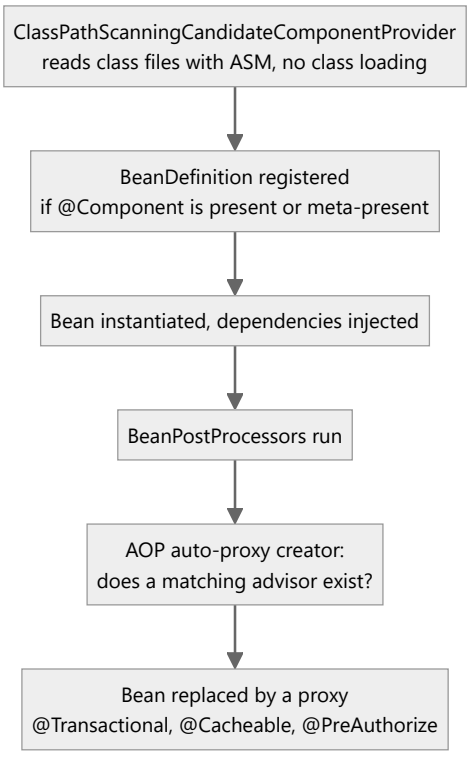
An annotation is an interface (`@interface` , which extends `java.lang.annotation.Annotation`) with metadata stored in the class file. It does nothing by itself: something must read it, which is either a compile-time **annotation processor** or runtime reflection.

Meta-annotations:

Meta-annotation	Purpose
`@Retention`	`SOURCE` (dropped by javac, `@Override` , Lombok), `CLASS` (in class file, not at runtime; the default), `RUNTIME` (visible to reflection)
`@Target`	Where it can go: `TYPE` , `METHOD` , `FIELD` , `PARAMETER` , `TYPE_USE` , `RECORD_COMPONENT` , and others
`@Inherited`	Subclasses inherit it, for **class-level** annotations only, not from interfaces or methods
`@Documented`	Included in Javadoc
`@Repeatable`	Can appear several times, wrapped in a container annotation

Attribute types are limited to primitives, `String` , `Class` , enums, other annotations, and one-dimensional arrays of those, and values must be compile-time constants. An attribute named `value` can be written without its name.

How Spring processes annotations



Plain Java reflection does not treat annotations as composable, but Spring's `MergedAnnotations` / `AnnotatedElementUtils` follow **meta-annotations**. `@Service` works because it is itself annotated with `@Component` , and `@AliasFor` lets a composed annotation pass attributes down.

```
// Real, from ryfn-platform: a composed security annotation
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@PreAuthorize("@orgAuth.isAdmin()")
public @interface CanManageOrganization { }
```

> **RefinePoint:** `@CanManageOrganization` on a controller method replaces a repeated SpEL string. Spring Security's method-security advisor finds the meta-present `@PreAuthorize` and evaluates it against the `orgAuth` bean, so the rule has one name and one definition.

Writing a custom annotation and aspect

```
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface Audited { String action(); }

@Aspect @Component
class AuditedAspect {
    private final AuditWriter writer;
    AuditedAspect(AuditWriter writer) { this.writer = writer; }

    @Around("@annotation(audited)")
    Object around(ProceedingJoinPoint pjp, Audited audited) throws Throwable {
        Object result = pjp.proceed();
        writer.record(audited.action(), pjp.getArgs());
        return result;
    }
}
```

It must have `RUNTIME` retention, the annotated method must be called through the proxy (self-invocation skips the aspect), and private methods are not advised.

Compile-time annotation processors

A processor extends `AbstractProcessor`, is registered through `META-INF/services/javax.annotation.processing.Processor`, and runs inside javac in rounds. It sees source elements and can generate new files or report errors, but officially cannot modify existing classes. Examples: MapStruct, Hibernate JPA Metamodel, Spring Boot's configuration processor. Lombok is the exception, as it patches javac's internal AST.

```
@SupportedAnnotationTypes("com.example.Audited")
@SupportedSourceVersion(SourceVersion.RELEASE_21)
public class AuditedProcessor extends AbstractProcessor {
    @Override public boolean process(Set<? extends TypeElement> types, RoundEnvironment env) {
        for (Element e : env.getElementsAnnotatedWith(Audited.class)) {
            if (e.getModifiers().contains(Modifier.PRIVATE)) {
                processingEnv.getMessager().printMessage(Diagnostic.Kind.ERROR,
                    "@Audited has no effect on private methods", e);
            }
        }
        return true;
    }
}
```

3.4 Reflection

Reflection inspects and invokes classes at runtime through `Class`, `Method`, `Field`, and `Constructor`.

```
Class<?> c = Class.forName("com.acme.refinepoint.AcmeFunctions");   // loads and initialises
Object o = c.getDeclaredConstructor().newInstance();
Method m = c.getMethod("functions");                // public, including inherited
Method p = c.getDeclaredMethod("helper");           // any access, this class only
p.setAccessible(true);                              // bypass the access check
Object r = m.invoke(o);                             // wraps failures in InvocationTargetException
```

`setAccessible` and modules. Since Java 16 and finalised by Java 17 (JEP 403), strong encapsulation means `setAccessible(true)` on JDK internals fails with `InaccessibleObjectException` unless the package is opened with `--add-opens java.base/java.lang=ALL-UNNAMED`. This was the classic failure when moving old libraries to Java 17.

Performance. Lookup (`getMethod`) is expensive because it searches and copies metadata, so cache `Method` objects. Invocation used to start in a native accessor and switch to generated bytecode after 15 calls ("inflation"). Since Java 18 (JEP 416), core reflection is reimplemented on method handles. Costs remaining: argument boxing, an `Object[]` allocation, and access checks unless accessible, plus limits on JIT inlining when the `Method` is not a constant.

`MethodHandle` and `VarHandle`

```
MethodHandles.Lookup lookup = MethodHandles.lookup();
MethodHandle len = lookup.findVirtual(String.class, "length", MethodType.methodType(int.class));
int n = (int) len.invokeExact("hello");      // signature-polymorphic, no boxing

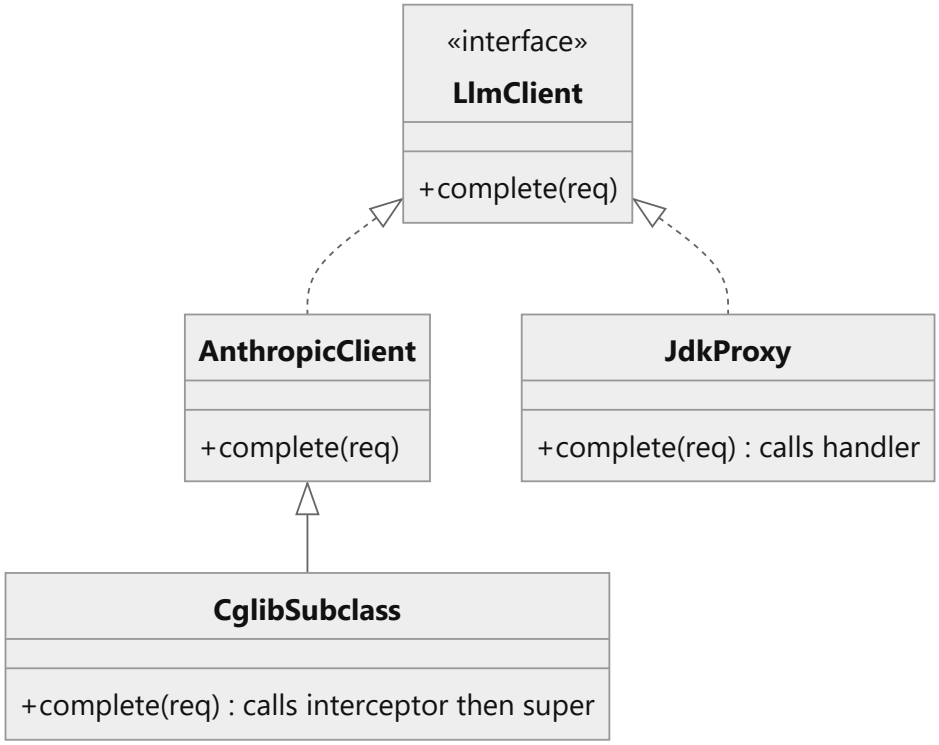
class Counter {
    private volatile int value;
    private static final VarHandle VALUE;
    static {
        try { VALUE = MethodHandles.lookup().findVarHandle(Counter.class, "value", int.class); }
        catch (ReflectiveOperationException e) { throw new ExceptionInInitializerError(e); }
    }
    void increment() { VALUE.getAndAdd(this, 1); }   // atomic, replaces sun.misc.Unsafe
}
```

	Reflection	`MethodHandle`	`VarHandle`
Access check	At each call, unless accessible	Once, at lookup	Once, at lookup
Boxing	Yes	No with `invokeExact`	No
JIT inlining	Limited	Full when `static final`	Full when `static final`
Use for	Frameworks, dynamic discovery	Fast dynamic calls, `invokedynamic` targets	Atomic field and array access, memory-order modes

Dynamic proxies

```
LlmClient proxy = (LlmClient) Proxy.newProxyInstance(
    LlmClient.class.getClassLoader(),
    new Class<?>[] { LlmClient.class },
    (self, method, args) -> {
        long start = System.nanoTime();
        try { return method.invoke(target, args); }
        catch (InvocationTargetException e) { throw e.getCause(); }  // unwrap, or callers see the wrapper
        finally { metrics.record(method.getName(), System.nanoTime() - start); }
    });
```

The JDK generates a class named like `$Proxy42` at runtime, implementing the interfaces, with each method forwarding to your `InvocationHandler`. CGLIB and ByteBuddy instead generate a **subclass** of the target.

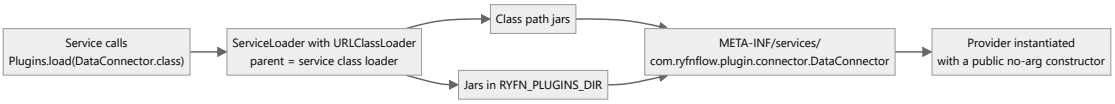


	JDK Proxy	CGLIB / ByteBuddy
Needs	At least one interface	A non-final class
How	Implements interfaces	Subclasses and overrides
Cannot intercept	Methods outside the interfaces	`final`, `private`, `static` methods
Self-invocation	Not intercepted	Not intercepted
Injectable as concrete class	No, `ClassCastException` or injection error	Yes
Used by	Spring when `proxyTargetClass=false`, Spring Data repositories	Spring Boot default (`proxyTargetClass=true`), Hibernate proxies (ByteBuddy), Mockito

Spring uses **Objenesis** to create a CGLIB proxy instance without calling the target's constructor, which is why proxy fields stay null and why reading a field directly through a proxy reference gives `null`.

3.5 `ServiceLoader` and the SPI pattern

An **API** is what callers use; a **Service Provider Interface** is what implementers provide. `ServiceLoader` finds implementations by reading `META-INF/services/<interface name>` from every jar visible to a class loader (or `provides ... with` in a module), and instantiates them lazily. JDBC drivers, `javax.annotation.processing.Processor` and logging bindings all work this way.




```
// Trimmed from ryfn-plugin-api
public static <T> List<Found<T>> load(Class<T> extensionPoint) {
    List<Found<T>> found = new ArrayList<>();
    var it = ServiceLoader.load(extensionPoint, loader()).stream().iterator();
    while (true) {
        try {
            if (!it.hasNext()) break;
```

```
                ServiceLoader.Provider<T> p = it.next();
                found.add(new Found<>(p.get(), source(p.type())));
            } catch (ServiceConfigurationError | RuntimeException | LinkageError e) {
                LOG.log(ERROR, "A plugin could not be loaded and is left out: " + e.getMessage(), e);
            }
        }
        return found;
    }
```

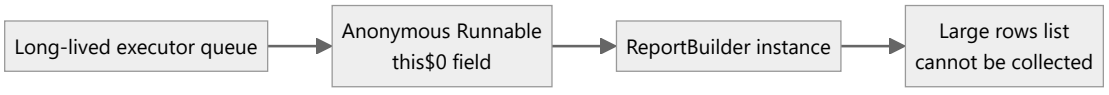
> **RefinePoint:** `ryfn-plugin-api` defines `DataConnector` and `FunctionPlugin`. Dropping a jar into the plugins folder and restarting adds connectors, or rule-language functions that the rule builder, checker and DuckDB SQL all pick up. Two design choices matter: a broken plugin is logged and left out rather than stopping the service, which is why `ServiceConfigurationError` and `LinkageError` are caught; and the plugin class loader's parent is the service's, so plugins share the API classes. Plugins run with the service's permissions, so only server operators install them.

`ServiceLoader` vs Spring's auto-configuration: `ServiceLoader` instantiates with a no-arg constructor and knows nothing about dependency injection or conditions. Spring Boot's `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` lists configuration classes that go through full bean processing with `@ConditionalOn...`. Same discovery idea, more machinery.

3.6 Nested classes and memory leaks

Kind	Declared	Holds outer instance	Can access	Typical use
Static nested	`static class` in a class	No	Outer statics; private members via an instance	Builders, `Map.Entry`, DTO parts
Inner	Non-static member	**Yes**, `this$0`	All outer members	Iterators bound to a collection
Local	Inside a method	Yes, in an instance method	Plus effectively final locals	Rare
Anonymous	Expression `new X() { }`	Yes, in an instance method	Plus effectively final locals	Largely replaced by lambdas

```
class Outer {
    class Inner { }                    // compiled to Outer$Inner with a constructor taking Outer
    static class Nested { }
}
Outer.Inner i = new Outer().new Inner();
Outer.Nested n = new Outer.Nested();
```



The leak: an inner class instance keeps its outer instance alive, even when it never uses it. Hand one to something long-lived (an executor, a cache, a listener registry, a `ThreadLocal`) and the outer object cannot be collected. Since JDK 18, javac omits `this$0` from an inner class that never uses it (unless the class is serialisable without a `serialVersionUID`), which removes some of these leaks but is not something to rely on. **Lambdas differ:** they capture `this` only if they actually use an instance member, and a non-capturing lambda is a shared singleton. Anonymous classes always create a new object, and `this` inside one means the anonymous object, whereas in a lambda it means the enclosing instance.

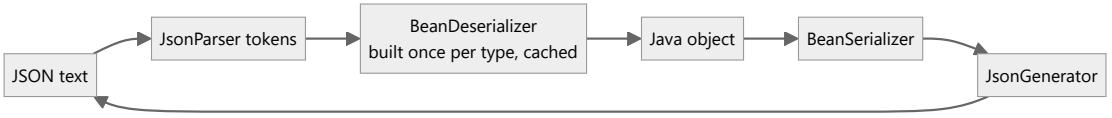
3.7 Java serialization

`Serializable` is a marker interface. `ObjectOutputStream` writes the class descriptor and field values, walking the object graph with reflection; `ObjectInputStream` creates the instance **without calling its constructor** (only the first non-serializable superclass's no-arg constructor runs), then sets the fields directly.

- `serialVersionUID`: a version identity. If it does not match on read you get `InvalidClassException`. If not declared, it is computed from the class structure, so any change such as a new method breaks old data. Declare `private static final long serialVersionUID = 1L;`.
- `transient`: excluded field, restored to its default. `static` fields are never serialised.
- **Custom hooks**: private `writeObject` / `readObject`, `writeReplace` / `readResolve` (singleton protection), and `readObjectNoData`.
- `Externalizable`: you write everything with `writeExternal` / `readExternal`. It needs a public no-arg constructor, which *is* called.

Why Java serialization is dangerous. Deserialising untrusted bytes can instantiate any class on the classpath with attacker-controlled fields, and "gadget chains" in libraries turn that into remote code execution (the Apache Commons Collections exploit of 2015). Controls: never deserialise untrusted data, use `ObjectInputFilter` (JEP 290, JEP 415) as a whitelist. Mark Reinhold, the Java platform's chief architect, has called serialization a horrible mistake. Other problems are fragile versioning, hidden constructors bypassing invariants, and large output. Use JSON or Protobuf instead. Records are the safer serialisable shape, since their deserialisation always goes through the canonical constructor.

JSON with Jackson



`ObjectMapper` is expensive to configure and thread-safe once configured, so share one (Spring Boot provides a bean and customises it through `Jackson2ObjectMapperBuilderCustomizer`). Since it builds serialisers using reflection and caches them, creating a new mapper per request throws away the cache and is a known performance bug. Key annotations: `@JsonProperty`, `@JsonIgnore`, `@JsonIgnoreProperties(ignoreUnknown = true)`, `@JsonInclude(NON_NULL)`, `@JsonCreator`, `@JsonTypeInfo` / `@JsonSubTypes`, `@JsonFormat`.

> **Trap:** `enableDefaultTyping()` and `@JsonTypeInfo(use = Id.CLASS)` on open types let incoming JSON name the class to instantiate, which reintroduces the gadget-chain problem. Use a closed list of logical names (`Id.NAME` + `@JsonSubTypes`), or configure a `PolymorphicTypeValidator`.

Records with Jackson. Since Jackson 2.12 records bind through the canonical constructor, with no setters or no-arg constructor needed, and validation in the compact constructor runs on every read.

```
@JsonIgnoreProperties(ignoreUnknown = true)   // tolerate fields you did not ask for
public record MatchRule(String name, List<String> keys) {
    public MatchRule { keys = keys == null ? List.of() : List.copyOf(keys); }
}
```

> **RefinePoint:** `ReconDefinition`, stored as `jsonb`, nests records such as `Control(name, type, side, severity, params)`, and the `LlmClient` types (`LlmRequest`, `Message`, `LlmResponse`) are records nested in the interface. `ReconDraftSpec`, the JSON the model returns when drafting a reconciliation, sets `@JsonIgnoreProperties(ignoreUnknown = true)`, because a model may add a field nobody asked for and that should not fail the whole draft.

3.8 Interview questions and answers

Q1. What is type erasure and why does Java use it?

The compiler checks generic types, then removes them: type parameters become their leftmost bound, and casts are inserted at use sites. Java chose it for migration compatibility, so generic and pre-Java-5 code could share the same class files and collection classes. The cost is that runtime code cannot ask for a type argument: no `new T()`, no `new T[]`, no `instanceof List<String>`.

Q2. Can you get the generic type at runtime?

Not from an instance, but yes from declarations. A field's declared type, a method's signature and a class's generic superclass are stored in the `Signature` attribute, readable with `getGenericType()`, `getGenericReturnType()` and `getGenericSuperclass()`. An anonymous subclass such as `new TypeReference<List<Foo>>() {}` records its superclass's arguments, which is how Jackson and Spring recover them.

Q3. Predict the output.

```
List<String> a = new ArrayList<>();
List<Integer> b = new ArrayList<>();
System.out.println(a.getClass() == b.getClass());
```

`true`. Both are `java.util.ArrayList` at runtime; there is only one class.

Q4. Why does this not compile: `void f(List<String> x)` and `void f(List<Integer> x)`?

Both erase to `f(List)`, so the class file would contain two methods with the same descriptor. The compiler reports "name clash: same erasure". Different return types do not help. Rename the methods, or give them different non-generic parameters.

Q5. What is a bridge method? When is it generated?

A synthetic method the compiler adds so that overriding still works after erasure, and for covariant return types. If `IntNode extends Node<Integer>` overrides `set(Integer)`, the compiler adds `set(Object)`, which casts and delegates. Reflection shows it with `isBridge()`, and frameworks that scan methods must skip it. Spring's `BridgeMethodResolver` exists to find the real method, so that annotations on it are seen.

Q6. Why can't you create a generic array?

Arrays check the element type on each store at runtime (`ArrayStoreException`), which needs a reified component type, and generics have erased theirs. If `new List<String>[2]` were legal, you could assign it to `Object[]`, store a `List<Integer>`, and nothing would catch it until a confusing `ClassCastException` later. Use `List<List<String>>`, or do what `ArrayList` does internally: hold `Object[]` and cast on read.

Q7. Are arrays covariant? Are generics? Which is safer?

Arrays are covariant, since `String[]` is an `Object[]`, and enforce safety at runtime with `ArrayStoreException`. Generics are invariant and enforce it at compile time, so the error surfaces earlier. Wildcards add controlled variance to generics when needed.

Q8. Explain PECS with a JDK example.

Producer Extends, Consumer Super. `Collections.copy(List<? super T> dest, List<? extends T> src)`: the source produces `T`s for reading, the destination consumes them. `Collection.addAll(Collection<? extends E>)` reads from its argument, and `removeIf(Predicate<? super E>)` feeds elements to its predicate, so a `Predicate<Object>` works on a `List<String>`.

Q9. What can you add to a `List<? extends Number>`?

Only `null`. The actual list could be `List<Integer>` or `List<Double>`, and the compiler cannot prove any concrete value fits. You can read elements as `Number`. Conversely, a `List<? super Integer>` accepts `Integer`s, but reads only give you `Object`.

Q10. `List<?>` vs `List<Object>` vs raw `List`?

`List<Object>` accepts any object but only a list declared with exactly `Object`. `List<?>` can refer to any list, is read-only in practice (except `null`), and stays type-checked. Raw `List` turns generics off for everything it touches, including unrelated generic methods on that type, and produces unchecked warnings. Use raw types only in class literals and `instanceof`.

Q11. What is wildcard capture?

The compiler gives a `?` a temporary name so that it can type-check a call. You cannot write `list.set(0, list.get(0))` on a `List<?>`, because the two `?`s are not known to be the same. A private generic helper `<T> void swapHelper(List<T> l)` captures it and compiles, and `Collections.swap` uses this technique.

Q12. What does `<T extends Comparable<? super T>>` mean?

`T` must be comparable to itself or to some supertype. Without `? super`, a type that inherits its `compareTo` from a parent, such as `java.sql.Timestamp` with `Comparable<Date>`, would be rejected. `Collections.sort` and `max` use this signature.

Q13. What is a recursive type bound? Why does `Enum` use one?

`E extends Enum<E>` means each enum is parameterised by itself, so `compareTo(E)` accepts only the same enum type, and `valueOf` returns it. Fluent builder hierarchies use the same pattern, `Builder<T extends Builder<T>>`, so that `self()` returns the subclass and chained calls keep their type.

Q14. What is heap pollution? Show one.

A variable of a parameterised type holds an object that is not that type. It usually comes from raw types or unchecked casts, and fails later with a `ClassCastException` on a line with no cast in sight.

```
static <T> T[] toArray(T... items) { return items; }
static <T> T[] twice(T a, T b) { return toArray(a, b); }   // creates Object[] at runtime
String[] s = twice("x", "y");                              // ClassCastException: Object[] to String[]
```

Q15. What are reifiable types? What does `@SafeVarargs` promise?

Reifiable types keep complete information at runtime: primitives, non-generic classes, raw types, `List<?>`, and arrays of those. `@SafeVarargs` asserts that a generic varargs method does not write to the array or let it escape, which silences the caller's warning. It is allowed only on `final`, `static`, `private` methods and constructors, because an override could break the promise.

Q16. Can a generic class have a static field of type `T`?

No. Statics belong to the single erased class, while `T` varies per use. `class Box<T> { static T x; }` does not compile. A static generic *method* declares its own parameter.

Q17. Can a generic class extend `Exception`? Can you catch `T`?

No to both. Exception handlers are matched at runtime by class, and the argument is gone by then. However, `<T extends Throwable> void f() throws T` is allowed, and it enables the "sneaky throw" trick to rethrow checked exceptions unchecked.

Q18. How would you implement a typesafe heterogeneous container?

Key the map by `Class<T>` and cast through `Class.cast`, so that one map holds many value types safely.

```
private final Map<Class<?>, Object> values = new HashMap<>();
<T> void put(Class<T> k, T v) { values.put(k, k.cast(v)); }
<T> T get(Class<T> k) { return k.cast(values.get(k)); }
```

Spring's `ApplicationContext.getBean(Class<T>)` has this shape. Note that a class token cannot express `List<String>`; a super type token can.

Q19. How does an enum compile? Can it extend a class or be subclassed?

To a `final` class extending `java.lang.Enum`, with `public static final` constants, a synthetic `$VALUES` array, `values()` and `valueOf`. It cannot extend another class but can implement interfaces. Constants with bodies are anonymous subclasses, and in that case the enum isn't marked `final`. Since Java 17 such enums are implicitly `sealed`.

Q20. Is `==` safe for enums?

Yes, and preferred. Each constant is a single instance per class loader, enforced by private constructors, class initialisation, the serialization mechanism and a reflection block. `==` is null-safe and checked at compile time: comparing different enum types is a compile error, whereas `equals` would just return `false`.

Q21. Why is an enum the best way to implement a singleton?

Three attacks that break hand-written singletons fail here. Reflection throws "Cannot reflectively create enum objects"; serialization writes only the name and resolves it back through `valueOf`; and class initialisation gives thread safety for free. The limits are that it cannot extend a class and initialises eagerly with the class. In a Spring application a singleton-scoped bean is the normal answer, since it can be replaced in tests.

Q22. `EnumMap` vs `HashMap`? How does `EnumSet` work?

`EnumMap` stores values in an array indexed by `ordinal()`: no hashing, no entry objects, iteration in declaration order. `EnumSet` holds a bit vector: one `long` up to 64 constants, `long[]` beyond. Operations like `addAll` and `containsAll` are single bitwise operations, so it replaces bit-flag ints with type safety. Both reject a `null` key or element.

Q23. Why should you never persist `ordinal()`?

It is a position, so inserting or reordering constants re-maps existing data with no error. JPA's `@Enumerated` default is `ORDINAL`, which makes this a default trap. Persist the name with `EnumType.STRING`, or a stable explicit code through an `AttributeConverter` if names may change.

Q24. How do you look up an enum from a value without a loop?

Build a static map in the enum, in a static field initialiser and not from the constructor, since constants are constructed before other statics are initialised.

```
private static final Map<String, IsoCurrency> BY_CODE =
    Arrays.stream(values()).collect(Collectors.toUnmodifiableMap(IsoCurrency::code, c -> c));
public static Optional<IsoCurrency> of(String code) { return Optional.ofNullable(BY_CODE.get(code)); }
```

`Enum.valueOf` throws `IllegalArgumentException` for an unknown name, which is awkward to use for parsing external input.

Q25. What happens if a new enum constant is added but a switch using it is not recompiled?

For an old-style switch statement with no `default`, no case runs and execution simply continues, and the mapping survives reordering thanks to the `$SwitchMap` array. For a switch expression without `default`, the compiler's synthetic default throws `MatchException` on Java 21 classes. This matters in shared libraries, and it is why services must be rebuilt together after an enum changes.

Q26. Can an enum have abstract methods, fields and constructors?

Yes, all three. Constructors are implicitly private, and each constant must implement any abstract method. Fields should be `final`, because constants are global singletons and mutable state on them is shared across the entire JVM.

Q27. Enum vs sealed interface with records?

An enum is a fixed set of *instances*, all with the same shape, so it fits statuses and strategies without per-instance data. A sealed hierarchy is a fixed set of *types*, each with its own data, so `sealed interface Node permits Literal, Binary, Call` fits an AST, and a pattern-matching switch handles it exhaustively.

Q28. What is an annotation really?

An interface extending `java.lang.annotation.Annotation`, with attribute values stored in the class file. At runtime `getAnnotation` returns a JDK dynamic proxy implementing that interface, which you can see by printing its class (`$Proxy12`). It has no behaviour; a processor, a framework or an aspect supplies the behaviour.

Q29. Explain each `@Retention` value with an example.

`SOURCE` : discarded by javac, as with `@Override` , `@SuppressWarnings` and Lombok's annotations. `CLASS` : in the class file but not loaded for reflection; it is the default, and bytecode tools use it. `RUNTIME` : available to reflection, as with `@Transactional` and `@Entity` . Forgetting `RUNTIME` is the classic reason a custom annotation "does nothing".

Q30. How does Spring find `@Service` when it is not literally `@Component` ?

Component scanning reads class files with ASM without loading them, and uses metadata that includes annotations on annotations. `@Service` is meta-annotated with `@Component` , so it matches. `MergedAnnotations` follows the hierarchy and applies `@AliasFor` attribute overrides, which is how `@GetMapping` is a composed `@RequestMapping(method = GET)` and `@SpringBootApplication` bundles three annotations. Plain `method.getAnnotation(Component.class)` would not see it.

Q31. Why doesn't an annotation on an interface method always work?

Java does not inherit method annotations, and `@Inherited` only applies to class-level annotations from superclasses. Spring compensates by searching super-methods and interfaces with `AnnotationUtils.findAnnotation` , but mechanisms vary: a

`@Transactional` on an interface works with JDK proxies and has had different rules with class proxies. The documented recommendation is to annotate concrete classes.

Q32. How do you write a custom annotation with an aspect? What are the ways it silently fails?

Define it with `@Retention(RUNTIME)`, `@Target(METHOD)`, and use an `@Aspect` bean with `@Around("@annotation(x)")`. It fails silently when: the retention is wrong; the call is self-invocation (`this.foo()`) that bypasses the proxy; the method is private, or final on a CGLIB proxy; the object was created with `new` rather than by Spring; or `spring-boot-starter-aop` is missing. The Spring chapters cover proxies in detail.

Q33. What is the difference between an annotation processor and runtime reflection?

A processor runs inside javac, sees source elements, and generates new source or reports compile errors. MapStruct, for example, has no runtime cost and fails at build time. Reflection reads annotations at runtime, which is flexible and allows conditional behaviour, but costs startup time and fails late. Spring AOT and Micronaut move framework work to build time for faster startup and native images.

Q34. How does Lombok work if processors cannot modify classes?

It uses the annotation-processing entry point, then casts to javac's internal classes and modifies the AST directly, adding getters, constructors and so on before bytecode generation. This is unsupported API use, which is why new JDK releases can break Lombok and it needs `--add-opens` style workarounds. Records replace much of what `@Value` gave you.

Q35. What is reflection and what are its downsides?

Runtime inspection and invocation of classes, without compile-time knowledge. Downsides: errors shift from compile time to runtime; invocation is slower with boxing; encapsulation is bypassed; and GraalVM native images need configuration for each reflective access. Refactoring tools also miss string names. Frameworks are its proper home, with application code rarely needing it.

Q36. `getMethods()` vs `getDeclaredMethods()`?

`getMethods` returns public methods including inherited ones. `getDeclaredMethods` returns every method declared in that class, of any access, but none inherited, and includes bridges and synthetics. `getDeclaredFields` follows the same rule, so to find a private inherited field you must walk `getSuperclass()`. Spring's `ReflectionUtils` does this. Order is not guaranteed.

Q37. Can reflection change a `final` field or a private field?

Private: yes, with `setAccessible(true)`, subject to module encapsulation. Instance `final`: yes after `setAccessible`, but JIT-compiled code may have assumed it constant, and inlined constants never change, so results are unreliable. `static final`: always `IllegalAccessException`, and the old hack of clearing the `modifiers` field stopped working in Java 12. Record and hidden-class fields cannot be changed either.

Q38. What does `InvocationTargetException` mean?

`Method.invoke` wraps any exception the target threw. The real problem is in `getCause()`. A proxy handler that does not unwrap it makes callers see `UndeclaredThrowableException`, because `InvocationTargetException` is a checked exception the interface does not declare. That hides the real exception from `catch` blocks and from Spring's rollback rules.

Q39. What is `InaccessibleObjectException`, and how did you handle it when upgrading to Java 17?

Strong encapsulation (JEP 403) blocks reflective access to JDK internals, so old library versions fail with "module java.base does not opens java.lang to unnamed module". The right fix is upgrading the library (common offenders were old ByteBuddy,

Lombok, Gson on JDK classes). The temporary one is `--add-opens java.base/java.lang=ALL-UNNAMED`. `jdeps --jdk-internals` finds usage before upgrading.

Q40. `MethodHandle` vs reflection?

A method handle is a typed, directly executable reference, with access checked once at lookup. In a `static final` field the JIT can inline through it, and `invokeExact` avoids boxing. Reflection is easier and is built on method handles since Java 18. Use reflection for one-off framework work and handles in hot paths. `invokedynamic`, lambdas and string concatenation all link through them.

Q41. What is `VarHandle` and what did it replace?

A typed reference to a field or array element, with access modes: plain, `getAcquire`/`setRelease`, `volatile`, and atomics like `compareAndSet`. It replaced `sun.misc.Unsafe` and field updaters as the supported way to do low-level concurrency. The JDK's own `AtomicBoolean` and `AtomicReference` are built on it.

Q42. How does `Proxy.newProxyInstance` work internally?

It generates a class at runtime that implements the given interfaces and extends `Proxy`, with each method looked up once and held in a static `Method` field. A call forwards to `handler.invoke(proxy, method, args)`. Generated classes are cached per class loader and interface list, so repeat calls reuse them. `equals`, `hashCode` and `toString` are routed through the handler too.

Q43. JDK proxy vs CGLIB: why does Spring Boot default to CGLIB?

The JDK proxy needs an interface and can only be injected as that interface; CGLIB subclasses the class, so injecting the concrete type also works. Boot set `proxyTargetClass=true` by default in 2.0 to avoid the confusing "bean is of type $Proxy" errors. CGLIB's limits are that `final` classes and methods cannot be advised; Spring repackages it inside `spring-core`.

Q44. Why does a `@Transactional` method fail silently when `final`, and what happens to fields?

A CGLIB subclass cannot override it, so the call runs directly on the proxy instance, not on the target. Because Objenesis created the proxy without running constructors, its fields are null, which gives a confusing `NullPointerException` on injected collaborators. Kotlin users hit this constantly, since classes are final by default, hence the `kotlin-spring` plugin.

Q45. How does Mockito create a mock?

With ByteBuddy. Older versions generated a subclass, so final classes could not be mocked. Since Mockito 5 the default is `mock-maker-inline`, which uses a Java agent to instrument the class's bytecode, so final classes, and with `mockStatic`, static methods work too. Objenesis skips constructors. It is the same family of techniques as Hibernate's proxies.

Q46. What is SPI? Give JDK examples, and how is it different from an API?

An API is the contract callers use, while an SPI is the contract implementers provide, and the framework calls them. Examples: JDBC `Driver`, `java.nio.file.spi.FileSystemProvider`, `javax.annotation.processing.Processor`, logging bindings. `ServiceLoader` connects them by reading `META-INF/services/<interface>` files, or `provides` in `module-info`.

Q47. How does JDBC find the driver without `Class.forName`?

Since JDBC 4, `DriverManager`'s initialisation uses `ServiceLoader` for `java.sql.Driver`. Each driver jar lists itself under `META-INF/services/java.sql.Driver`, and its static initialiser registers the driver. Dropping PostgreSQL's jar on the class path is enough. Old `Class.forName("org.postgresql.Driver")` code simply forced that registration by hand.

Q48. What are common `ServiceLoader` problems?

The provider needs a public no-arg constructor (or, in a named module, a static `provider()` method). The file name must be the fully qualified interface name, exactly. A provider failure surfaces as `ServiceConfigurationError` during iteration, so a single bad jar can break the loop unless caught around `hasNext` and `next`. Loaded providers are not reloaded unless `reload()`. The class loader matters: a shaded fat jar merging `META-INF/services` files badly loses providers, so use the Maven Shade `ServicesResourceTransformer`.

Q49. Describe the plugin system in your project.

"`ryfn-plugin-api` defines two extension points, `DataConnector` and `FunctionPlugin`, with no dependency on the services. `Plugins.load` runs `ServiceLoader` over a `URLClassLoader` that adds the jars in `RYFN_PLUGINS_DIR`, with the service's loader as parent. So a jar, with its `META-INF/services` file, adds a connector or a rule function after a restart, and core code never changes. A broken plugin is logged and left out, so it can't stop a service. Plugins are trusted code, so operators install them, not tenants. `ryfn-plugin-example` is the reference implementation."

Q50. Static nested vs inner class: when to use each?

Default to static nested. An inner class carries a hidden `this$0` reference, costing memory and risking leaks, and is worth it only when each instance truly belongs to an outer instance and uses it, as `ArrayList.Itr` does with its list. Effective Java item 24: "favour static member classes over nonstatic."

Q51. Show how an inner class can leak memory. What is the fix?

```
class ReportBuilder {
    private final String name = "July close";
    private final List<Row> rows = loadMillions();
    Runnable notifier() {
        return new Runnable() { public void run() { log.info("done: " + name); } };  // reads this.name
    }
}
scheduler.scheduleAtFixedRate(new ReportBuilder().notifier(), 0, 1, MINUTES);
```

The scheduler keeps the `Runnable`, which keeps the `ReportBuilder` through `this$0`, and with it all the rows, just to read one string. Fix: copy what is needed first, `String n = name; return () -> log.info("done: " + n);`, so the lambda captures only the string; or use a static nested class that takes only what it needs. In a heap dump this appears as a retained path through `this$0`.

Q52. Lambda vs anonymous class internally?

An anonymous class is a real class file (`Outer$1`), with a new object on each evaluation, and `this` means that object. A lambda compiles to a private synthetic method plus `invokedynamic` linked through `LambdaMetafactory`, with a hidden class created at first use. It captures only what it uses, and a non-capturing lambda is reused as one instance. `this` in a lambda means the enclosing object. Full details are in the functional programming chapter.

Q53. Can an inner class have static members? Can a local class capture variables?

Since Java 16 (the records JEP), inner classes may declare static members. Before that, only constants were allowed. Local and anonymous classes can capture effectively final locals, copied into synthetic fields as `val$x`. Local records, enums and interfaces are allowed and are implicitly static, so they cannot capture.

Q54. How does an inner class access the outer's private fields?

Before Java 11, javac generated package-private synthetic accessors (`access$000`), which widened the access surface and showed up in stack traces. Since Java 11 (JEP 181, nest-based access control), nestmates share private access directly at the JVM level, and no accessors are generated.

Q55. What exactly happens during Java deserialization?

`ObjectInputStream` reads the class descriptor, checks `serialVersionUID` , and allocates the object without running constructors of `Serializable` classes, running only the no-arg constructor of the first non-serialisable superclass. It then sets the fields via reflection, including `final` ones, and invokes `readObject` and `readResolve` if defined. Because this bypasses constructors, invariants are not checked unless `readObject` re-validates them.

Q56. What is `serialVersionUID` and what happens if you don't declare it?

It's the version number compared when reading. If undeclared, the JVM computes it by hashing the class's name, interfaces, fields and methods, so an innocent change such as adding a method causes `InvalidClassException` on old data. Declare it explicitly, and increment only for incompatible changes. Adding a field (default on read) or changing a field from `transient` to non-transient is compatible; deleting a field, changing its type, or making it `transient` is not.

Q57. `transient` vs `static` vs `volatile` for serialization?

`transient` and `static` fields are not written. `transient` restores to the default (or set it in `readObject`), and a `static` field just has whatever value is current in the reader's JVM. `volatile` has nothing to do with serialization. `transient` on a `final` field leaves it at default, and without custom hooks it can't be reassigned, so avoid that combination.

Q58. What if a parent is not `Serializable` but the child is? And the reverse?

Child serialisable, parent not: parent fields are not saved, and the parent's no-arg constructor runs on read, so it must exist and be accessible, or you get `InvalidClassException` . Parent serialisable: children inherit that, whether wanted or not. A non-serialisable field reference that is not `transient` throws `NotSerializableException` at write time.

Q59. `Serializable` vs `Externalizable` ?

`Serializable` : the JVM writes the fields by reflection, and no constructor runs. `Externalizable` : you control all reading and writing, a public no-arg constructor is required and does run, and it can be more compact. Both are out of fashion: choose JSON, Protobuf or Avro for data that crosses a process boundary.

Q60. Why is Java deserialization a security risk, and how do you defend against it?

Reading bytes instantiates attacker-chosen classes from the class path, and `readObject` methods in common libraries can be chained into code execution, before your code ever looks at the object. Defences: never deserialise untrusted input; use data formats that bind to known types; apply `ObjectInputFilter` allow-lists (JEP 290, global filter factories in JEP 415); keep gadgets out with dependency updates. The same risk applies to Jackson default typing, as well as to old XML and YAML libraries (SnakeYAML's constructor).

Q61. How does Jackson work under the hood? Why reuse `ObjectMapper` ?

It introspects a type once (properties, creators, annotations), builds a serialiser or deserialiser, and caches it per mapper. Reads stream through a `JsonParser` token by token, and the optional Afterburner and Blackbird modules generate accessors to replace reflection. A new mapper per request re-does the introspection every time, which is far slower. Once configured it is thread-safe. For custom settings, derive one with `mapper.copy()` or get an immutable `ObjectReader` / `ObjectWriter` .

Q62. How do records work with Jackson? What are the gotchas?

Since 2.12, records bind through the canonical constructor using the component names, so no extra annotations are needed, and compact constructor validation always runs. Gotchas: unknown properties still fail on a bare `new ObjectMapper()` unless you add `@JsonIgnoreProperties(ignoreUnknown = true)` or disable `FAIL_ON_UNKNOWN_PROPERTIES`, while Spring Boot's mapper disables it by default; versions before 2.15 had edge cases around `@JsonProperty` and `@JsonIgnore` on components; and an extra accessor such as `isActive()` on a record can add a JSON property you did not intend.

Q63. How do you handle polymorphic JSON safely?

Use `@JsonTypeInfo(use = Id.NAME, property = "type")` with `@JsonSubTypes` listing the allowed classes, or a sealed hierarchy with a custom deserialiser. Avoid `Id.CLASS` and default typing on broad types like `Object`. For versioned event payloads, also ignore unknown fields so that older consumers tolerate new ones.

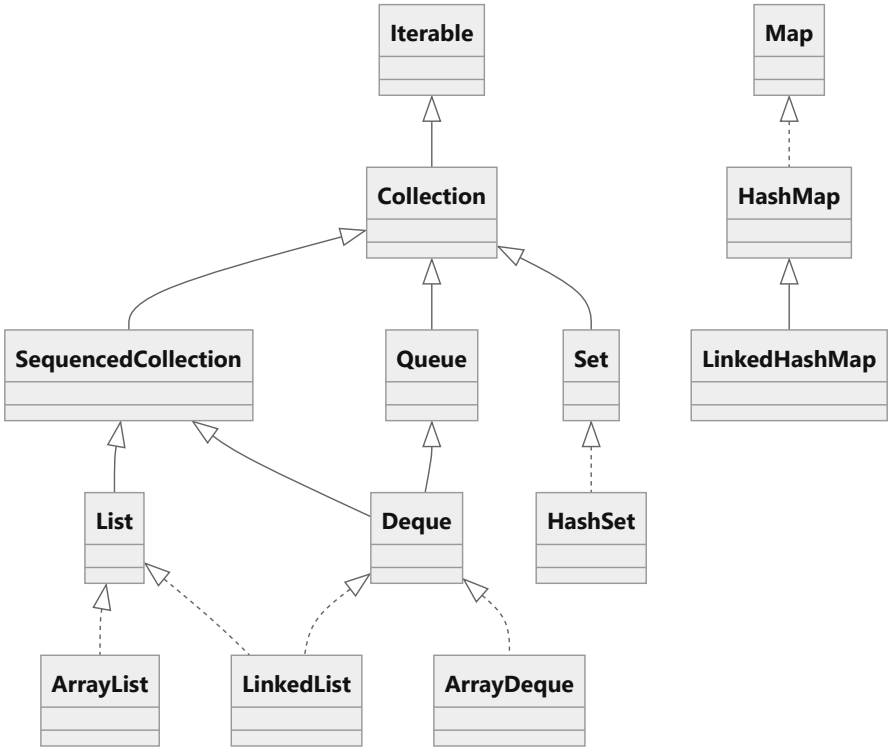
Q64. How would you evolve a stored JSON format?

Add fields as optional with defaults, tolerate unknown ones, never rename or re-type without a migration, and keep a version number where the shape may change meaningfully. Enum values need care too: a removed constant fails on read unless `READ_UNKNOWN_ENUM_VALUES_AS_NULL` or a `@JsonEnumDefaultValue` is used. In RefinePoint, a process's definition is JSON in a `jsonb` column bound to `ReconDefinition`, so enum names and field names are effectively part of the database schema.

4. Collections Internals I: Lists, HashMap, HashSet

"How does HashMap work?" is the most-asked Java interview question in the world, so a one-paragraph answer will not stand out. A senior answer goes through hash spreading, the index mask, the resize split and treeification, and connects each detail to a production consequence. This chapter gives you that answer, along with the internals of the list family and the iteration rules that catch people out.

4.1 The collections hierarchy



`Map` is not a `Collection`: it has key-value pairs, and exposes three collection *views* (`keySet`, `values`, `entrySet`) backed by the map itself. Modifying a view modifies the map. `SequencedCollection` arrived in Java 21 and is covered in chapter 5.

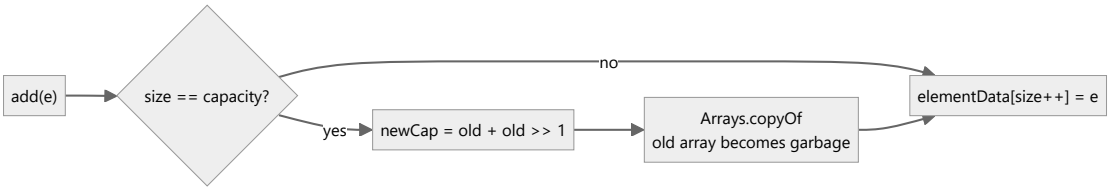
4.2 ArrayList

`ArrayList` is a growable wrapper over `Object[] elementData`, with a `size` field separate from the array's capacity.

```
elementData (capacity 10)         size = 4
+----+----+----+----+----+----+----+----+----+----+
| a  | b  | c  | d  |null|null|null|null|null|null|
+----+----+----+----+----+----+----+----+----+----+
  0    1    2    3    add("e") writes index 4, size++ : O(1)

add(1, "x"): System.arraycopy shifts 1..3 one to the right : O(n)
remove(1):   System.arraycopy shifts left, last slot nulled for GC : O(n)
```

Growth. `new ArrayList<>()` shares an empty array and allocates capacity 10 on the first `add` (lazy since Java 8). When full, `grow()` computes `newCapacity = old + (old >> 1)`, which is **1.5x**, then `Arrays.copyOf`. Growth is **amortised O(1)**: across n appends the total copying is bounded by about 3n element moves, since the geometric series converges.



Details interviewers probe:

- `System.arraycopy` is a JVM intrinsic that uses bulk memory moves, so shifting is much faster than a Java loop, though still O(n).
- `RandomAccess` is a marker interface. Algorithms such as `Collections.binarySearch` check `instanceof RandomAccess` to choose index-based or iterator-based paths.
- `trimToSize()` shrinks the capacity, and `ensureCapacity(n)` or `new ArrayList<>(n)` avoids repeated copying. Removing elements **never** shrinks the array automatically, so a list that once held a million items keeps that array.
- Why 1.5x and not 2x? It wastes less memory, and with a factor below the golden ratio the sum of freed blocks can eventually fit the next allocation. Overflow is handled by `ArraysSupport.newLength`, with `OutOfMemoryError` above the maximum array size.
- `elementData` is `transient`. `writeObject` serialises only `size` elements, not the empty capacity.

4.3 LinkedList, Vector, Stack, ArrayDeque

`LinkedList` is a doubly linked list, with `Node<E> { item, next, prev }` and pointers to `first` and `last`. `get(i)` walks from the nearer end, so it is O(n). Each node costs roughly 24–32 bytes of overhead on top of the element reference, and the nodes are scattered over the heap, causing cache misses when traversing.

Big-O flatters `LinkedList`. `add(0, x)` is O(1), but you usually have to find the position first, which is O(n) with pointer chasing. An `ArrayList` `arraycopy` over contiguous memory beats it in nearly all real workloads. Even Joshua Bloch, who wrote it, has said he never uses it. For queue or stack behaviour, use `ArrayDeque`.

`ArrayDeque` is a **circular buffer** with `head` and `tail` indices. Both ends are O(1) without node allocation, and it doubles capacity (or grows by 50% once large) when full. `null` elements are not allowed, because `null` signals "empty" from `poll`.

```
capacity 8: head = 6, tail = 2 (wraps around)
index:  0   1   2   3   4   5   6   7
value: [c] [d] [ ] [ ] [ ] [ ] [a] [b]
        addFirst writes index 5; addLast writes index 2
```

`Vector` is a synchronised `ArrayList` from Java 1.0 that doubles when full. Each method locks, but compound actions like "check then add" still race. `Stack extends Vector` is a design mistake: it inherits `add(int, E)`, which breaks LIFO. Use `ArrayDeque` for a stack, and `CopyOnWriteArrayList` or an explicit lock for concurrent lists.

Complexity table

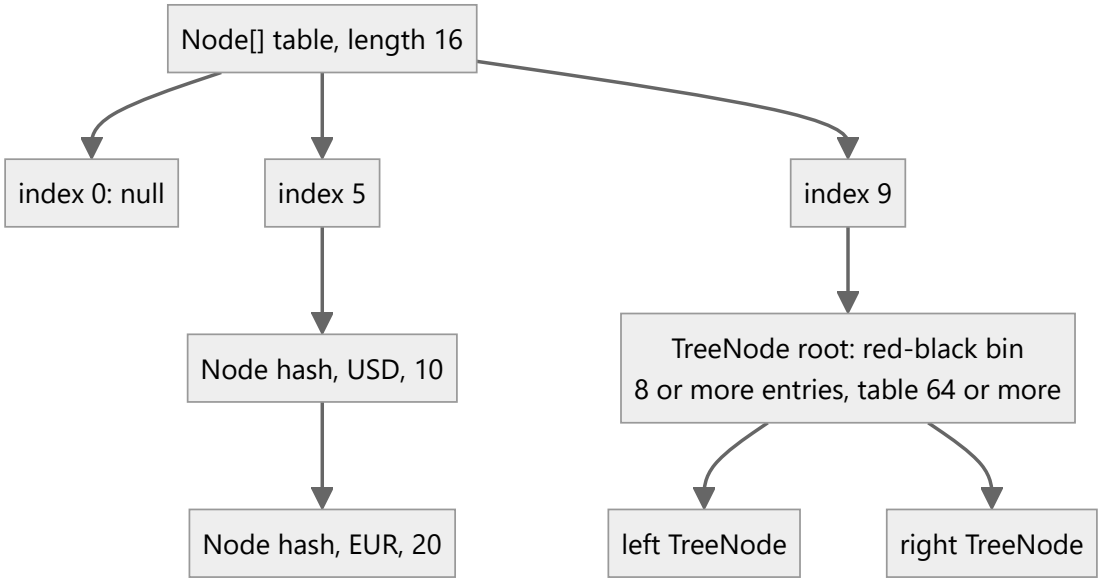
Operation	ArrayList	LinkedList	ArrayDeque	HashMap/HashSet	TreeMap/TreeSet
get by index	O(1)	O(n)	n/a	n/a	n/a
add at end	O(1) amortised	O(1)	O(1) amortised	n/a	n/a
add/remove at front	O(n)	O(1)	O(1)	n/a	n/a
add/remove in middle	O(n)	O(n) to find, O(1) to link	n/a	n/a	n/a
contains	O(n)	O(n)	O(n)	O(1) average	O(log n)
put/remove by key	n/a	n/a	n/a	O(1) average, O(log n) worst	O(log n)
Memory per element	Lowest	High	Low	High (entry object)	High (entry with color and parent)

4.4 HashMap internals

The structure

```
transient Node<K,V>[] table;     // allocated lazily on the first put
transient int size;
int threshold;                   // capacity * loadFactor
final float loadFactor;          // default 0.75
transient int modCount;          // structural changes, for fail-fast iterators

static class Node<K,V> { final int hash; final K key; V value; Node<K,V> next; }
```



From key to bucket, step by step

```
static final int hash(Object key) {
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
```

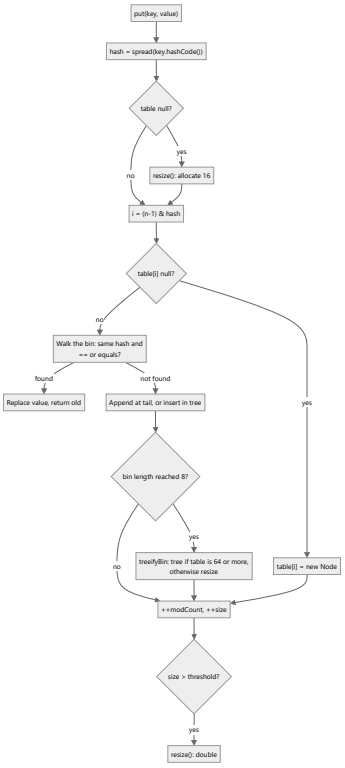
```
}
int index = (table.length - 1) & hash;
```

Why spread (`h ^ h >>> 16`)? The index uses only the low bits of the hash. Table sizes are small, so with 16 buckets only 4 bits matter. Hash codes that differ only in their high bits (such as `Float` keys, or sequences of multiples of 2^16) would all collide, so folding the high 16 bits into the low 16 gives them a say at negligible cost.

Why a power-of-two capacity? `(n - 1) & hash` is a very fast modulo, and it only works because `n - 1` is all ones in binary. It is also what makes the resize split cheap. `new HashMap<>(17)` is rounded up to 32 by `tableSizeFor`.

```
hashCode          1011 0110 1101 0010  0001 1100 1010 0101
h >>> 16          0000 0000 0000 0000  1011 0110 1101 0010
hash = h ^ h>>>16 1011 0110 1101 0010  1010 1010 0111 0111
n - 1 = 15        0000 0000 0000 0000  0000 0000 0000 1111
index             ................................... 0111  = 7
```

`put`, the full flow



Note the comparison order: **hash first, then `==`, then `equals`**. Comparing stored ints is cheap, and it means `equals` is only called on real hash matches. `get` follows the same path without the insert.

Replacing a value does not increment `modCount`, since it is not a structural change. That is why you can call `map.put(existingKey, v)` while iterating without getting `ConcurrentModificationException`.

Load factor and capacity

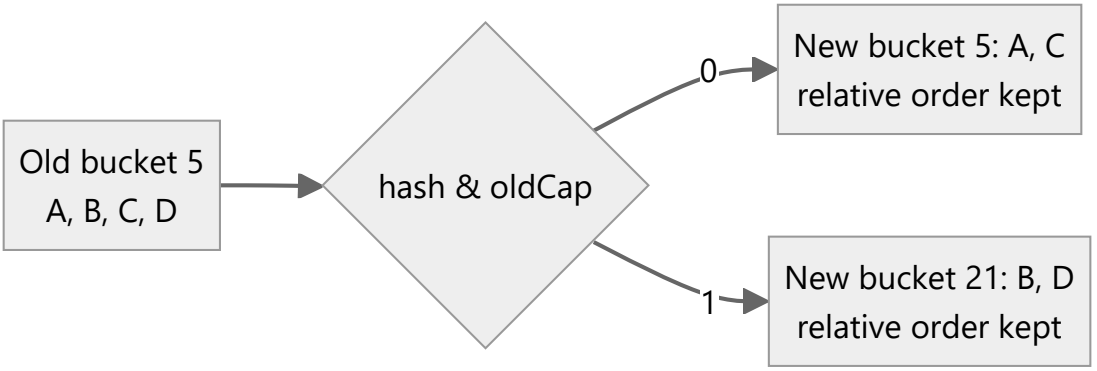
Threshold = capacity × load factor: 16 × 0.75 = 12, so the 13th entry triggers a resize to 32. The 0.75 default is a trade-off between memory and collisions. With random hashes, bin lengths follow a Poisson distribution with λ ≈ 0.5, giving P(bin has 8 entries) ≈ 0.00000006. That figure is why bins rarely treeify unless the hash function is poor or under attack.

Presizing: to hold `n` entries without resizing, use capacity `n / 0.75 + 1`. `new HashMap<>(n)` does **not** mean "n entries": `new HashMap<>(1000)` gets a table of 1024, with a threshold of 768, and resizes once. Java 19 added `HashMap.newHashMap(int numMappings)`, which does the arithmetic for you.

Resize and the bucket split

Doubling the table means each entry either stays at index `j` or moves to `j + oldCap`, decided by a single bit: `hash & oldCap`. There is no rehash and no re-modulo.

```
oldCap = 16, entries in old bucket 5:
hash ...0 0101 : hash & 16 == 0  stays at  5      (lo list)
hash ...1 0101 : hash & 16 != 0  moves to  5 + 16 = 21  (hi list)
```



Each resize is O(n), and a large one is a visible allocation plus a copy pause on that one `put`. A resize of a many-million-entry map shows up as a latency spike, so presize when the size is known.

Treeification

Constant	Value	Meaning
`TREEIFY_THRESHOLD`	8	A bin at this length is converted to a red-black tree
`MIN_TREEIFY_CAPACITY`	64	If the table is smaller than this, **resize instead** of treeifying
`UNTREEIFY_THRESHOLD`	6	During a resize split, a tree bin with 6 or fewer entries becomes a list again

Treeifying changes the worst case from O(n) to O(log n) per bin. The gap between 8 and 6 provides **hysteresis**, so a bin hovering around one value does not flip back and forth. `TreeNode` objects are about twice the size of `Node`.

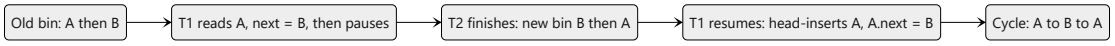
How can a tree be ordered when keys may not be `Comparable`? It orders by hash first. If hashes are equal and the key implements `Comparable`, it uses `compareTo`. Otherwise `tieBreakOrder` uses class name and identity hash so that insertion can continue, but a *lookup* may need to search both subtrees. So a tree bin only reaches O(log n) lookup with distinct hashes or `Comparable` keys.

Why was this added? Hash-flooding denial of service: an attacker submits POST parameter names that all collide (`"Aa"` and `"BB"` have the same `hashCode`, and any concatenation of them does too), degrading a request-parsing map to $O(n^2)$ overall. Java 8's trees cap the damage, since `String` is `Comparable`.

Java 7 vs Java 8

	Java 7	Java 8+
Bin insert	At **head**	At **tail**
Worst-case bin	O(n) linked list	O(log n) tree
Hash function	Several shifts and XORs	One `h ^ h>>>16`, since trees handle bad cases
Resize	Rehash each entry; reverses bin order	`hash & oldCap` split; keeps order
Concurrent resize	Can create a **cycle**, causing an infinite loop	No cycle, but still loses data
Null key	Special-cased	Hash 0, bucket 0
Table allocation	Lazy since 7u40	Lazy

The Java 7 infinite loop. Two threads resize at once. Because Java 7 reverses each bin as it moves entries, the threads see opposite link directions, and one can write `A.next = B` while the other has `B.next = A`. A later `get` that walks that bin never terminates, showing up as a thread stuck at 100% CPU inside `HashMap.get`.



Java 8 preserves order with its lo and hi lists, so the cycle no longer forms, but `HashMap` remains unsafe for concurrent writes: updates can be lost, and `size` can be wrong. Use `ConcurrentHashMap`.

Null keys and iteration order

One `null` key is allowed (hash 0, bucket 0), and any number of `null` values. `map.get(k) == null` therefore does not distinguish "absent" from "maps to null", so use `containsKey`, or better, do not store nulls. `Map.of`, `ConcurrentHashMap` and `Hashtable` reject nulls.

Iteration order is **not guaranteed** and can change on resize or between JDK versions. Iteration scans the whole table, so its cost is O(capacity + size). A map with a large capacity and few entries iterates slowly even if it is almost empty, and `clear()` keeps the capacity.

> **Trap:** code that "works" because it relied on `HashMap` order breaks when a key is added or the JDK changes, as when Java 8 changed `String`-keyed iteration order and broke many brittle tests. Use `LinkedHashMap` if order matters.

4.5 HashSet

`HashSet` is a thin wrapper around a `HashMap`, where each element is a key and the value is a shared dummy object:

```
private transient HashMap<E,Object> map;
private static final Object PRESENT = new Object();
public boolean add(E e) { return map.put(e, PRESENT) == null; }
public boolean contains(Object o) { return map.containsKey(o); }
```

That is why it allows one `null`, has the same complexity, and has the same memory overhead (about 32–48 bytes per element with entry plus header). `add` returns `false` for a duplicate, and the existing element is kept, not replaced. `LinkedHashSet` uses a `LinkedHashMap`, and `TreeSet` a `TreeMap`.

```
new HashSet<>(List.of("a", "b")).size();  // 2
Set<String> s = new HashSet<>(collection);  // capacity max(n / .75 + 1, 16), no resizing while copying
```

4.6 `equals`, `hashCode` and mutable keys

A bucket lookup first needs the same `hashCode`, and then `equals`. Break the contract and entries disappear.

Situation	Effect in a HashMap
`equals` overridden, `hashCode` not	Equal keys land in different buckets: duplicates, failed lookups
`hashCode` overridden, `equals` not	Same bucket, but only the same instance matches
Constant `hashCode`	Legal; everything in one bin, tree or O(n), very slow
Mutable field in `hashCode`, then mutated	Entry stored under the old hash: unreachable by `get`, and still counted and iterated

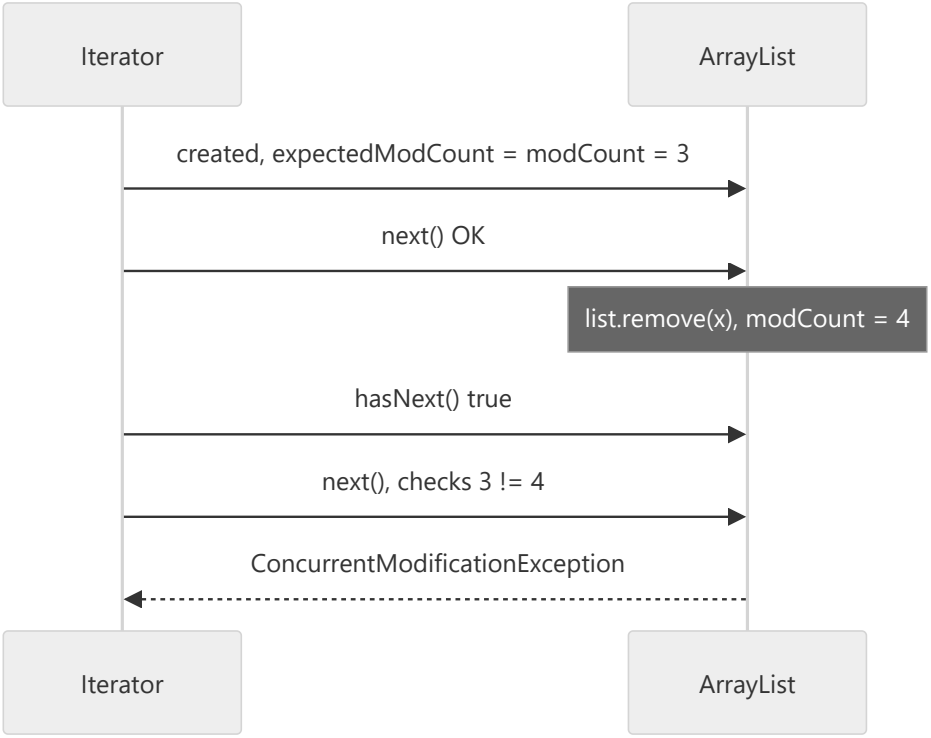
```
List<String> key = new ArrayList<>(List.of("a"));
Map<List<String>, String> m = new HashMap<>();
m.put(key, "v");
key.add("b");                   // hashCode changes
m.get(key);                     // null: looks in the new bucket
m.containsKey(List.of("a"));    // false: right bucket, but stored key's equals now fails
m.size();                       // 1: the entry leaks, unreachable through lookups
```

Stored `Node.hash` is a final copy of the hash at insert time, so resizing also never re-reads `hashCode()`. Good keys are immutable: `String`, wrappers, enums, records of immutable values, and `UUID`.

> **RefinePoint:** `ConsumedEvents` guards idempotency in the database, but in-memory keys follow the same rule. The gateway's rate-limit keys are plain strings (`"ip:" + address`, `"user:" + subject`, `"key:" + digest`), so the bucket map's keys cannot mutate.

4.7 Iterators and fail-fast behaviour

`ArrayList`, `HashMap` and most non-concurrent collections have **fail-fast** iterators. Each structural change increments `modCount`. The iterator saves it as `expectedModCount` on creation, and each `next()` compares the two.



This has nothing to do with threads, despite its name: one thread is enough. It is **best-effort**, and is not a concurrency guarantee, since the check is unsynchronised.

```
List<String> list = new ArrayList<>(List.of("a", "b", "c"));
for (String s : list) if (s.equals("a")) list.remove(s);   // CME on the next iteration

// The famous exception: removing the second-to-last element does NOT throw
List<String> two = new ArrayList<>(List.of("a", "b"));
for (String s : two) if (s.equals("a")) two.remove(s);     // no CME, and "b" is never visited
```

The second loop does not throw because after the removal `size` is 1, and the cursor is 1, so `hasNext()` returns false, and the check in `next()` never runs.

Ways to remove while iterating:

```
list.removeIf(s -> s.startsWith("x"));            // best: O(n), one bulk shift
Iterator<String> it = list.iterator();            // explicit iterator
while (it.hasNext()) if (bad(it.next())) it.remove();    // syncs expectedModCount; O(n) each for ArrayList
for (int i = list.size() - 1; i >= 0; i--) if (bad(list.get(i))) list.remove(i);  // backwards
map.entrySet().removeIf(e -> e.getValue() == null);
```

Iterator type	Collections	Behaviour under modification
Fail-fast	`ArrayList`, `HashMap`, `TreeMap`	Throws CME on a best-effort basis
Weakly consistent	`ConcurrentHashMap`, `ConcurrentLinkedQueue`	Never throws; may or may not show changes
Snapshot	`CopyOnWriteArrayList`	Never throws; sees the array at creation; `remove` unsupported

4.8 `Comparable` vs `Comparator`

	`Comparable<T>`	`Comparator<T>`
Package	`java.lang`	`java.util`
Method	`compareTo(T)`	`compare(T, T)`
Where defined	In the class itself: the natural order	External: any number of orders
Used by	`TreeMap`, `Collections.sort(list)`	`list.sort(cmp)`, `TreeMap(cmp)`, streams

```
Comparator<Run> byOrg = Comparator.comparing(Run::orgCode)
    .thenComparing(Run::startedAt, Comparator.nullsLast(Comparator.reverseOrder()))
    .thenComparingInt(Run::matched);
```

Rules: be **consistent with** `equals`, or a `TreeSet` will drop elements that `compareTo` says are equal, as with `BigDecimal` `2.0` and `2.00`. Never subtract to compare: `a - b` overflows, whereas `Integer.compare` does not. And a violated contract can make TimSort throw "Comparison method violates its general contract!".

`List.sort` and `Arrays.sort(Object[])` use **TimSort**, which is stable and O(n log n) with O(n) for nearly sorted data. `Arrays.sort(int[])` uses dual-pivot quicksort, which is not stable, because stability is meaningless for primitives.

4.9 Immutable and fixed-size collections

	`List.of`	`Collections.unmodifiableList(x)`	`Arrays.asList(a)`	`List.copyOf(x)`	`stream.toList()`
Add/remove	UOE	UOE	UOE	UOE	UOE
`set`	UOE	UOE	**Allowed**, writes to the array	UOE	UOE
Reflects source changes	n/a	**Yes**, it is a view	**Yes**, it wraps the array	No, independent	No
Nulls	NPE	Allowed if source does	Allowed	NPE	Allowed
Truly immutable	Yes	No	No	Yes	Unmodifiable

(UOE = `UnsupportedOperationException`.)

```
List<String> src = new ArrayList<>(List.of("a"));
List<String> view = Collections.unmodifiableList(src);
List<String> copy = List.copyOf(src);
src.add("b");
view.size();  // 2: the view sees it
copy.size();  // 1

int[] nums = {1, 2};
Arrays.asList(nums).size();   // 1: a List<int[]>, because int is not a reference type
String[] names = {"x"};
List<String> fixed = Arrays.asList(names);
fixed.set(0, "y");            // names[0] is now "y"
List.of("a").contains(null);  // NPE: List.of rejects even a null query
```

Implementation notes. `List.of` uses compact classes such as `ImmutableCollections.List12` and `ListN`, with `@Stable` fields. `Set.of` and `Map.of` randomise iteration order with a per-JVM salt, deliberately, so nobody relies on it. They also throw `IllegalArgumentException` on duplicates, which surprises people who build a `Set.of(a, b)` from runtime values.

4.10 Collections utility methods

`sort`, `reverse`, `shuffle`, `binarySearch` (the list must be sorted), `frequency`, `disjoint`, `nCopies`, `max`, `min`, `swap`, `rotate`, `addAll`, `emptyList`, `singletonList`, `unmodifiableX`, `synchronizedX`, and `checkedX`. `Collections.emptyList()` returns a shared immutable singleton, so returning it costs no allocation. Prefer returning empty collections over `null`.

> **RefinePoint:** the engine's `SideProcessing` presizes its row lists with `new ArrayList<>((int) count)` when the count is known, as large files would otherwise repeatedly copy a large array. `removeIf` on `entrySet()` is how `RateLimitFilter` and `SlidingWindowLimiter` evict idle buckets, with no iterator handling.

4.11 Interview questions and answers

Q1. How does `HashMap` work internally?

`HashMap` is an array of buckets, where each bucket holds a linked list or, past a threshold, a red-black tree. `put` spreads the key's `hashCode` with `h ^ h >>> 16`, finds the bucket with `(n-1) & hash`, then compares the stored hash and `equals` along the bin to replace or append. When `size` passes `capacity × 0.75` the table doubles and entries split between `j` and `j + oldCap`. A bin of 8 or more converts to a tree if the table has at least 64 slots; otherwise the table resizes instead. Average get and put are O(1), with O(log n) as the worst case per bin.

Q2. Why is the HashMap capacity always a power of two?

So that `hash & (n - 1)` can replace the slower `%`, and so that a resize split can be decided by one bit. It requires the low bits of the hash to be good, which is exactly why the hash is spread with `h ^ (h >>> 16)`. `tableSizeFor` rounds any requested capacity up to the next power of two.

Q3. Why XOR with the high 16 bits instead of just using `hashCode()`?

A small table uses only the low bits of the hash, so hash codes that differ only in their high bits would always collide. XOR mixes the high half into the low half, cheaply, so those bits affect the index. XOR is preferred over `&` and `|` because it keeps the bits balanced. It's only one shift because trees now handle pathological cases, so a complex hash function is no longer worth it.

Q4. Why is the default load factor 0.75?

A balance of memory against collisions. Higher values fill the table better, but lengthen bins and slow lookups; lower values waste space and resize more. At 0.75 the expected bin length stays well under 1, and the JDK comments give a Poisson estimate of about 0.00000006 for a bin of 8. It is rarely worth changing, and presizing is usually the better tuning.

Q5. What is the threshold? When does a resize happen, and what is the size after `new HashMap<>(10)`?

Threshold is `capacity × loadFactor`, and a resize happens when `++size > threshold` after an insert. `new HashMap<>(10)` allocates nothing yet: it stores 16 in `threshold`, and on the first `put` allocates a table of 16 with a threshold of 12. The size is still 0, since size counts entries and not slots.

Q6. How many resizes does it take to insert 1,000 items into a default HashMap? How do you avoid them?

Seven: capacity grows 16 → 32 → 64 → 128 → 256 → 512 → 1024 → 2048, since 1024 × 0.75 = 768 < 1000. To avoid them, use `HashMap.newHashMap(1000)` (Java 19+) or `new HashMap<>((int) (1000 / 0.75f) + 1)`.

Q7. What happens on a hash collision?

Keys with the same bucket index chain in that bin: appended at the tail in Java 8+, with each lookup comparing stored hash, then `==`, then `equals`. Note the difference between a **hash collision** (same hash) and a **bucket collision** (different hashes, same index). Comparing stored hashes filters out the second kind without calling `equals`.

Q8. When exactly does a bin convert to a tree? Back to a list?

On inserting into a bin that already has 8 nodes, making it the 9th (`binCount >= TREEIFY_THRESHOLD - 1`), `treeifyBin` runs. If the table is under 64, it **resizes instead**, as spreading entries is cheaper than building trees. It converts back at 6 or fewer during a resize split, and also on removal when the tree becomes very small (its root or close children are null). The 8/6 gap prevents flip-flopping.

Q9. How is a tree bin ordered if keys are not `Comparable`?

By hash, then by `compareTo` if the key's class implements `Comparable<SameClass>`, then by `tieBreakOrder` (class name and `identityHashCode`) so that inserts always have a place. On lookup, equal hashes with non-comparable keys force searching both subtrees, which degrades towards O(n). So keys with a poor hash but no `Comparable` get no real benefit.

Q10. What is a hash flooding attack and how did Java respond?

An attacker crafts many keys with identical `hashCode`, like form parameter names built from `"Aa"` and `"BB"`, turning map operations O(n) and a request $O(n^2)$, which burns CPU with a tiny payload. Java 8's treeified bins cap it at O(log n), since `String` is `Comparable`. Servers also limit parameter counts, as Tomcat does with `maxParameterCount`.

Q11. Describe the resize in Java 8. Why doesn't it recompute hashes?

It allocates a table of double the size and walks each old bin. Each node's new position depends on one extra bit, so `hash & oldCap == 0` keeps it at `j`, and otherwise it moves to `j + oldCap`. Stored hashes are reused, so `hashCode()` isn't called, and two sublists keep their relative order. Tree bins are split the same way, and they untreeify if a side has 6 or fewer.

Q12. Why could a Java 7 `HashMap` loop forever?

Its resize reversed bin order through head insertion. Two threads resizing at once could produce `A.next = B` and `B.next = A`. The next `get` on that bin spins forever at 100% CPU, which a thread dump shows as a thread stuck in `HashMap.get`. Java 8 keeps order and avoids this cycle, but concurrent writes still corrupt state.

Q13. Is Java 8 `HashMap` safe for concurrent use now that the loop is fixed?

No. Concurrent `put`s can overwrite each other's bucket head, resize can lose entries, `size` can be wrong (`++size` is not atomic), and tree bins can throw `ClassCastException` during concurrent treeify. Visibility is not guaranteed either. If you need concurrent access, use `ConcurrentHashMap`. A map published once and only read afterwards is safe, if safely published, and `Map.copyOf` makes that explicit.

Q14. Can `HashMap` have a null key? Where is it stored?

One null key, with hash 0, so bucket 0, and any number of null values. Because `get` returns `null` both for absent keys and null values, use `containsKey` or `getOrDefault` with care: `getOrDefault` returns the stored `null`, not the default, when the key maps to null.

Q15. Predict the output.

```
Map<String, Integer> m = new HashMap<>();
System.out.println(m.put("a", 1));
System.out.println(m.put("a", 2));
System.out.println(m.get("a"));
System.out.println(m.putIfAbsent("a", 3));
System.out.println(m.computeIfPresent("a", (k, v) -> null));
System.out.println(m.size());
```

`null`, `1`, `2`, `2`, `null`, `0`. `put` returns the previous value, `putIfAbsent` returns the existing value and changes nothing, and a remapping function that returns `null` removes the entry.

Q16. What happens if two keys are equal but have different `hashCode`s?

They usually land in different buckets, so the map holds both, and a lookup with a third equal key finds at most one of them. Even in the same bucket, the stored-hash check fails first, so `equals` is not reached. The contract is violated, and the behaviour looks random, which is why IDE-generated methods and records are the safe default.

Q17. What if `hashCode` always returns 1?

It's legal, and every entry lands in one bin. Once it reaches 8 entries the table resizes up to 64, and then that bin treeifies. With `Comparable` keys, lookups are O(log n); without, ties force a search of both subtrees, near O(n). Inserts stay O(log n) only with

`Comparable` keys, since an equality scan is needed otherwise. The map still works correctly, just slowly.

Q18. What happens if you mutate a key after inserting it?

Its stored hash is stale, so a `get` with the mutated object computes a new bucket and misses. A `get` with an object equal to the old state finds the bucket but `equals` fails against the mutated stored key. The entry can't be found but still occupies memory, counts in `size`, and appears in iteration, which is a real leak. Use immutable keys.

Q19. What makes a good HashMap key? Why is `String` ideal?

It should be immutable, with consistent `equals` and `hashCode`, and a well-distributed hash. `String` is final and immutable, caches its hash after the first computation, and implements `Comparable`, so tree bins work well. `UUID`, wrappers, enums and records of immutable values also fit. Avoid `double`-based keys, mutable objects, and JPA entities with generated IDs.

Q20. Why not use an enum as a `HashMap` key?

It works, but `EnumMap` is better: an array by ordinal, no hashing, smaller and iterating in declaration order. An enum's `hashCode` is the identity hash, which varies between JVM runs, so `HashMap<Status, ...>` iteration order can differ between runs, which can make output and tests non-deterministic.

Q21. `HashMap` vs `Hashtable` vs `ConcurrentHashMap` vs `Collections.synchronizedMap`?

	HashMap	Hashtable	synchronizedMap	ConcurrentHashMap
Thread safety	None	One lock on every method	One lock, wrapper	CAS and per-bin locks
Null key or value	Allowed	No	As the wrapped map	No
Iterator	Fail-fast	Fail-fast, plus legacy Enumeration	Fail-fast; caller must lock	Weakly consistent
Index	`hash & (n-1)`	`(hash & 0x7FFFFFFF) % n`, table size prime-ish	Same as wrapped map	`spread & (n-1)`
Use `HashMap` for local use and `ConcurrentHashMap` for shared use. `Hashtable` is legacy. Chapter 5 covers CHM.				

Q22. `HashSet` internals: how does it guarantee uniqueness?

It delegates to `HashMap`, storing each element as a key with a shared `PRESENT` value. `add` calls `map.put` and returns true when the previous value was `null`. Uniqueness therefore means `hashCode` + `equals`, and a duplicate add does not replace the existing element, because `put` keeps the old key and only overwrites the value.

Q23. Predict the output.

```
record P(int x) {}
class Q { int x; Q(int x) { this.x = x; } }
Set<Object> s = new HashSet<>();
s.add(new P(1)); s.add(new P(1));
s.add(new Q(1)); s.add(new Q(1));
System.out.println(s.size());
```

`3`. Records implement value-based `equals` and `hashCode`, so the second `P` is a duplicate. `Q` inherits identity equality from `Object`, so both instances stay.

Q24. How would you find duplicates in a list in O(n)?

`Set<T> seen = new HashSet<>(); list.stream().filter(x -> !seen.add(x)).toList()`, using the boolean from `add`. For counts, use `Collectors.groupingBy(identity(), counting())`, or `map.merge(x, 1, Integer::sum)`. A stateful lambda like that is fine in a sequential stream, but not in a parallel one.

Q25. `ArrayList` vs `LinkedList`: which do you use and why?

`ArrayList`, almost always. Indexing is O(1), iteration is cache-friendly, and memory overhead is low. `LinkedList` theoretically wins at inserts in the middle, but finding the position is O(n) pointer chasing. In practice it loses benchmarks even in many "insert heavy" scenarios. It's only useful with a long-held `ListIterator`, and for queue or stack use `ArrayDeque` is faster.

Q26. How does `ArrayList` grow? What is its amortised cost?

On overflow, it allocates `old + old >> 1` (1.5x) and copies with `Arrays.copyOf`. The first add to a default list allocates capacity 10. Appends are amortised O(1), since doubling-style growth keeps total copying linear in n. For a known size, presize to skip the copies: a list growing to 1 million elements from the default makes about 28 copies and much garbage.

Q27. Is the default capacity of `ArrayList` allocated when it is created?

No. Since Java 8 `new ArrayList<>()` points at a shared empty array and allocates 10 on the first add. `new ArrayList<>(0)` uses a different empty array, and grows 0 → 1 → 2 → 3 → 4 → 6 and so on, which is worse if the list does get filled. This matters for classes that create many mostly empty lists.

Q28. How is an `ArrayList` serialised? Why is `elementData` `transient`?

The array usually has unused capacity. `writeObject` writes `size` and only the live elements, and `readObject` allocates exactly that many. It avoids serialising nulls. The same reasoning applies to `HashMap`, whose `table` is transient too, and which writes pairs and rebuilds on read, as hash codes may differ between JVMs.

Q29. Does removing elements shrink an `ArrayList`?

No. `remove` shifts and nulls the last slot, but capacity stays. A list that peaked at 1M elements holds that array until `trimToSize()` or until the list is discarded. In a long-lived cache or buffer, that shows up in heap dumps as a huge `Object[]` that is mostly nulls. Replace the list, or call `trimToSize` after large removals.

Q30. Predict the output.

```
List<Integer> a = new ArrayList<>(List.of(10, 20, 30));
a.remove(Integer.valueOf(10));
a.remove(0);
a.add(0, 5);
System.out.println(a);
```

`[5, 30]`. Removing the value 10 leaves `[20, 30]`, removing index 0 leaves `[30]`, and inserting at 0 gives `[5, 30]`.

Q31. What is `RandomAccess`?

A marker interface saying that indexed access is fast. Generic algorithms use it to pick a strategy: `Collections.shuffle`, `binarySearch` and others use `get(i)` on `RandomAccess` lists, and iterators otherwise. Calling `list.get(i)` in a loop on a `LinkedList` is $O(n^2)$, which is the kind of hidden cost this avoids.

Q32. What is `Arrays.asList` and what are its traps?

It returns a fixed-size list **backed by the array**, a private nested class named `Arrays.ArrayList`, not `java.util.ArrayList`. `set` writes through, while `add` and `remove` throw `UnsupportedOperationException`. With an `int[]`, you get a single-element `List<int[]>`. For a growable list use `new ArrayList<>(Arrays.asList(a))`, and for an immutable one use `List.of(a)`.

Q33. `List.of` vs `Collections.unmodifiableList` vs `List.copyOf`?

`List.of` creates a new immutable list and rejects nulls. `unmodifiableList` is a read-only **view**, so changes to the source show through. `List.copyOf` makes an immutable copy, and returns the same instance if the input is already an immutable list, so it is cheap to use defensively. A public API should take `List.copyOf` of input it stores.

Q34. What does `list.subList` return? What is the trap?

A **view** on the parent, so `list.subList(0, 10).clear()` removes from the parent efficiently. Trap one: structurally modifying the parent afterwards makes the view throw `ConcurrentModificationException`. Trap two: holding a small sublist of a huge list keeps the whole list reachable. Copy it with `new ArrayList<>(sub)` if it outlives the operation.

Q35. What is a `ConcurrentModificationException`? Does it need multiple threads?

It is thrown when a fail-fast iterator sees `modCount` differ from its saved copy. No threads are needed: modifying a list inside a for-each over it is the common cause. It is best-effort and may not fire at all, so never rely on it for correctness.

Q36. Why doesn't removing the second-last element throw CME?

For-each calls `hasNext()` before `next()`, and `hasNext()` is simply `cursor != size`. After the removal `size` is one smaller and equals `cursor`, so the loop ends without calling `next()`, where the check lives, and the last element is silently skipped. It is a good demonstration that fail-fast is not guaranteed.

Q37. How do you safely remove while iterating?

Best is `collection.removeIf(predicate)`, which is one O(n) pass for `ArrayList`. Other options: `Iterator.remove`, which updates `expectedModCount` but shifts each time and is $O(n^2)$ overall for many removals in a large list; an index loop running backward; or building a new list using a stream filter. For maps, `entrySet().removeIf(...)`, or `keySet().removeAll(...)`.

Q38. Does a concurrent read during `HashMap.put` cause CME?

No. CME comes only from iterators, not from `get`. A concurrent `get` may simply miss an entry or read stale data. Without synchronisation there's no guarantee it sees the latest value at all, which is why races can pass tests but fail in production.

Q39. Why does `map.put(existingKey, newValue)` during iteration not throw?

Replacing a value is not a structural change, so `modCount` doesn't change. Adding a new key does. `Map.Entry.setValue` is also safe, and is the proper way to update values while iterating. `compute` and `merge` on `HashMap` detect changes: a mapping function that modifies the map throws CME on Java 9+.

Q40. What do fail-fast, fail-safe and weakly consistent mean?

Fail-fast iterators throw on detecting a change (`ArrayList` , `HashMap`). "Fail-safe" is an informal term for iterators that never throw, working on a copy or tolerating changes. Javadoc distinguishes **snapshot** (`CopyOnWriteArrayList` , frozen at creation) from **weakly consistent** (`ConcurrentHashMap` : reflects some changes, each element returned at most once).

Q41. `Comparable` vs `Comparator` : how do you choose?

`Comparable` defines the type's single natural order, when one obvious ordering exists (dates, money), and `compareTo` should be consistent with `equals` . `Comparator` is for alternative orders, chained with `comparing` and `thenComparing` . Comparators can also order types you don't own. Null handling should be explicit with `nullsFirst` and `nullsLast` .

Q42. Why is `return a.x - b.x` a bug in a comparator?

Integer subtraction overflows for values of large magnitude: `Integer.MIN_VALUE - 1` becomes positive, inverting the order, and also breaking transitivity. Use `Integer.compare` , `Long.compare` , or `Comparator.comparingInt` . For doubles, use `Double.compare` , since it handles `NaN` and `-0.0` .

Q43. What does "Comparison method violates its general contract" mean?

TimSort detected an inconsistent comparator while merging: for example, a non-transitive one, or one that returns random values or reads mutable state. It only fails on some inputs, usually with 32 or more elements, so it surfaces in production and not in small tests. Fix the comparator, using `Comparator.comparing` chains and consistent null handling.

Q44. Why does a `TreeSet` drop elements that `equals` says are different?

`TreeSet` uses `compareTo` or the comparator for identity, not `equals` . A comparator on `name` alone treats two people with the same name as duplicates. `new TreeSet<>(List.of(new BigDecimal("2.0"), new BigDecimal("2.00")))` has one element, while a `HashSet` of the same keeps two. Keep the comparator consistent with `equals` , or understand that consequence.

Q45. Which sorting algorithm does Java use?

`Arrays.sort(Object[])` , `List.sort` and `Collections.sort` use TimSort, a stable merge and insertion hybrid that takes advantage of existing runs. Primitive arrays use dual-pivot quicksort, with fallbacks such as heap sort on bad input since Java 14; stability doesn't matter there. `parallelSort` splits work over the common fork-join pool. `ArrayList.sort` sorts its array in place and bumps `modCount` .

Q46. `ArrayDeque` vs `Stack` vs `LinkedList` as a stack or queue?

`ArrayDeque` : a circular array with no per-element allocation, usually the fastest, but no nulls. `Stack` : synchronised and legacy, and inherits `Vector` 's index methods, breaking LIFO. `LinkedList` : allocates a node per element, but allows nulls. Note that `Deque.push` adds at the front, so iteration runs LIFO, the reverse of `Stack` 's iteration order.

Q47. Why was `Vector` replaced?

Every method synchronises, so uncontended calls still pay, while compound actions like `if (!v.contains(x)) v.add(x)` still race. It also doubles rather than growing 1.5x. Modern alternatives are `ArrayList` with explicit locking, `CopyOnWriteArrayList` , or a concurrent queue.

Q48. What is the difference between `size()` and capacity? Can you read a list's capacity?

Size counts elements and capacity is the array's length. `ArrayList` has no public capacity accessor. `get(i)` with `i >= size` throws `IndexOutOfBoundsException`, even when capacity is sufficient.

Q49. `Iterator` vs `ListIterator` vs `Spliterator`?

`Iterator` moves forward and removes. `ListIterator` moves both ways, with indices, `set` and `add`. `Spliterator` (Java 8) supports `trySplit` for parallel streams, plus characteristics like `SIZED`, `ORDERED` and `DISTINCT` that let streams skip work. `ArrayList`'s splitter divides the array range in half, which is why it parallelises much better than `LinkedList`.

Q50. Are `keySet()`, `values()` and `entrySet()` copies?

No. They are live views created lazily and cached in the map. `map.keySet().remove(k)` removes from the map, while `add` throws `UnsupportedOperationException`. To keep a stable snapshot, copy with `List.copyOf(map.keySet())`.

Q51. How do you iterate a map efficiently?

Use `entrySet()` or `map.forEach((k, v) -> ...)`. Iterating `keySet()` and calling `get` per key hashes each key a second time, wasting CPU. All iteration is O(capacity + size), so a `HashMap` that once grew huge and then shrank iterates slowly even when nearly empty, as `remove` never shrinks the table. Create a new map to compact.

Q52. What does `computeIfAbsent` do, and why does it beat get-then-put?

It returns the existing value, or computes, stores and returns a new one, with one hash lookup. `map.computeIfAbsent(k, x -> new ArrayList<>()).add(v)` is the idiomatic multimap. On a `HashMap`, the function must not modify the map, or it throws `ConcurrentModificationException` since Java 9. A function that returns `null` stores nothing.

Q53. `merge` vs `compute`?

`merge(key, value, fn)` puts `value` when absent, and otherwise applies `fn(old, value)`, removing if it returns null. It suits counters: `map.merge(word, 1, Integer::sum)`. `compute(key, fn)` always runs with the key and its possibly null value, for more general updates.

Q54. How do you create an unmodifiable map from an existing map? Why might `Map.of` fail at runtime?

`Map.copyOf(m)` for an independent copy, or `Collections.unmodifiableMap(m)` for a view. `Map.of` takes up to 10 pairs, with `Map.ofEntries` for more. It throws `IllegalArgumentException` on duplicate keys and `NullPointerException` on nulls, which can break at runtime when the keys come from variables rather than literals.

Q55. Why does `HashMap`'s iteration order change between runs with enum or object keys, but stay stable with `String` keys?

Order depends on hash codes and table size. `String.hashCode` is defined by a stable formula. `Object.hashCode` and hence enum hash codes are identity-based and vary per JVM. Even so, never rely on the order with `String` keys, since it changes with table size and across JDK releases.

Q56. How much memory does a `HashMap` use, and how would you reduce it?

Each entry costs a `Node` of about 32 bytes with compressed oops, plus a table slot of 4 bytes (roughly 5–8 with unused capacity), plus the key and value objects. A `HashMap<Long, Long>` costs on the order of 80–100 bytes per entry to hold 16

bytes of real data. For huge maps use primitive collections (fastutil, Eclipse Collections, Agrona), arrays with sorted keys, or off-heap storage. Or, as RefinePoint does for large files, process the data in DuckDB instead of on the Java heap.

Q57. A service's p99 latency spikes periodically. How could a `HashMap` be involved?

Resizing a very large map copies and reallocates on a single `put`, so an ever-growing cache causes spikes and growing GC pressure from a huge table array, which may be a humongous allocation in G1. A map with poor hashing may show slow lookups, visible in a profiler as `TreeNode.find` or `equals`. Check with JFR or async-profiler, then presize, bound the cache, or use a concurrent one like Caffeine.

Q58. A thread dump shows threads stuck in `HashMap.get` at 100% CPU. What is happening?

On Java 7 or earlier, a concurrent resize created a cycle. On Java 8+, unsynchronised concurrent use can still produce a corrupted tree bin that loops in `TreeNode` methods such as `balanceInsertion` or `root()`. Either way the cause is a shared `HashMap` written by several threads. Restart to recover, then replace it with `ConcurrentHashMap` and find the sharing, commonly a singleton Spring bean with a map field.

Q59. You see `OutOfMemoryError`, and a heap dump shows one huge `HashMap`. What do you check?

Whether it's an unbounded cache with no eviction, a key that does not implement `equals` / `hashCode` so that "same" keys accumulate, or mutable keys leaking entries. Look at its retained size in Eclipse MAT, and the actual key values, then cap it with `TtlCache`-style limits, Caffeine, or a `LinkedHashMap` LRU. Static collections and singleton beans are the typical holders.

Q60. How does `Collections.synchronizedList` handle iteration?

Individual methods lock a mutex, but iteration doesn't, so the caller must wrap it: `synchronized (list) { for (var x : list) ... }`, or else it can throw CME. The same goes for compound actions. This is the core reason concurrent collections exist.

Q61. How did you use these collections in your project?

"In `FxRates` we build a `HashMap<String, TreeMap<LocalDate, BigDecimal>>` keyed by currency pair, using `computeIfAbsent` to create the inner map. In the engine, row lists are presized to the known count. The gateway's rate limiter uses a `ConcurrentHashMap` of string keys, evicting idle entries with `entrySet().removeIf`, and the rule parser's cache is a bounded `LinkedHashMap`. The general rule I follow is that key types are immutable, sizes are known up front where possible, and each cache has a bound."

5. Collections Internals II: Sorted, Linked, Concurrent & Specialised

Chapter 4 covered the collections everybody uses. This chapter covers the ones that separate a senior engineer: the sorted maps, the linked and reference-based maps, the heap behind `PriorityQueue`, the blocking queues behind every thread pool, and `ConcurrentHashMap`, whose Java 8 rewrite is one of the most-asked internals questions after `HashMap` itself. It ends with a decision table and with the concurrent structures RefinePoint actually runs: rate-limit buckets, a TTL cache, a sliding-window limiter and LRU caches.

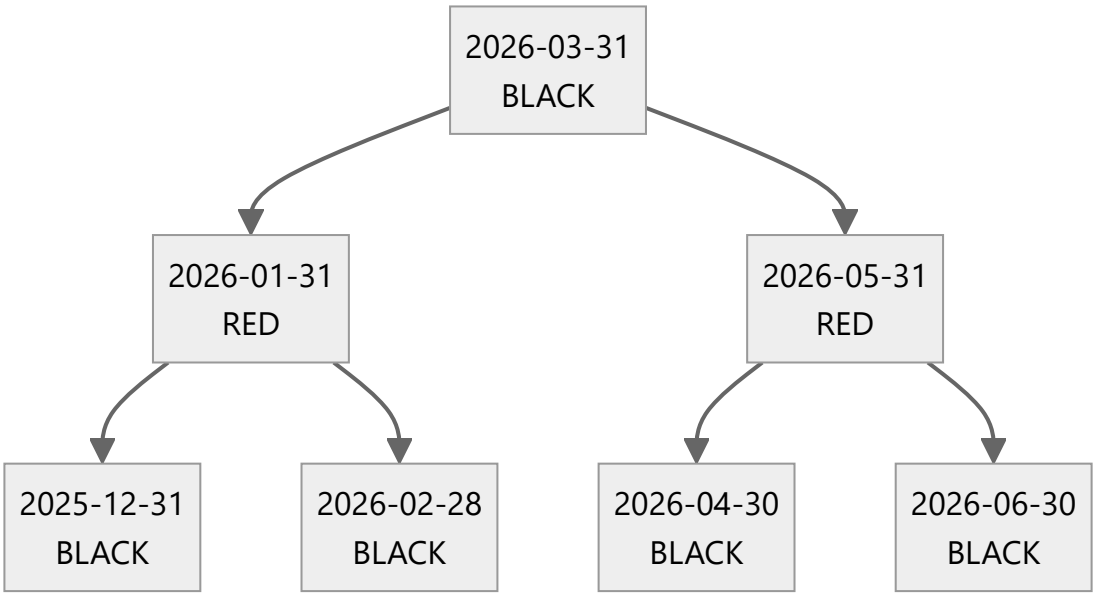
5.1 TreeMap and the red-black tree

`TreeMap` is a **red-black tree**, a self-balancing binary search tree. Each `Entry` holds `key`, `value`, `left`, `right`, `parent` and a `color` boolean. Keys are ordered by `compareTo` or by the comparator passed to the constructor, and **the comparator defines equality**: `equals` is never called.

The red-black rules keep the tree balanced:

1. Every node is red or black, and the root is black.
2. A red node has no red child.
3. Every path from a node down to its null leaves passes the same number of black nodes (the *black height*).

Together these guarantee that the longest path is at most twice the shortest, so the height is at most `2 log2(n + 1)`, and `get`, `put` and `remove` are O(log n) in the worst case, not just on average.



Insertion: insert as in a plain binary search tree, colour the new node red, then repair. If the parent is red, either recolour (the uncle is red: push the blackness down from the grandparent) or rotate (the uncle is black: at most two rotations). Deletion needs at most three rotations. Rotations change a few pointers, so they are O(1); recolouring can walk up the tree, which is O(log n).

Operation	TreeMap	HashMap
get / put / remove	O(log n) worst case	O(1) average
Ordered iteration	Yes, in key order	No
Range queries, floor and ceiling	Yes	No
Memory per entry	~40 bytes plus key and value	~32 bytes plus table slot
Null key	No with natural order (NPE); a comparator may allow it	One allowed
Key needs	`Comparable` or a comparator	`equals` and `hashCode`

Since Java 7, the first `put` into an empty `TreeMap` calls `compare(key, key)`, so a non-comparable or `null` key fails immediately instead of on the second insert.

NavigableMap

`TreeMap` implements `NavigableMap` (Java 6), which adds the operations that make sorted maps worth their cost:

```
NavigableMap<LocalDate, BigDecimal> rates = new TreeMap<>();
rates.floorEntry(date);        // greatest key <= date ("rate on or before")
rates.ceilingKey(date);        // least key >= date
rates.lowerKey(date);          // strictly less
rates.higherKey(date);         // strictly greater
rates.headMap(date, true);     // view of keys <= date
rates.tailMap(date, false);    // view of keys > date
rates.subMap(from, true, to, false);
rates.descendingMap();         // reverse-order view
rates.pollFirstEntry();        // remove and return the smallest
```

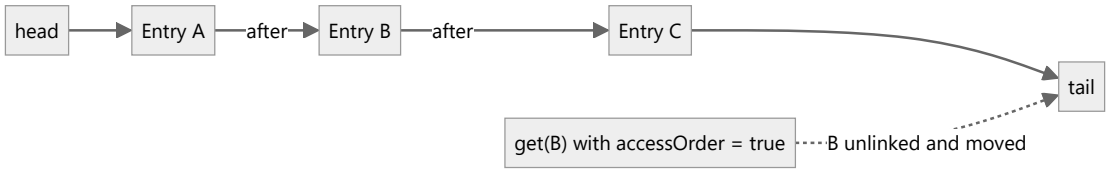
Range views are **live**: they write through to the map, and inserting a key outside a view's range through that view throws `IllegalArgumentException`.

> **RefinePoint:** `FxRates.of(quotes)` builds a `HashMap<String, TreeMap<LocalDate, BigDecimal>>`, one tree per currency pair, and answers "the rate on or before this date" with `floorEntry(date)`. A rate published on Friday is the right rate for a Sunday transaction, and the lookup is O(log n) without any scanning. Elsewhere, `RowChecker` uses `new TreeMap<>(String.CASE_INSENSITIVE_ORDER)` so that field names match regardless of case, and `JobRegistry` keeps handlers in a `TreeMap` so that the job list always appears in the same order.

`TreeSet` is backed by a `TreeMap` exactly as `HashSet` is backed by a `HashMap`.

5.2 LinkedHashMap and the LRU cache

`LinkedHashMap` extends `HashMap` and threads a **doubly linked list** through all its entries. Its `Entry` extends `HashMap.Node` with `before` and `after` pointers, and the map keeps `head` and `tail`. Buckets still give O(1) lookup, and the list gives predictable iteration order.



Mode	Constructor	Order	Use
Insertion order (default)	`new LinkedHashMap<>()`	Order keys were first inserted; re-putting a key does not move it	Predictable JSON, reports, stable output
Access order	`new LinkedHashMap<>(16, 0.75f, true)`	Least recently accessed first; `get`, `put`, `getOrDefault`, `compute` move the entry to the tail	LRU caches

`HashMap` calls three hooks that `LinkedHashMap` overrides: `afterNodeAccess` (move to tail in access order), `afterNodeInsertion` (offer eviction) and `afterNodeRemoval` (unlink). After each insertion it calls `removeEldestEntry(head)`, which returns `false` by default. Override it and you have an LRU cache:

```
// Real, from ryfn-expression's ExpressionEngine: parsed rules cached by rule text
private final Map<String, Parsed> cache = Collections.synchronizedMap(new LinkedHashMap<>(256, 0.75f, true) {
    @Override
    protected boolean removeEldestEntry(Map.Entry<String, Parsed> eldest) {
        return size() > CACHE;      // CACHE = 2000
    }
});
```

RefinePoint: parsing a rule (lexer, parser, AST, compile) is far more expensive than looking it up, and a run evaluates the same few rules millions of times. The access-ordered `LinkedHashMap` keeps the 2,000 most recently used parsed rules, and `ai-service`'s `SchemaInsightService` uses the same pattern for schema insights. Both are wrapped in `Collections.synchronizedMap` because they are singleton beans called from many request threads.

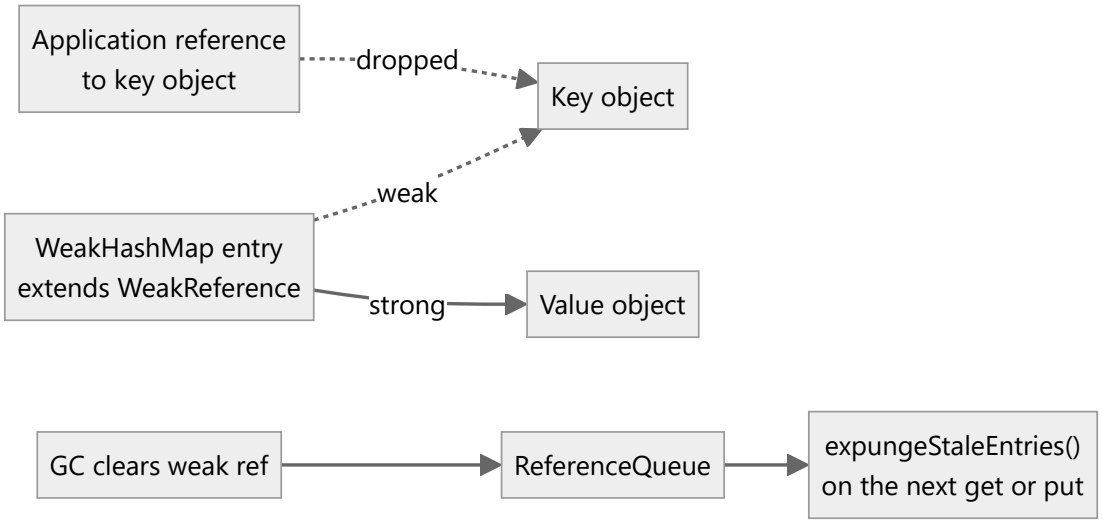
Trap: in access-order mode `get` **is a structural modification**: it relinks the list and increments `modCount`. Iterating over the map while calling `get` throws `ConcurrentModificationException`, and two threads calling `get` on an unsynchronised access-ordered map can corrupt the list. That is why the wrapper above is not optional. For a heavily concurrent cache, use Caffeine, which approximates LRU (W-TinyLFU) without a global lock.

5.3 IdentityHashMap, WeakHashMap and EnumMap

`IdentityHashMap` compares keys with `==` and hashes with `System.identityHashCode`, deliberately breaking the `Map` contract. Internally it uses **open addressing with linear probing** in a single `Object[]` in which keys and values alternate (`table[i]` is a key, `table[i + 1]` its value), so there are no entry objects. Uses: graph algorithms that must track visited objects (serialisers, deep-copy, proxies), where two equal-but-distinct objects must count separately, and where calling `equals` on every node would be slow or wrong.

RefinePoint: the engine's `Controls` deduplicates matched groups with `Collections.newSetFromMap(new IdentityHashMap<>())`. A group is a `List<Rec>` shared by every record in it, so "have I seen this group?" is an identity question. A `HashSet<List<Rec>>` would hash and compare whole lists, element by element, for every record.

`WeakHashMap` holds its **keys** through `WeakReference`s registered with a `ReferenceQueue`. When a key is no longer strongly reachable, the GC clears the reference, and the next map operation calls `expungeStaleEntries()`, which removes the entry.



Traps: values are held **strongly**, so a value that refers to its own key keeps the key alive forever; `String` literals and small boxed integers are always reachable (the pool and caches hold them), so they never disappear; and it is not thread-safe. `ThreadLocal`'s internal `ThreadLocalMap` uses the same weak-key idea, with the same trap for values, which is the root of the classic thread-pool `ThreadLocal` leak. For caches, weak keys are rarely what you want; use Caffeine with size and expiry bounds.

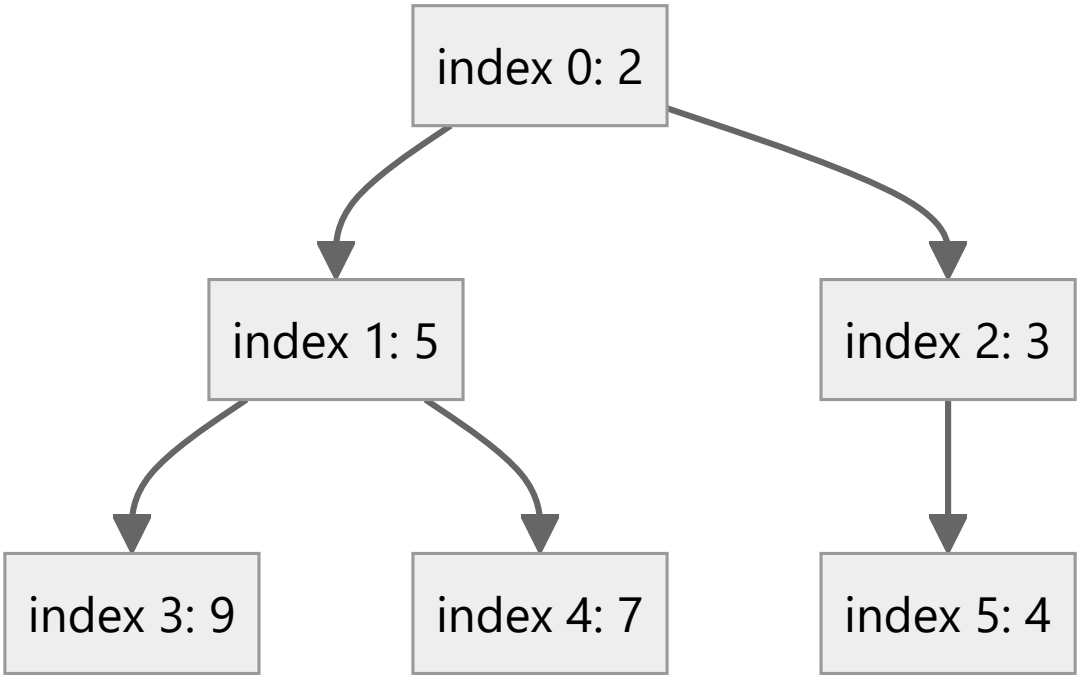
`EnumMap` (covered in chapter 3) is an array indexed by `ordinal()`: no hashing, no entries, iteration in declaration order, no null keys. Use it whenever the keys are an enum.

5.4 PriorityQueue: a binary heap in an array

`PriorityQueue` is a **binary min-heap** stored in `Object[] queue`. The smallest element (by natural order or comparator) is at index 0, and each parent is no greater than its children. No pointers are needed:

```
parent(i) = (i - 1) >>> 1       left(i) = 2i + 1       right(i) = 2i + 2

array:  [ 2 | 5 | 3 | 9 | 7 | 4 ]
index:    0   1   2   3   4   5
```



Operation	Cost	How
`offer` / `add`	O(log n)	Append at the end, **sift up** while smaller than the parent
`poll`	O(log n)	Take index 0, move the last element there, **sift down** by swapping with the smaller child
`peek`	O(1)	Index 0
`remove(Object)`, `contains`	O(n)	Linear scan
Build from a collection	O(n)	Floyd's heapify, sifting down from the middle

Iteration is not sorted: the iterator walks the array, which is only partially ordered. Printing a `PriorityQueue` shows heap order, a favourite output-prediction trick. To consume in order, `poll` repeatedly. Nulls are rejected, and the queue is not thread-safe; `PriorityBlockingQueue` is the concurrent version. Capacity starts at 11 and grows by doubling while small (under 64), then by 50%.

Top-K pattern: to keep the k largest of n items, use a **min-heap of size k**. Offer each item, and when the size exceeds k, `poll` the smallest. That is O(n log k) time and O(k) memory, instead of sorting everything.

```
PriorityQueue<Break> top = new PriorityQueue<>(Comparator.comparing(Break::absAmount));
for (Break b : breaks) {
    top.offer(b);
    if (top.size() > 10) top.poll();          // drop the smallest of the 11
}
```

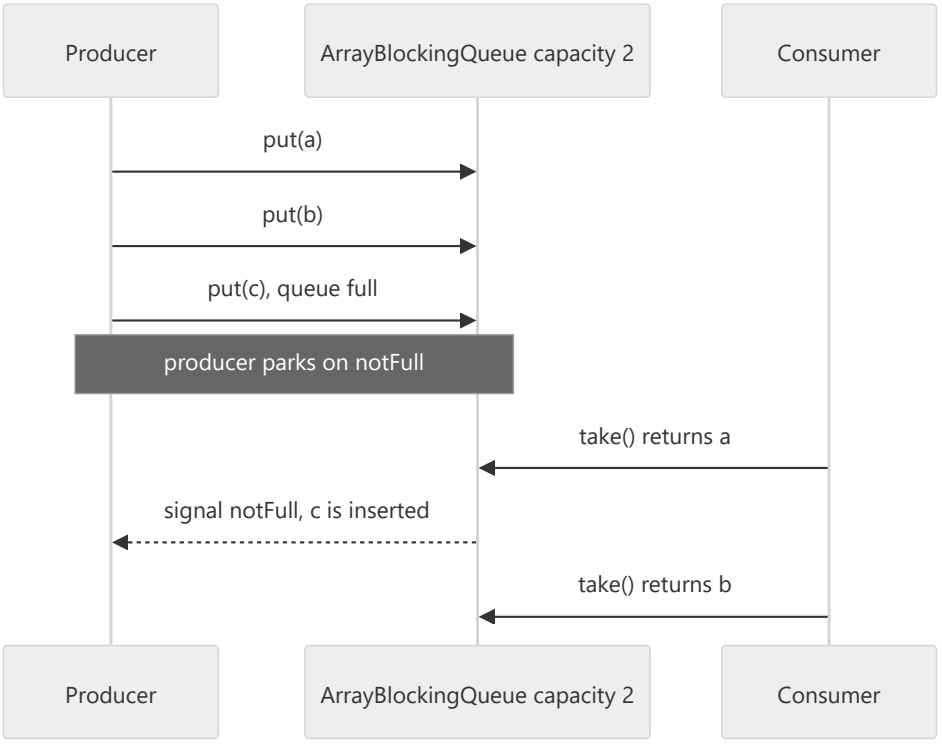
5.5 The BlockingQueue family

A `BlockingQueue` adds operations that wait for space or for an element, which makes it the standard way to connect producers and consumers, and the work queue inside every `ThreadPoolExecutor`.

	Throws	Returns special value	Blocks	Times out
Insert	`add(e)`	`offer(e)` returns false	`put(e)`	`offer(e, time, unit)`
Remove	`remove()`	`poll()` returns null	`take()`	`poll(time, unit)`
Examine	`element()`	`peek()`	n/a	n/a

No `BlockingQueue` accepts `null`, because `null` is the "nothing there" value of `poll`.

Implementation	Bounded	Internals	Notes
`ArrayBlockingQueue`	Yes, fixed at creation	Circular array, **one** `ReentrantLock`, conditions `notEmpty` and `notFull`	Optional fairness; predictable memory; producers and consumers contend on one lock
`LinkedBlockingQueue`	Optional, **default** `Integer.MAX_VALUE`	Linked nodes, **two locks** (`putLock` , `takeLock`), `AtomicInteger count`	Producers and consumers rarely block each other, so higher throughput; one node allocated per element
`SynchronousQueue`	Capacity **zero**	Direct hand-off; each `put` waits for a `take`	Used by `Executors.newCachedThreadPool`
`PriorityBlockingQueue`	No	Array heap under one lock	`take` returns the smallest; `put` never blocks
`DelayQueue`	No	`PriorityQueue` of `Delayed` elements plus a **leader** thread	`take` returns an element only once its delay has expired
`LinkedTransferQueue`	No	Lock-free dual queue	`transfer(e)` waits until a consumer receives it
`LinkedBlockingDeque`	Optional	Linked, one lock	Blocking at both ends; work stealing designs



The DelayQueue leader pattern: only one thread (the leader) waits with a timed `awaitNanos` for the head's delay; other takers wait indefinitely. When the head expires, the leader takes it and signals the next waiter to become leader. That avoids every waiting thread waking up for the same deadline.

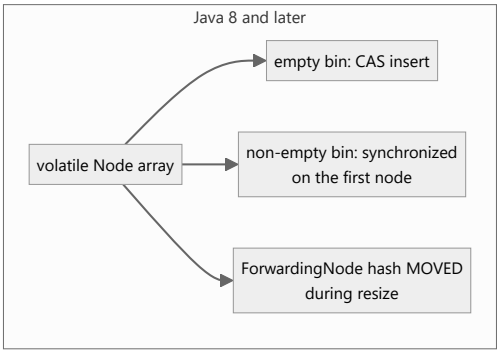
How the queue shapes a `ThreadPoolExecutor` : the pool grows to `corePoolSize` , then **queues**, and only creates threads beyond the core (up to `maximumPoolSize`) when `offer` to the queue fails. Consequences:

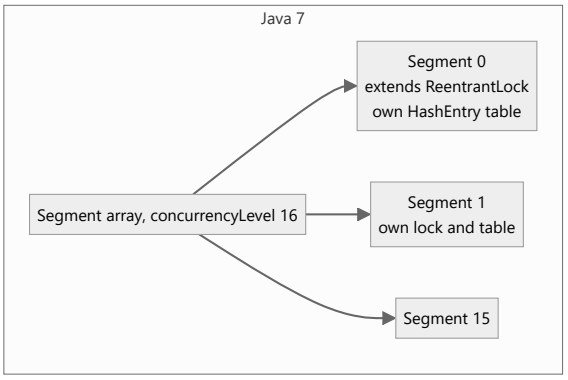
- With an unbounded `LinkedBlockingQueue` (what `Executors.newFixedThreadPool` uses), `maximumPoolSize` is never used, and a slow consumer lets the queue grow until `OutOfMemoryError` .
- With a `SynchronousQueue` (what `newCachedThreadPool` uses), every task that finds no idle thread creates a new one, which is unbounded threads instead.
- In production, use a **bounded** queue with an explicit rejection policy (`CallerRunsPolicy` gives natural back-pressure).

> **RefinePoint:** the durable work queues (runs, jobs, exports, data-source refreshes) are **PostgreSQL tables** claimed with `FOR UPDATE SKIP LOCKED`, not in-memory `BlockingQueue`s. A queue in the heap disappears on restart and cannot be shared by several instances; a table survives both, and adding instances adds consumers. In memory, each worker only tracks what it is running, in a `ConcurrentHashMap.newKeySet()` of run IDs, checked against its slot count before it claims more.

5.6 ConcurrentHashMap internals

Java 7: segments



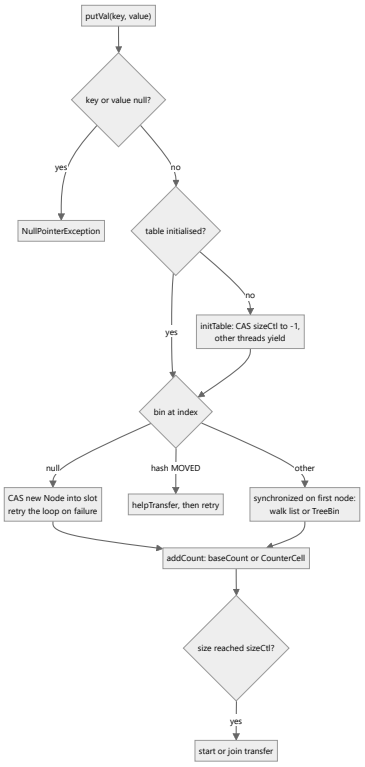


Java 7's map was an array of `Segment`s, each a small hash table that extended `ReentrantLock`. A write locked one segment, so up to `concurrencyLevel` (default 16) writers could proceed in parallel; reads were mostly lock-free through `volatile` fields. `size()` summed the segments twice without locking, compared their `modCount`s, and on disagreement locked every segment. The concurrency level was fixed at creation, and each segment cost memory even when empty.

Java 8: CAS plus per-bin locks

Java 8 rewrote it on the same table-of-bins layout as `HashMap`, with the lock granularity reduced to **one bin**:

- **Empty bin:** insert with a single **CAS** (`casTabAt`), with no lock at all.
- **Non-empty bin:** `synchronized` on the bin's **first node**, then walk the list or tree as `HashMap` does. Two writers conflict only if they hit the same bin.
- **Reads take no lock.** `table` is `volatile`, each `Node`'s `val` and `next` are `volatile`, and bins are read with acquire semantics (`tabAt`). A `get` sees a fully constructed node or none.
- **Negative hashes are reserved** for special nodes: `MOVED = -1` (a `ForwardingNode` during resize), `TREEBIN = -2` (the head of a treeified bin), `RESERVED = -3` (a placeholder used by `computeIfAbsent` and `compute`). `spread(h)` is `(h ^ (h >>> 16)) & 0x7fffffff`, so user hashes are never negative.
- **Tree bins** use the same 8 and 64 thresholds as `HashMap`, but are wrapped in a `TreeBin` with its own small read-write lock, so that readers can fall back to walking the `next` links while a writer is rebalancing.



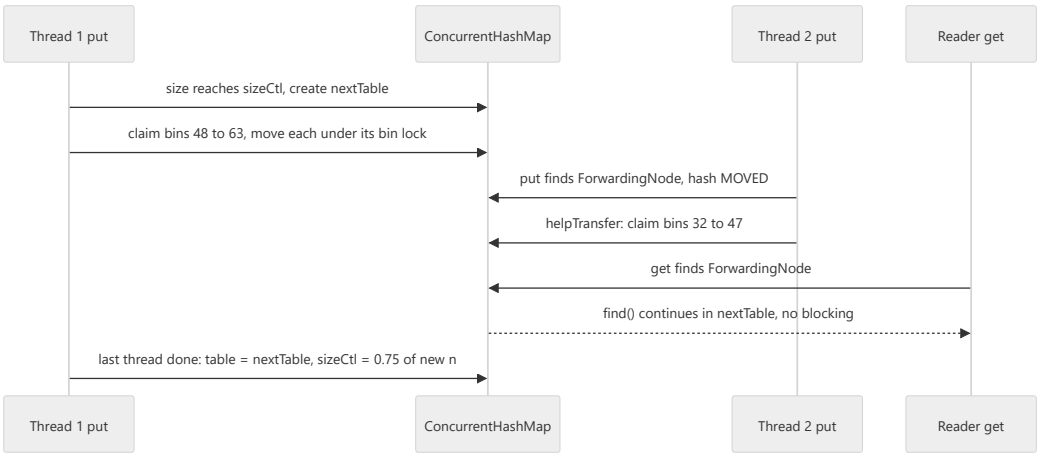
`sizeCtl`

One `volatile int` coordinates initialisation and resizing:

Value	Meaning
`0`	Table not created yet; use the default capacity of 16
Positive, before initialisation	The initial capacity to use
Positive, after initialisation	The next resize threshold, `n - (n >>> 2)`, which is 0.75n
`-1`	A thread is initialising the table
Other negative	A resize is in progress: the high bits hold a *resize stamp* for this table size, the low bits one more than the number of resizing threads

Resize: transfer and helping

Resizing is done **cooperatively**. The thread that crosses the threshold allocates `nextTable` (double size) and starts moving bins from the highest index downwards, claiming a *stride* of bins at a time through a CAS on `transferIndex` (at least 16 bins per claim). Every moved bin is replaced with a `ForwardingNode` pointing at the new table.



Each bin is split into a low and a high list exactly as in `HashMap` (`hash & n`), with a `lastRun` optimisation that reuses the unchanged tail of the list. Readers are never blocked: a `get` that hits a `ForwardingNode` simply continues its search in the new table. Writers that hit one help with the transfer instead of waiting, so a resize under heavy write load is finished by all writers together.

Counting: `baseCount` and `CounterCell`s

A single `AtomicLong` size would make every insert contend on one memory location. `addCount` first tries a CAS on `baseCount`; if that fails because of contention, it adds to one of an array of `CounterCell`s (padded with `@Contended` against false sharing), chosen by a per-thread probe value. That is the same design as `LongAdder`. `size()` and `mappingCount()` return `baseCount` plus the sum of all cells, which is an **estimate** under concurrent updates. `mappingCount()` returns a `long` and is preferred; `size()` caps at `Integer.MAX_VALUE`.

Why no nulls?

In a concurrent map, `get(k) == null` could mean "absent" or "mapped to null", and the usual way to tell, `containsKey(k)`, is a second call that can race with another thread's change. Banning nulls makes `null` an unambiguous "absent", which is what `putIfAbsent`, `computeIfAbsent` and `getOrDefault` rely on. Doug Lea has also said that allowing nulls even in non-concurrent maps was a mistake.

Atomic compound operations

Individual operations are thread-safe, but a sequence of them is not:

```
if (!map.containsKey(k)) map.put(k, v);           // race: two threads both put
map.put(k, map.get(k) + 1);                       // race: lost updates

map.putIfAbsent(k, v);                             // atomic
map.merge(k, 1, Integer::sum);                     // atomic increment
map.computeIfAbsent(k, x -> new LongAdder()).increment();   // best for hot counters
map.replace(k, expectedOld, newValue);             // compare-and-set per key
```

`computeIfAbsent` pitfalls

`computeIfAbsent` guarantees that the function runs **at most once per absent key**. It does that by holding the bin lock (or a `ReservationNode` in an empty bin) **while the function runs**. Consequences:

1. **A slow function blocks other keys.** Every other write that hashes to the same bin waits, and so does any thread computing the same key. Never do network I/O or database calls inside it.
2. **No recursive updates.** A function that modifies the same map (for example, a memoised recursive Fibonacci) can deadlock or corrupt the map. Java 8 could spin forever (JDK-8062841); Java 9+ detects most cases and throws

`IllegalStateException("Recursive update")`.

3. **Java 8 locked even on a hit.** It took the bin lock when the key was already present. Java 9 checks the first node of the bin without locking, and a `get` first is still a cheap fast path for hot keys on Java 8.
4. **Returning `null` stores nothing**, so the function will run again on the next call.

Weakly consistent iteration and bulk operations

Iterators and the `keySet`, `values` and `entrySet` views are **weakly consistent**: they never throw `ConcurrentModificationException`, they traverse elements as they existed at some point since the iterator was created, they may or may not reflect later updates, and they return each element at most once. `putAll` and `clear` are not atomic. Java 8 added parallel bulk operations, `forEach`, `search`, `reduce` and their typed variants, which take a `parallelismThreshold` and run on the common fork-join pool. `ConcurrentHashMap.newKeySet()` gives a concurrent `Set`.

5.7 CopyOnWriteArrayList, ConcurrentSkipListMap and the other concurrent collections

`CopyOnWriteArrayList` holds a `volatile Object[]`. Every write takes a lock, copies the whole array, modifies the copy and publishes it with one volatile write. Reads and iteration take no lock and work on whatever array they started with, so the iterator is a **snapshot**: it never throws CME, never sees later changes, and does not support `remove`.

	Reads	Writes	Iteration	Best for
`CopyOnWriteArrayList`	Lock-free, O(1)	O(n) copy under a lock	Snapshot	Listener lists, configuration: read very often, written rarely
`Collections.synchronizedList`	Locked	Locked, O(1) amortised	Must lock manually	Moderate, balanced access

A list of 100,000 elements written 1,000 times a second copies 100 million references a second: it is the wrong tool for write-heavy lists. `CopyOnWriteArraySet` is backed by a `CopyOnWriteArrayList`, so `contains` is O(n).

`ConcurrentSkipListMap` is a concurrent, sorted `NavigableMap`, built as a **skip list**: a sorted linked list of nodes, plus layers of index lists that skip over progressively more nodes. Each level is chosen randomly, giving expected O(log n) search, and all updates use CAS, with no locks. Nothing equivalent can be built from a red-black tree without locking, because a rebalancing rotation touches several nodes at once.

```
level 3:  head ------------------------------> 50 ----------------------> null
level 2:  head ---------> 20 -----------------> 50 --------> 80 --------> null
level 1:  head -> 10 ---> 20 ---> 30 --------> 50 -> 60 --> 80 -> 90 --> null
search 60: level 3 to 50, level 2 to 50 (80 too far), level 1 to 60
```

Use it for concurrent ordered data: leaderboards, time-ordered indexes, "next deadline" maps. `ConcurrentSkipListSet` is the set version. Its `size()` is only an estimate while other threads update it, and nulls are not allowed.

Others: `ConcurrentLinkedQueue` (lock-free Michael-Scott queue; `size()` is O(n)), `ConcurrentLinkedDeque`, and the blocking queues of section 5.5.

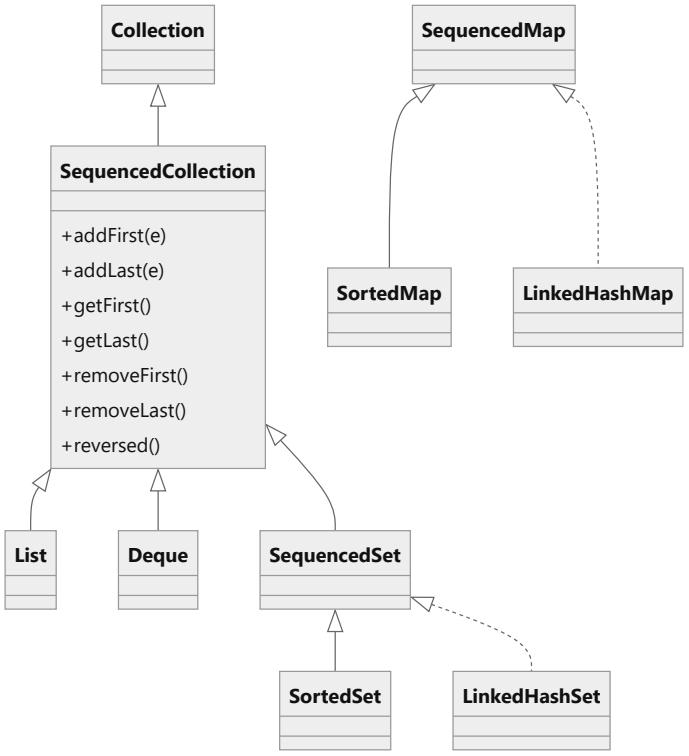
`Collections.synchronizedX` vs concurrent collections

	`Collections.synchronizedMap(m)`	`ConcurrentHashMap`
Locking	One mutex for every call	CAS and per-bin locks; reads lock-free
Read scalability	Readers block each other	Readers never block
Iteration	Fail-fast; caller must hold `synchronized (map)`	Weakly consistent; no lock needed
Compound actions	Caller holds the mutex around them	Atomic methods (`compute` , `merge` , …)
Nulls	As the wrapped map	Not allowed
Wraps any map	Yes, for example a `LinkedHashMap` LRU	No, it is its own structure

The wrapper is still the right choice when you need the behaviour of a specific map, like the LRU ordering of `LinkedHashMap`, and contention is modest.

5.8 Java 21 sequenced collections

Before Java 21, getting the last element was different for each type: `list.get(list.size() - 1)`, `deque.getLast()`, `sortedSet.last()`, and for a `LinkedHashSet` there was no direct way at all. JEP 431 added three interfaces that give every collection with a defined encounter order one API:



Interface	Methods
`SequencedCollection<E>`	`addFirst` , `addLast` , `getFirst` , `getLast` , `removeFirst` , `removeLast` , `reversed()`
`SequencedSet<E>`	Same, with `reversed()` returning a `SequencedSet`
`SequencedMap<K,V>`	`firstEntry` , `lastEntry` , `pollFirstEntry` , `pollLastEntry` , `putFirst` , `putLast` , `sequencedKeySet` , `sequencedValues` , `sequencedEntrySet` , `reversed()`

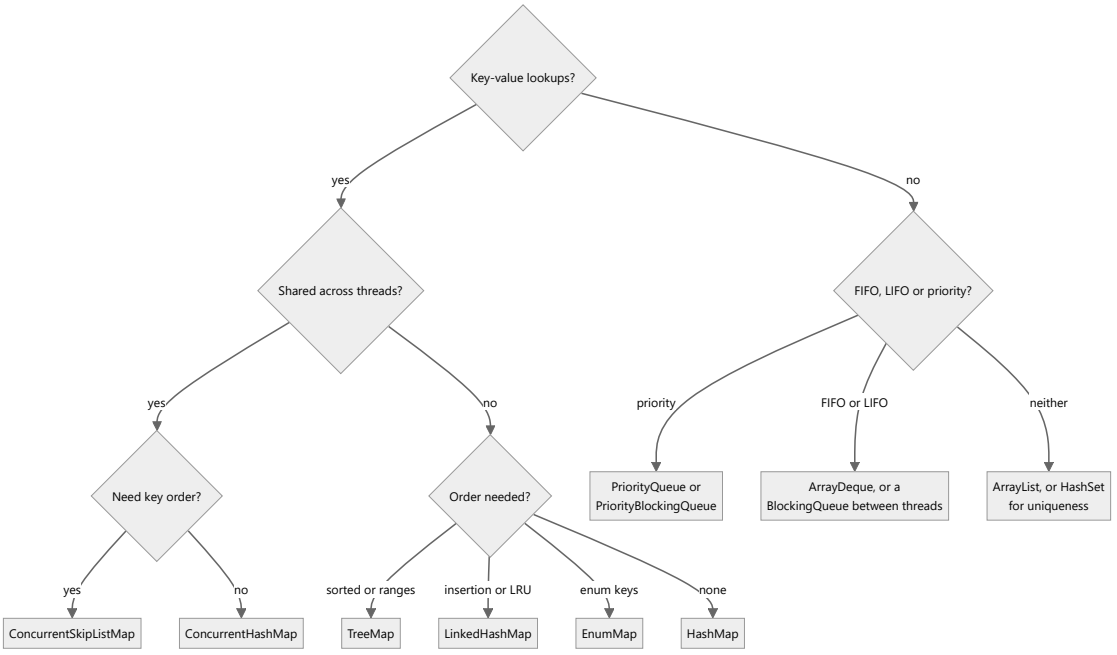
Facts that interviewers check:

- `reversed()` returns a **view**, not a copy, so writes through it reach the original.

- `getFirst()` on an empty collection throws `NoSuchElementException`, unlike `peekFirst()`.
- Sorted collections reject `addFirst`, `addLast`, `putFirst` and `putLast` with `UnsupportedOperationException`, because the comparator decides the position.
- `LinkedHashMap.putFirst` and `putLast` move an existing key; `LinkedHashSet.addFirst` moves an existing element to the front.
- `HashMap`, `HashSet` and `PriorityQueue` are not sequenced, because they have no defined order.
- `Collections.unmodifiableSequencedCollection`, `unmodifiableSequencedSet` and `unmodifiableSequencedMap` were added too.

```
LinkedHashMap<String, Integer> steps = new LinkedHashMap<>();
steps.put("load", 1); steps.put("match", 2); steps.put("report", 3);
steps.firstEntry();            // load=1
steps.pollLastEntry();         // removes report=3
steps.putFirst("validate", 0); // validate now comes first
steps.reversed().keySet();     // [match, load, validate]
```

5.9 Choosing the right collection



Need	Single thread	Shared between threads
Fast lookup by key	`HashMap`	`ConcurrentHashMap`
Sorted keys, ranges, floor and ceiling	`TreeMap`	`ConcurrentSkipListMap`
Insertion order	`LinkedHashMap` / `LinkedHashSet`	`Collections.synchronizedMap(new LinkedHashMap<>())`
LRU cache	`LinkedHashMap` with access order	Caffeine, or a synchronised `LinkedHashMap` at low contention
Enum keys or elements	`EnumMap` / `EnumSet`	Wrap with `synchronizedMap`, or a `ConcurrentHashMap`
Unique elements	`HashSet`	`ConcurrentHashMap.newKeySet()`
List read often, written rarely	`ArrayList`	`CopyOnWriteArrayList`
Stack or queue	`ArrayDeque`	`ConcurrentLinkedQueue` / `ConcurrentLinkedDeque`
Producer-consumer with back-pressure	n/a	`ArrayBlockingQueue` / bounded `LinkedBlockingQueue`
Priority order	`PriorityQueue`	`PriorityBlockingQueue`
Run at a later time	n/a	`DelayQueue`, `ScheduledThreadPoolExecutor`
Identity semantics	`IdentityHashMap`	Rarely needed
Entries that vanish with their key	`WeakHashMap`	Caffeine `weakKeys()`

5.10 RefinePoint: concurrent collections in production code

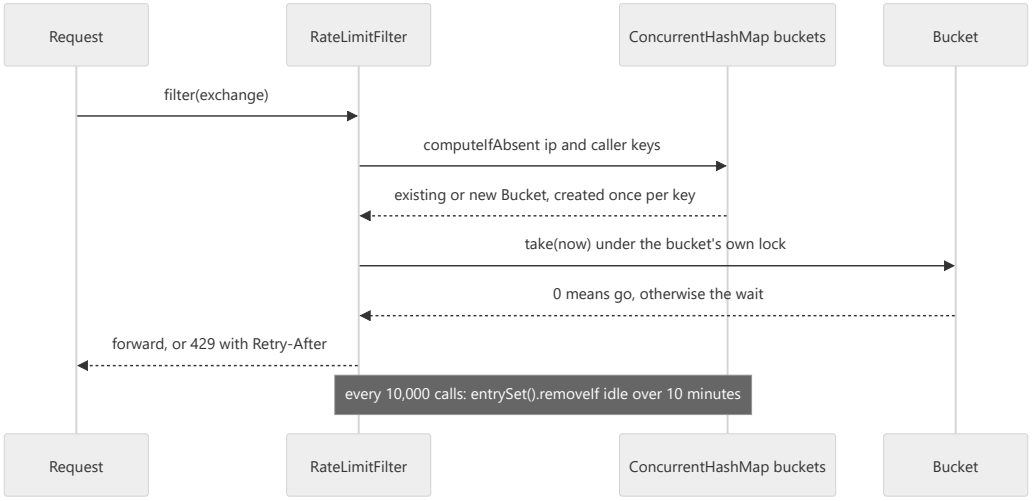
Gateway rate limiting: token buckets in a `ConcurrentHashMap`

`api-gateway`'s `RateLimitFilter` limits each caller per minute: an API key (300 by default), a signed-in user identified by the token's subject (900), and every IP address (1,800). Each limit is a token bucket that holds a minute's requests and refills continuously.

```
private final Map<String, Bucket> buckets = new ConcurrentHashMap<>();

Bucket byAddress = buckets.computeIfAbsent("ip:" + address, k -> new Bucket(perAddress, now));
Bucket own = caller == null ? byAddress : buckets.computeIfAbsent(caller, k -> new Bucket(limit, now));
long wait = Math.max(own.take(now), own == byAddress ? 0 : byAddress.take(now));

static final class Bucket {
    private double tokens; private long last;           // guarded by the bucket's monitor
    synchronized long take(long now) {
        refill(now);                                     // tokens += elapsed * perNano, capped
        if (tokens >= 1) { tokens -= 1; return 0; }
        return (long) Math.ceil((1 - tokens) / perNano); // nanoseconds until a token is due
    }
}
```



Why it is built this way:

- **Two levels of concurrency.** The map only finds or creates the bucket, and `computeIfAbsent` guarantees one bucket per key even when the first requests arrive together. The arithmetic on the bucket is guarded by the bucket's own monitor, so callers with different keys never contend with each other, and the critical section is a few floating-point operations.
- **The construction is cheap**, so holding the bin lock inside `computeIfAbsent` is harmless here.
- **Bounded memory.** Every 10,000 calls, `buckets.entrySet().removeIf(...)` drops buckets idle for over 10 minutes. Weakly consistent iteration means it never throws, even with requests arriving during the sweep.
- **A known, accepted race.** A sweep can remove a bucket that a concurrent request has just fetched. That request takes from a detached bucket, and the next one creates a fresh full bucket, so the limit is briefly lenient for one caller who had been idle for ten minutes. Making it exact would need a lock around every lookup, which isn't worth it.
- **Per instance.** The buckets live in one gateway's heap, so with several gateways each limits what passes through it. A shared limit would need Redis or similar.

`TtlCache`: why it does not use `computeIfAbsent`

`ryfn-platform`'s `TtlCache` caches answers fetched from other services (access decisions, plan limits, API keys) with an expiry time:

```
public V get(K key, Supplier<V> loader) {
    long now = System.currentTimeMillis();
    Entry<V> entry = entries.get(key);
    if (entry != null && entry.expiresAt() > now) return entry.value();
    V value = loader.get();                                  // remote call, outside any lock
    if (ttlMillis > 0) {
        if (entries.size() >= maxEntries) entries.clear();    // crude but safe bound
        entries.put(key, new Entry<>(value, now + ttlMillis));
    }
    return value;
}
```

The loader is an HTTP call to another service. Inside `computeIfAbsent` it would run while holding a bin lock, so one slow call would block every other key in that bin. The trade-off is explicit: two threads that miss at the same moment may both call the loader, which is harmless because the loads are idempotent reads. Clearing the whole map when it is full is cruder than LRU eviction, but it can never grow without bound, and the cost of a clear is only a few extra calls.

`SlidingWindowLimiter`: a `Deque` per key

For limits like AI requests per user per hour and per workspace per day (`UsageLimits` in `ryfn-platform`), SSO lookups, and the public try-it import and confirmation pages, `SlidingWindowLimiter` keeps the timestamps of recent calls per key:

```
private final Map<String, Deque<Long>> calls = new ConcurrentHashMap<>();

Deque<Long> window = calls.computeIfAbsent(key, k -> new ArrayDeque<>());
synchronized (window) {
    while (!window.isEmpty() && now - window.peekFirst() >= windowMillis) window.pollFirst();
    if (window.size() >= max) throw new RateLimitExceededException(..., retryAfterSeconds);
    window.addLast(now);
}
if (calls.size() > 10_000) evictIdle(now);
```

`ArrayDeque` fits exactly: new timestamps go on the tail and expired ones leave from the head, both O(1), with no node allocation. `ArrayDeque` is not thread-safe, so each deque is used only under its own monitor, which again limits contention to one key. Compared with a token bucket, a sliding log is exact (never more than `max` calls in any window), at the cost of storing up to `max` timestamps per key, which is fine for small limits like hourly AI quotas. The same idea appears in `PublicContactController`, which keeps a `Deque<Instant>` per sender to throttle the public contact form.

Other small patterns from the codebase

Code	Collection	Why
`RunWorker`, `ExportWorker`, `RefreshWorker`	`ConcurrentHashMap.newKeySet()` of IDs	Track in-flight work on this instance; compare with slot count before claiming more
`InboxStream`	`ConcurrentHashMap<String, Set<SseEmitter>>`, inner sets from `newKeySet()`	One user can have several open browser tabs; emitters are added and removed from different threads
`BankFeedService`	`locks.computeIfAbsent(consentId, k -> new Object())` with `synchronized`	Per-key locking: refreshes of the same bank consent are serialised, different consents run in parallel
`ExpressionEngine`, `SchemaInsightService`	Synchronised access-order `LinkedHashMap`	Bounded LRU caches
`FxRates`	`TreeMap.floorEntry`	Rate on or before a date
`Controls`	`newSetFromMap(new IdentityHashMap<>())`	Visit each shared group object once

5.11 Interview questions and answers

Q1. How does `TreeMap` work internally?

It is a red-black tree: a binary search tree ordered by `compareTo` or a comparator, where node colours and three rules keep the height at most `2 log2(n + 1)`. `get`, `put` and `remove` are O(log n) in the worst case. Inserts add a red leaf, then fix any red-red violation by recolouring or by at most two rotations. Equality is decided by the comparator returning 0, never by `equals`.

Q2. Why a red-black tree and not an AVL tree?

AVL trees are more strictly balanced (height about 1.44 log n), so lookups are slightly faster, but inserts and deletes rotate more often. Red-black trees need at most two rotations per insert and three per delete, which suits a general-purpose map with mixed reads and writes. `HashMap`'s tree bins use red-black trees for the same reason.

Q3. `HashMap` vs `TreeMap` vs `LinkedHashMap`?

`HashMap`: O(1) average, no order. `LinkedHashMap`: O(1) average plus insertion or access order, at the cost of two extra pointers per entry. `TreeMap`: O(log n), sorted, with range and nearest-key queries, requires comparable keys and forbids `null` keys under natural ordering. Choose by the ordering you need, not by habit.

Q4. Can a `TreeMap` hold a `null` key?

Not with natural ordering: `compareTo` on `null` throws `NullPointerException`, and since Java 7 even the first `put` checks this. With a comparator such as `Comparator.nullsFirst(Comparator.naturalOrder())`, `null` keys work. `null` values are always allowed.

Q5. What happens if a `TreeMap` comparator is inconsistent with `equals`?

The map follows the comparator. Two keys that compare as 0 are the same key, so the second `put` replaces the first's value and keeps the original key. A `TreeMap` with `String.CASE_INSENSITIVE_ORDER` treats "USD" and "usd" as one key, which is useful when intended and a lost-data bug when not.

Q6. What do `floorKey`, `ceilingKey`, `lowerKey` and `higherKey` return?

The greatest key less than or equal to the argument, the least key greater than or equal, the greatest strictly less, and the least strictly greater, each `null` if none exists. They are O(log n). Typical uses: the rate in effect on a date (`floorEntry`), the next scheduled slot (`ceilingKey`), and IP-range lookups.

Q7. Are `headMap`, `tailMap` and `subMap` copies?

No, they are live views backed by the map. Changes in either direction show up in the other, and putting a key outside the view's range through the view throws `IllegalArgumentException`. Copy with `new TreeMap<>(map.headMap(k))` if you need a snapshot.

Q8. How would you implement "find the rate in effect on a date"?

Keep a `TreeMap<LocalDate, BigDecimal>` per currency pair and call `floorEntry(date)`. That is what RefinePoint's `FxRates` does, and it also falls back to the inverse pair, dividing with an explicit scale and `RoundingMode` when only the other direction is quoted. For concurrent updates, `ConcurrentSkipListMap` offers the same `floorEntry`.

Q9. How does `LinkedHashMap` maintain order?

Each entry has `before` and `after` pointers in a doubly linked list with `head` and `tail`. `HashMap`'s hooks keep the list in step: `afterNodeInsertion` appends, `afterNodeRemoval` unlinks, and in access-order mode `afterNodeAccess` moves the accessed entry to the tail. Iteration walks the list, so it costs O(size), not O(capacity) as in `HashMap`.

Q10. Does re-inserting an existing key change `LinkedHashMap`'s order?

Not in insertion-order mode: the key keeps its original position. In access-order mode, the `put` counts as an access and moves it to the tail. Java 21's `putLast` and `putFirst` move it explicitly in either mode.

Q11. Implement an LRU cache in Java.

Extend `LinkedHashMap` with access order and override `removeEldestEntry`:

```
class Lru<K, V> extends LinkedHashMap<K, V> {
    private final int max;
```

```
    Lru(int max) { super(16, 0.75f, true); this.max = max; }
    @Override protected boolean removeEldestEntry(Map.Entry<K, V> e) { return size() > max; }
}
```

Every operation is O(1). It is not thread-safe, so wrap it with `Collections.synchronizedMap` for modest concurrency, as RefinePoint's `ExpressionEngine` does for its 2,000 parsed rules, or use Caffeine for high concurrency. Interviewers sometimes ask for it from scratch: a `HashMap<K, Node>` plus a hand-written doubly linked list with sentinel head and tail nodes.

Q12. Why can iterating an access-ordered `LinkedHashMap` throw CME even without any `put`?

In access-order mode `get` moves the entry and increments `modCount`, so it is a structural modification. Calling `get` inside a loop over the map, or a second thread calling `get`, invalidates the iterator. It is also why such a map needs locking even for "read-only" use from several threads.

Q13. What is `IdentityHashMap` and when would you use it?

A map that compares keys with `==` and hashes with `System.identityHashCode`, stored with linear probing in one array of alternating keys and values. Use it when identity matters more than equality: tracking visited objects in graph traversal, serialisation or deep copy, or proxies. It intentionally violates the general `Map` contract, so never expose it as an ordinary `Map` to code that expects `equals` semantics.

Q14. How does `WeakHashMap` remove entries?

Each entry is a `WeakReference` to its key, registered with a `ReferenceQueue`. When the GC finds the key only weakly reachable, it clears the reference and queues the entry. The next `get`, `put` or `size` calls `expungeStaleEntries`, which polls the queue and unlinks those entries. Removal is lazy, so stale entries can sit there until the map is touched.

Q15. Why might entries in a `WeakHashMap` never disappear?

The value strongly references the key (directly, or through an object graph), so the key stays reachable from the map itself. Or the keys are always reachable anyway: string literals held by the pool, cached `Integer`s, enum constants, or `Class` objects. Or the map is never touched, so expunging never runs.

Q16. Weak vs soft vs phantom references?

Weak: cleared at the next GC that finds the object only weakly reachable, which suits canonicalising maps and metadata. Soft: cleared only under memory pressure, which looks like a cache but makes heap usage and GC behaviour unpredictable, so explicit size-bounded caches are preferred. Phantom: never returns the referent, and is enqueued after the object is finalised, for cleanup actions; `java.lang.ref.Cleaner` is built on it.

Q17. How is a `PriorityQueue` implemented?

A binary min-heap in an `Object[]`: the element at index `i` has children at `2i + 1` and `2i + 2` and is no greater than them. `offer` appends and sifts up, `poll` moves the last element to the root and sifts down, both O(log n). `peek` is O(1), `contains` and `remove(Object)` are O(n), and building from a collection is O(n) heapify.

Q18. Predict the output.

```
PriorityQueue<Integer> pq = new PriorityQueue<>(List.of(5, 1, 4, 2, 3));
System.out.println(pq);
while (!pq.isEmpty()) System.out.print(pq.poll() + " ");
```

The first line prints the heap array, such as `[1, 2, 4, 5, 3]`, which is not sorted. The loop prints `1 2 3 4 5`. Only `poll` returns elements in priority order; `toString`, iterators and streams follow the array.

Q19. How do you make a max-heap? How do you find the top K elements?

`new PriorityQueue<>(Comparator.reverseOrder())` gives a max-heap. For the top K of N, keep a **min**-heap of size K and poll whenever it exceeds K, which leaves the K largest: O(N log K) time and O(K) memory. Sorting everything is O(N log N), and a max-heap of all N elements uses O(N) memory.

Q20. What does each `BlockingQueue` method do when the queue is full or empty?

`add` and `remove` throw; `offer` returns `false` and `poll` returns `null`; `put` and `take` block; `offer` and `poll` with a timeout wait up to that long. `peek` and `element` examine the head without removing it. None of them accept `null`.

Q21. `ArrayBlockingQueue` vs `LinkedBlockingQueue`?

`ArrayBlockingQueue`: bounded, fixed array, one lock shared by producers and consumers, no allocation per element, optional fairness. `LinkedBlockingQueue`: optionally bounded, but unbounded (`Integer.MAX_VALUE`) by default; one node per element, but separate put and take locks, so producers and consumers do not contend, which usually gives higher throughput. Always give `LinkedBlockingQueue` a capacity in production code.

Q22. Why does `LinkedBlockingQueue` use two locks but `ArrayBlockingQueue` only one?

In a linked queue, producers touch only the tail and consumers only the head, and an `AtomicInteger` count keeps them in step, so the two ends can be locked separately (the two-lock queue algorithm). In a circular array, both ends share one array and index arithmetic that depends on the count, so a single lock is simpler. Benchmarks show the array version often competes well at moderate contention because it avoids allocation.

Q23. What is a `SynchronousQueue`? Where is it used?

A queue with no capacity: every `put` waits until a `take` receives the element, which makes it a direct hand-off. `Executors.newCachedThreadPool` uses it, so a task submitted when no thread is idle makes the pool create a new thread rather than queue it. That is why a cached pool can create thousands of threads under load.

Q24. How does `DelayQueue` work?

It is an unbounded queue of `Delayed` elements kept in a `PriorityQueue` ordered by remaining delay. `take` returns the head only when its delay has expired. A single leader thread waits with a timeout for the head, while other takers wait indefinitely, which avoids every waiting thread waking for the same deadline. It suits simple timeouts or retry-after-delay in a single JVM; for durable scheduling use a database table or a scheduler.

Q25. How does the work queue affect `ThreadPoolExecutor` behaviour?

Up to `corePoolSize` threads are created first; then tasks are queued; only when the queue rejects an `offer` are threads added up to `maximumPoolSize`; after that the rejection policy runs. With an unbounded queue the pool never exceeds the core size and the queue can grow until OOM. With a `SynchronousQueue`, every task without an idle thread gets a new thread. A bounded queue with `CallerRunsPolicy` gives back-pressure.

Q26. Why is `Executors.newFixedThreadPool` considered risky in production?

It uses an unbounded `LinkedBlockingQueue`. When producers outpace the threads, tasks pile up in memory until `OutOfMemoryError`, with nothing to slow the producers and nothing in the logs until it is too late. Build a `ThreadPoolExecutor` directly with a bounded queue, a rejection policy, named threads and metrics. In Java 21, I/O-bound tasks may suit virtual threads with a `Semaphore` to bound concurrency instead.

Q27. How did `ConcurrentHashMap` work in Java 7?

It was an array of `Segment`s, each a small hash table extending `ReentrantLock`. A write locked only its segment, so up to `concurrencyLevel` (16 by default) writes could run in parallel; reads were mostly lock-free through `volatile` fields. `size()` summed segments without locking twice and compared modification counts, locking all segments if they differed. The concurrency level was fixed at creation.

Q28. How does `ConcurrentHashMap` work in Java 8 and later?

It uses one table of bins like `HashMap`. Inserting into an empty bin is a single CAS; otherwise the writer synchronises on the bin's first node. Reads take no lock, relying on `volatile` table slots and node fields. Resizing is cooperative, with threads claiming ranges of bins and leaving `ForwardingNode`s behind. Size is tracked with `baseCount` plus striped `CounterCell`s. Long bins treeify into a `TreeBin`, as in `HashMap`.

Q29. Why did Java 8 replace segments?

Segments fixed the maximum write concurrency at creation, cost memory even for small maps, and locked a whole segment for a write that touched one bin. Per-bin locking scales with the table size, the empty-bin CAS needs no lock at all, and `synchronized` on an uncontended object is cheap. The rewrite also added tree bins, cooperative resizing, scalable counting and the functional methods `compute`, `merge` and the bulk operations.

Q30. Is `ConcurrentHashMap.get` ever blocked?

No. `get` reads the table slot with volatile (acquire) semantics and walks nodes whose `val` and `next` are `volatile`. During a resize, a `ForwardingNode` redirects the search to the new table, and in a tree bin under modification the reader walks the linked `next` pointers instead. Readers never take the bin lock.

Q31. What are `MOVED`, `TREEBIN` and `RESERVED`?

Reserved negative hash values marking special nodes. `MOVED` (-1) is a `ForwardingNode`: this bin has been transferred to the new table. `TREEBIN` (-2) is the head of a treeified bin. `RESERVED` (-3) is a placeholder that holds an empty bin while `computeIfAbsent` or `compute` runs the function. User hashes are masked with `0x7fffffff` so they can never collide with these.

Q32. What is `sizeCtl`?

A `volatile int` that controls initialisation and resizing. 0 means use the default capacity; a positive value before initialisation is the requested capacity, and after it the next resize threshold (0.75n); -1 means a thread is initialising; other negative values mean a resize is running, encoding a resize stamp and the number of helping threads. Threads CAS it to coordinate who initialises and who joins a resize.

Q33. How does a `ConcurrentHashMap` resize without stopping everything?

The thread that crosses the threshold creates a table twice the size and starts moving bins from the top index down, claiming strides of at least 16 bins through a CAS on `transferIndex`. Each bin is locked only while it is split into low and high lists, then replaced with a `ForwardingNode`. Writers that hit a forwarding node help with the transfer; readers follow it into the new table. The last thread to finish installs the new table and sets the new threshold.

Q34. How does `ConcurrentHashMap` count its size?

`addCount` tries to CAS `baseCount`; under contention it adds to a `CounterCell` chosen by the thread's probe hash, and the cell array grows when contention persists, as in `LongAdder`. `size()` and `mappingCount()` add `baseCount` and all cells.

Under concurrent updates the result is a moment-in-time estimate, not a linearisable count. Prefer `mappingCount()`, which returns a `long`.

Q35. Why doesn't `ConcurrentHashMap` allow `null` keys or values?

Because `null` must mean "absent". In a concurrent map, distinguishing "absent" from "mapped to null" would need `containsKey` after `get`, and another thread could change the map between the two calls. Without nulls, `get`, `putIfAbsent`, `computeIfAbsent` and `getOrDefault` have unambiguous single-call semantics. Use `Optional` or a sentinel value object if you need to represent "known to be empty".

Q36. Is this code thread-safe with a `ConcurrentHashMap`?

```
if (!map.containsKey(k)) map.put(k, load(k));
map.put(word, map.getOrDefault(word, 0) + 1);
```

No. Each call is atomic, but the check-then-act and read-modify-write sequences are not, so two threads can both load, or both read 5 and both write 6. Use `computeIfAbsent(k, this::load)` (if `load` is quick) and `merge(word, 1, Integer::sum)`. For very hot counters, `computeIfAbsent(k, x -> new LongAdder()).increment()`.

Q37. What are the pitfalls of `ConcurrentHashMap.computeIfAbsent`?

The function runs while the bin is locked, so a slow function blocks writes to every key in that bin, and other threads asking for the same key wait. The function must not modify the map: Java 8 could hang (JDK-8062841), and Java 9+ throws `IllegalStateException("Recursive update")`. On Java 8 it locked even when the key was present, so hot paths often call `get` first. A function returning `null` stores nothing, so it runs again next time. Never put network or database calls inside it.

Q38. Why does RefinePoint's `TtlCache` call `get` and `put` instead of `computeIfAbsent`?

Its loader is a remote call to another service. Inside `computeIfAbsent` that call would hold a bin lock for the length of a network round trip, blocking unrelated keys in the same bin. The chosen trade-off is that two threads missing at the same moment may both load the value, which is harmless because the loads are idempotent reads, and the second `put` just overwrites the first with an equal value.

Q39. What does "weakly consistent iterator" mean?

It never throws `ConcurrentModificationException`, it returns each element at most once, it reflects the map as it was at some point at or after the iterator was created, and it may or may not reflect later changes. So the gateway can sweep idle rate-limit buckets with `entrySet().removeIf(...)` while requests keep adding new buckets.

Q40. Are `ConcurrentHashMap.putAll` and `clear` atomic?

No. They apply entry by entry, so a concurrent reader can see some of the new entries and not others, or a partly cleared map. If you need an atomic swap of the whole content, build a new map and publish it through a `volatile` field or an `AtomicReference`.

Q41. What are the `ConcurrentHashMap` bulk operations?

`forEach`, `search`, `reduce` and typed variants such as `reduceValuesToLong`, each taking a `parallelismThreshold`. If the map has more elements than the threshold, the work is split across the common fork-join pool. `Long.MAX_VALUE` forces sequential execution and `1` maximises parallelism. They run on weakly consistent snapshots, so results reflect concurrent changes only partly.

Q42. How do you create a concurrent `Set`?

`ConcurrentHashMap.newKeySet()`, or `map.keySet(defaultValue)` to get a view that adds entries with that value. For a sorted concurrent set use `ConcurrentSkipListSet`, and for a rarely changing one, `CopyOnWriteArraySet`. RefinePoint's workers track their in-flight run, export and refresh IDs in `ConcurrentHashMap.newKeySet()`.

Q43. `Hashtable` vs `Collections.synchronizedMap` vs `ConcurrentHashMap`?

`Hashtable` and `synchronizedMap` lock one monitor for every operation, so readers block each other, iteration needs manual locking, and compound actions need external synchronisation. `ConcurrentHashMap` has lock-free reads, per-bin writes, weakly consistent iteration and atomic compound methods. Use `synchronizedMap` only to add thread safety to a map whose specific behaviour you need, such as an access-ordered `LinkedHashMap`.

Q44. How does `CopyOnWriteArrayList` work, and when should you avoid it?

It holds a `volatile` array; every write locks, copies the whole array, applies the change and publishes the copy, while reads and iterators use whichever array they saw, without locking. Iterators are snapshots that never throw and do not support `remove`. Avoid it for large or frequently written lists: each write is O(n) in time and allocation. It is ideal for listener and observer lists, and configuration read on every request but changed rarely.

Q45. What is a skip list, and why does `ConcurrentSkipListMap` use one?

A sorted linked list plus randomly built index levels that skip ahead, giving expected O(log n) search, insert and delete. Each update changes only a few pointers that can each be CASed independently, so the whole structure is lock-free. A balanced tree's rotations change several nodes atomically, which is hard to do without locks. That makes it the standard concurrent sorted map.

Q46. `ConcurrentSkipListMap` vs `TreeMap` wrapped in `synchronizedSortedMap`?

The skip list scales with concurrent readers and writers and has weakly consistent iterators, at the cost of more memory per entry and somewhat slower single-threaded operations. The synchronised `TreeMap` serialises every access, and iterating or using views needs the caller to hold the lock. Choose the skip list when several threads read or write a sorted map at the same time.

Q47. Which concurrent collections have an O(n) `size()`?

`ConcurrentLinkedQueue` and `ConcurrentLinkedDeque` traverse the nodes to count them, and the result can be stale. `ConcurrentHashMap`'s size is a fast sum of counters but only an estimate under change. Do not use `size()` of a concurrent queue as a loop condition or a capacity check; use `isEmpty()` or a separate counter.

Q48. What are sequenced collections in Java 21?

JEP 431 added `SequencedCollection`, `SequencedSet` and `SequencedMap`, giving every collection with a defined encounter order uniform first, last and reverse operations: `getFirst`, `getLast`, `addFirst`, `addLast`, `removeFirst`, `removeLast`, `reversed()`, and for maps `firstEntry`, `lastEntry`, `pollFirstEntry`, `putFirst` and sequenced views. `List`, `Deque`, `LinkedHashSet`, `SortedSet`, `LinkedHashMap` and `SortedMap` implement them; `HashMap`, `HashSet` and `PriorityQueue` do not.

Q49. Predict the output.

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3));
List<Integer> rev = list.reversed();
```

```
rev.add(0);
System.out.println(list + " " + rev + " " + list.getLast());
```

`[0, 1, 2, 3] [3, 2, 1, 0] 3`. `reversed()` is a view, so adding at the end of the reversed view adds at the front of the original.

Q50. Why does `TreeSet.addFirst` throw?

A sorted set's comparator decides every element's position, so "put this first" has no meaning. `addFirst`, `addLast`, and on sorted maps `putFirst` and `putLast`, throw `UnsupportedOperationException`. `getFirst` and `removeFirst` work, and mean the smallest element.

Q51. Which collection would you use for each: unique visitor IDs per day; a leaderboard updated by many threads; the latest N events; tasks due at a time; a per-request list of DTOs?

Unique IDs: `HashSet`, or `ConcurrentHashMap.newKeySet()` if shared. Concurrent leaderboard: `ConcurrentSkipListMap` keyed by score and ID. Latest N events: `ArrayDeque` with `pollFirst` when over N, guarded if shared, or a ring buffer. Tasks due at a time: `ScheduledThreadPoolExecutor` or `DelayQueue` in one JVM, and a database table with `next_run_at` across instances. Per-request DTOs: `ArrayList`, presized if the count is known.

Q52. Describe the rate limiter in your gateway.

"`RateLimitFilter` is a Spring Cloud Gateway global filter with a token bucket per caller: per API key, per user from the token's subject, and per IP address. Buckets live in a `ConcurrentHashMap<String, Bucket>`, created with `computeIfAbsent` so each key gets exactly one, and each `Bucket` guards its token arithmetic with its own `synchronized` methods, so different callers never contend. Over the limit we return 429 with `Retry-After` and `X-RateLimit-*` headers. Every 10,000 calls we drop buckets idle for ten minutes with `entrySet().removeIf`, which is safe during traffic because the iteration is weakly consistent. It's per gateway instance; a shared limit across instances would need Redis."

Q53. What race exists in that design, and why accept it?

The idle sweep can remove a bucket that a concurrent request has just fetched with `computeIfAbsent`. That request then takes a token from a bucket no longer in the map, and the caller's next request gets a fresh, full bucket. The effect is a brief, slightly lenient limit for a caller who had been idle for ten minutes. Eliminating it would require locking around every lookup, slowing every request to fix a harmless edge case.

Q54. Token bucket vs sliding window log vs fixed window?

Fixed window: one counter per period, cheap, but allows twice the limit around a window boundary. Sliding log: stores a timestamp per call and is exact, but memory grows with the limit. Token bucket: two numbers per key, allows controlled bursts and smooth refill. RefinePoint uses token buckets for high-volume gateway limits (hundreds per minute) and `SlidingWindowLimiter`'s exact sliding log for small quotas like AI calls per user per hour, where storing up to `max` timestamps per key is cheap.

Q55. Why does `SlidingWindowLimiter` use `ArrayDeque` with `synchronized` instead of a concurrent queue?

The check and the add must be atomic together: drop expired timestamps, compare the size with the limit, then append. A `ConcurrentLinkedDeque` makes each step thread-safe but not the sequence, and its `size()` is O(n). An `ArrayDeque` under the deque's own monitor makes the whole sequence atomic per key, with O(1) operations at both ends and no node allocation, while different keys use different monitors.

Q56. How do you lock per key in Java?

The simplest is a map of lock objects: `synchronized (locks.computeIfAbsent(id, k -> new Object())) { ... }`, which RefinePoint's `BankFeedService` uses so that refreshes of one bank consent are serialised while different consents run in parallel. Watch growth, since the map holds one object per key, and is fine when keys are bounded, such as consents. Alternatives are lock striping (a fixed array of locks indexed by hash, as Guava's `Striped` offers), or `ConcurrentHashMap.compute` itself when the critical section only updates that entry. Across instances, use a database row lock or advisory lock.

Q57. Your service's `ConcurrentHashMap` cache keeps growing until OOM. How do you fix it?

`ConcurrentHashMap` has no eviction. Bound it: by size (Caffeine's `maximumSize`, or `TtlCache`-style clear-when-full), by time (`expireAfterWrite`), or both, and record hit rates and evictions as metrics. Also check the keys: unbounded key spaces like full URLs, query strings or user input need normalising or should not be cached. A heap dump in Eclipse MAT shows the map's retained size and sample keys.

Q58. A `ConcurrentHashMap` shows high contention in a profiler. What could cause it?

Several threads writing the same few keys, which all land in the same bins and serialise on the same bin lock; a slow `compute` or `computeIfAbsent` function holding a bin lock; a poor `hashCode` sending many keys to few bins; or many threads incrementing one entry with `merge`. Fixes: `LongAdder` values for hot counters, moving slow work outside the lock, a better hash, or sharding the hot key.

Q59. Why is the `RateLimitFilter.Bucket` using `synchronized` acceptable with virtual threads and reactive code?

The gateway runs on Netty event-loop threads, not virtual threads, and the critical section is a few arithmetic operations with no I/O, so a monitor is held for nanoseconds. Pinning a virtual thread (an issue in Java 21, fixed in Java 24 by JEP 491) only matters when a thread blocks inside `synchronized`, which never happens here. A `ReentrantLock` would add allocation and code for no benefit.

Q60. What is false sharing and how do concurrent collections avoid it?

Two variables written by different threads that sit on the same 64-byte cache line make the cores invalidate each other's caches on every write, even though they are logically independent. `ConcurrentHashMap`'s `CounterCell` and `LongAdder`'s cells are annotated with `@jdk.internal.vm.annotation.Contended`, which pads them onto separate cache lines. Application code can use `-XX:-RestrictContended` with the annotation, or pad manually, but rarely needs to.

Q61. Tell me how you chose collections in your project.

"The rule was: pick the structure by access pattern, and know who else touches it. Single-threaded engine code uses `ArrayList`, `HashMap` and `TreeMap`: the FX rate lookup is a `TreeMap.floorEntry`, case-insensitive field matching uses a `TreeMap` with `CASE_INSENSITIVE_ORDER`, and matched groups are deduplicated with an identity set. Shared state in singleton beans is concurrent and bounded: `ConcurrentHashMap` for rate-limit buckets and short-lived caches, with per-key monitors for compound updates; `newKeySet()` for in-flight work; synchronised access-order `LinkedHashMap`s for LRU caches. Work that must survive restarts or be shared across instances is not in a Java collection at all, but in PostgreSQL with `SKIP LOCKED`."

6. Functional Java: Lambdas, Streams & Optional

Every Java interview since 2016 includes a stream question, and at 12 years of experience the bar is higher than "I can write `filter().map().collect()`". Interviewers want to know *how* a lambda becomes an object without a class file, *why* a stream does nothing until the terminal operation, *when* a parallel stream makes things slower, and whether you can write a `groupingBy` with a downstream collector on a whiteboard without hesitation. This chapter covers the machinery first, then a long set of questions including the classic "write this stream" coding problems.

6.1 Functional interfaces: the types behind every lambda

A **functional interface** is an interface with exactly one abstract method (SAM — single abstract method). A lambda or method reference has no type of its own; it takes the type of the functional interface it is assigned to (its **target type**).

```
@FunctionalInterface                       // optional, but the compiler then enforces "exactly one abstract
method"
public interface MatchRule {
    boolean matches(Txn bank, Txn ledger);
    default MatchRule and(MatchRule other) {          // default methods do not count
        return (b, l) -> matches(b, l) && other.matches(b, l);
    }
    String toString();                                // public Object methods do not count either
}
```

Rules worth saying out loud: default and static methods do not count; abstract methods that re-declare public `Object` methods (`equals`, `toString`) do not count; `@FunctionalInterface` is optional but documents intent and turns accidental breakage into a compile error. `Comparator` has two abstract-looking methods (`compare` and `equals`) and is still functional for that reason.

The java.util.function catalog

Interface	Method	Shape	Typical use
`Function<T,R>`	`R apply(T)`	T → R	`map`
`BiFunction<T,U,R>`	`R apply(T,U)`	(T,U) → R	`Map.merge`, `replaceAll`
`UnaryOperator<T>`	`T apply(T)`	T → T	`List.replaceAll`, `Stream.iterate`
`BinaryOperator<T>`	`T apply(T,T)`	(T,T) → T	`reduce`, `toMap` merge
`Supplier<T>`	`T get()`	() → T	`orElseGet`, lazy init, `collect` supplier
`Consumer<T>`	`void accept(T)`	T → void	`forEach`, `ifPresent`
`BiConsumer<T,U>`	`void accept(T,U)`	(T,U) → void	`Map.forEach`, collector accumulator
`Predicate<T>`	`boolean test(T)`	T → boolean	`filter`, `removeIf`
`BiPredicate<T,U>`	`boolean test(T,U)`	(T,U) → boolean	`Files.find`
`IntFunction<R>`, `ToIntFunction<T>`, `IntUnaryOperator`, `IntBinaryOperator`, `IntPredicate`, `IntSupplier`, `IntConsumer`, `ObjIntConsumer<T>`, `IntToLongFunction` …		primitive specialisations	avoid boxing in `IntStream` etc.

There are 43 interfaces in `java.util.function`. You do not memorise them all; you memorise the **pattern**: `Function` / `Supplier` / `Consumer` / `Predicate` / `Operator`, the `Bi` prefix for two arguments, `Int` / `Long` / `Double` prefixes for primitive inputs, `To<Prim>` for primitive outputs, and `Obj<Prim>Consumer` for mixed.

Composition methods you should be able to use fluently:

```
Function<String, String> trim = String::trim;
Function<String, Integer> length = trim.andThen(String::length);     // trim first, then length
Function<String, Integer> same   = String::length.compose(trim);     // does NOT compile: a method ref needs a
target type
Function<String, Integer> ok     = ((Function<String, Integer>) String::length).compose(trim);

Predicate<Break> open = b -> b.status() == BreakStatus.OPEN;
Predicate<Break> big  = b -> b.amount().compareTo(LIMIT) > 0;
breaks.stream().filter(open.and(big.negate())).toList();
breaks.stream().filter(Predicate.not(Break::isResolved)).toList();    // Predicate.not – Java 11
Comparator<Break> order = Comparator.comparing(Break::status).thenComparing(Break::amount,
Comparator.reverseOrder());
```

`Function.identity()` returns `t -> t`; `UnaryOperator.identity()` is the same with the narrower type.

6.2 Lambda internals: no class file, one `invokedynamic`

An anonymous class compiles to a separate `Outer$1.class` file, loaded like any other class. A lambda does **not**. Compile this:

```
public class Demo {
    public static void main(String[] args) {
        String prefix = "ORG-";
        Function<String, String> f = code -> prefix + code;
        System.out.println(f.apply("42"));
```

```
    }
}
```

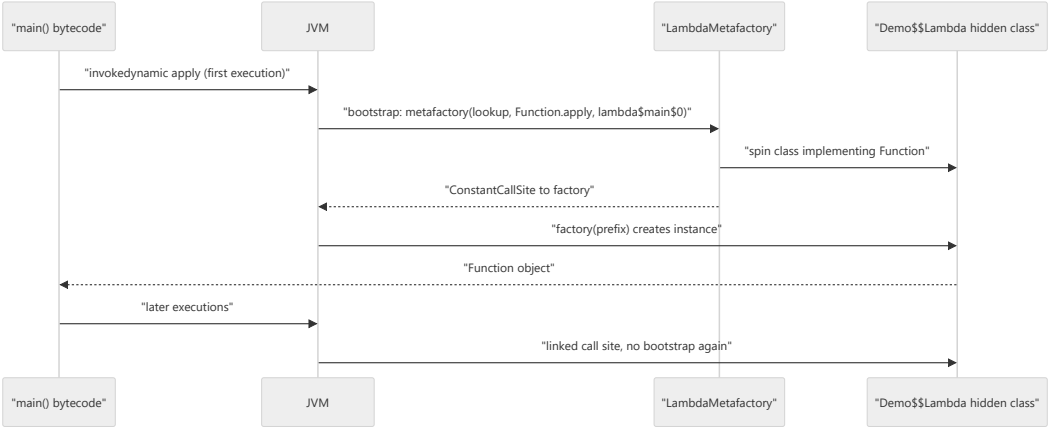
`javac` does two things:

1. **Desugars the body** into a private synthetic method in the same class: `private static String lambda$main$0(String prefix, String code)`. Captured variables become leading parameters. If the body uses `this` (or an instance field), the method is an *instance* method instead of static.
2. **Emits one `invokedynamic`** instruction at the place the lambda is evaluated. Its bootstrap method is `LambdaMetafactory.metafactory`, and its static arguments describe the target interface method (`Function.apply`), the implementation method handle (`lambda$main$0`) and the instantiated signature.

```
javap -c -p Demo.class (abridged)
  0: ldc           #7      // String ORG-
  2: astore_1
  3: aload_1
  4: invokedynamic #9,  0  // InvokeDynamic #0:apply:(Ljava/lang/String;)Ljava/util/function/Function;
  9: astore_2
 ...
private static java.lang.String lambda$main$0(java.lang.String, java.lang.String);

BootstrapMethods:
  0: #40 REF_invokeStatic java/lang/invoke/LambdaMetafactory.metafactory
     Method arguments: (Ljava/lang/Object;)Ljava/lang/Object;
                       REF_invokeStatic Demo.lambda$main$0:
(Ljava/lang/String;Ljava/lang/String;)Ljava/lang/String;
                       (Ljava/lang/String;)Ljava/lang/String;
```

At runtime, the **first time** that `invokedynamic` executes, the JVM calls the bootstrap method. `LambdaMetafactory` spins a tiny class in memory that implements `Function`, has one field per captured value and an `apply` that calls `lambda$main$0`. Since JDK 15 it is defined as a **hidden class** (`Lookup.defineHiddenClass`), so it is not discoverable by name and can be unloaded with its defining class. The bootstrap returns a `CallSite` bound to a factory; the call site is linked once and every later execution goes straight to the factory.



Capturing vs non-capturing:

	Non-capturing `s -> s.trim()`	Capturing `s -> prefix + s`
Instance per evaluation	The **same singleton** every time	A **new object** each evaluation (holds captured values)
Allocation	None after linkage	One small object (often removed by escape analysis)
`f1 == f2` for two evaluations	`true` (implementation detail, never rely on it)	`false`

Why this design beat inner classes: no class file bloat (a codebase with 5,000 lambdas would otherwise ship 5,000 classes), the translation strategy can change in a future JDK without recompiling your code, and the JIT sees through the linked call site and inlines the body. The same `invokedynamic` trick is used for string concatenation since Java 9 (`StringConcatFactory`) and for record `equals/hashCode/toString` (`ObjectMethods`).

To see the generated classes: `java -Djdk.internal.lambda.dumpProxyClasses=/tmp/lambdas Demo`.

Capture and "effectively final"

A lambda may read local variables of the enclosing method only if they are **final or effectively final** (never reassigned after initialisation).

```
int count = 0;
runs.forEach(r -> count++);          // compile error: local variables referenced from a lambda must be final
or effectively final
```

Why: the lambda captures the **value**, copied into a field of the lambda object (look at the leading parameters of `lambda$main$0`). The local variable lives in a stack frame that may be gone when the lambda runs — on another thread, after the method returned. If Java allowed mutation, you would have two copies that silently diverge, or would need to move locals to the heap and synchronise them. Java chose "capture by value, and forbid anything that would make the copy observable".

Instance and static fields are **not** restricted: the lambda captures `this` (for instance fields) and reads the field through it, so mutation is visible — and racy if the lambda runs on another thread.

```
int[] counter = {0};                 // compiles: the array reference is effectively final
runs.forEach(r -> counter[0]++);     // works sequentially, data race in parallel — a code smell
long n = runs.stream().filter(Run::failed).count();   // the functional answer
```

`this` in a lambda vs an anonymous class

```
public class Outer {
    String name = "outer";
    void test() {
        Runnable lambda = () -> System.out.println(this.name);          // "outer": lexical this
        Runnable anon = new Runnable() {
            String name = "anon";
            public void run() { System.out.println(this.name); }        // "anon": the anonymous instance
        };
    }
}
```

A lambda is **lexically scoped**: `this`, `super` and variable names mean exactly what they mean in the enclosing method. It cannot declare a parameter or local with the same name as an enclosing local (compile error), whereas an anonymous class introduces a new scope and can shadow.

6.3 Method references: four kinds

Kind	Syntax	Equivalent lambda	Example
Static method	`Class::staticMethod`	`(a, b) -> Class.staticMethod(a, b)`	`Integer::parseInt`, `Math::max`, `Objects::nonNull`
Bound instance (specific object)	`obj::method`	`x -> obj.method(x)`	`System.out::println`, `cache::get`
Unbound instance (arbitrary object of a type)	`Class::instanceMethod`	`(obj, x) -> obj.method(x)`	`String::length`, `BigDecimal::add`, `String::compareToIgnoreCase`
Constructor	`Class::new`	`x -> new Class(x)`	`ArrayList::new`, `int[]::new`, `TreeMap::new`

`BigDecimal::add` is the unbound kind: `add` is an instance method, the first lambda parameter becomes the receiver, so it fits `BinaryOperator<BigDecimal>` as `(a, b) -> a.add(b)`. That is why `reduce(BigDecimal.ZERO, BigDecimal::add)` works.

Two subtle differences from the equivalent lambda:

```
String s = null;
Supplier<Integer> lambda = () -> s.length();   // fine now, NPE only when get() is called
Supplier<Integer> ref    = s::length;          // NPE immediately: the receiver is evaluated (and null-checked)
at creation

Supplier<Run> eager = repo.findLatest()::copy; // findLatest() runs ONCE, when the reference is created
Supplier<Run> lazy  = () -> repo.findLatest().copy();   // runs on every get()
```

The bound receiver is evaluated when the method reference expression is evaluated, not when the function is called. That matters for side effects and for nulls.

Overload ambiguity is the other trap: `Integer::toString` can mean the static `toString(int)` or the instance `toString()`, so `Function<Integer,String> f = Integer::toString;` fails to compile ("ambiguous"). Use `String::valueOf` or a lambda.

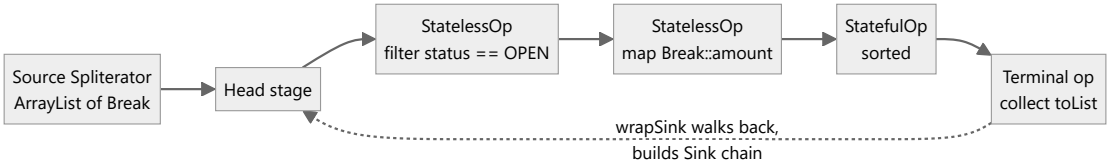
6.4 Stream pipeline internals

A stream is **not a data structure**. It is a description of a computation over a source, executed once by a terminal operation. Three parts:

1. **Source** — a `Spliterator` (collection, array, `Files.lines`, generator, `IntStream.range`).
2. **Intermediate operations** — `filter`, `map`, `sorted` ... each returns a new stream *stage*; nothing executes.
3. **Terminal operation** — `collect`, `forEach`, `reduce`, `findFirst`, `count` ... triggers execution and consumes the stream.

The stage linked list

Internally each stage is an `AbstractPipeline` node (`ReferencePipeline.Head`, `StatelessOp`, `StatefulOp`) holding a link to its upstream stage, the op flags and the source. Building a pipeline only allocates these nodes.



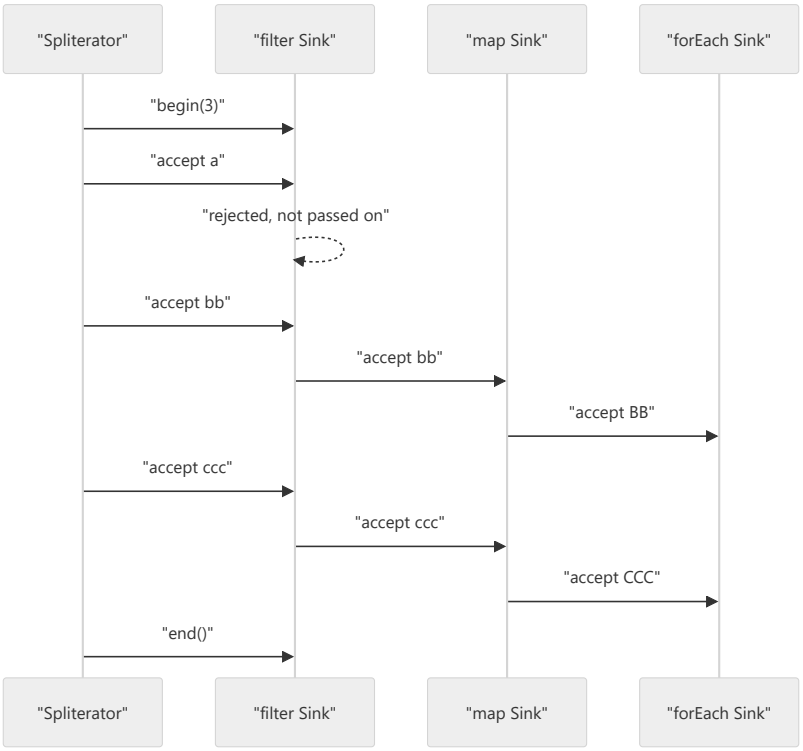
When the terminal operation runs, it calls `wrapSink`, which walks from the last stage back to the head, wrapping each stage's logic around the next one as a `Sink` (a `Consumer` with `begin(size)`, `accept(t)`, `end()` and `cancellationRequested()`). Then it calls `spliterator.forEachRemaining(sinkChain)` — the source **pushes** elements one at a time through the whole chain.

Vertical execution (loop fusion)

Because of the sink chain, a stream processes element 1 through *all* stateless stages, then element 2, and so on. It does **not** filter the whole list, then map the whole list. This is effectively loop fusion: one pass, no intermediate collections.

```
Stream.of("a", "bb", "ccc")
      .filter(s -> { System.out.println("filter " + s); return s.length() > 1; })
      .map(s -> { System.out.println("map " + s); return s.toUpperCase(); })
      .forEach(s -> System.out.println("forEach " + s));
```

```
filter a
filter bb
map bb
forEach BB
filter ccc
map ccc
forEach CCC
```



Stateless vs stateful operations

Category	Operations	Behaviour
Stateless intermediate	`filter`, `map`, `flatMap`, `mapMulti`, `peek`, `mapToInt`, `boxed`, `unordered`	Handle each element independently; fully fused
Stateful intermediate	`sorted`, `distinct`, `limit`, `skip`, `takeWhile`, `dropWhile`	Need to remember previous elements; `sorted` is a full **barrier**
Short-circuiting intermediate	`limit`, `takeWhile`	Can finish without consuming the whole source
Terminal	`forEach`, `forEachOrdered`, `collect`, `reduce`, `toList`, `count`, `min`, `max`, `toArray`	Consume everything
Short-circuiting terminal	`findFirst`, `findAny`, `anyMatch`, `allMatch`, `noneMatch`	Stop at the first decisive element

`sorted()` cannot emit anything until it has seen everything: its sink buffers all elements in `accept`, sorts in `end()`, then pushes them downstream. That changes the printed order in peek-based puzzles:

```
Stream.of(3, 1, 2)
      .peek(x -> System.out.println("peek " + x))
      .sorted()
      .forEach(x -> System.out.println("each " + x));
// peek 3, peek 1, peek 2, each 1, each 2, each 3
```

Short-circuiting

When a pipeline contains a short-circuit op, the terminal switches from `forEachRemaining` to a loop of `tryAdvance` calls that checks `sink.cancellationRequested()` before each element:

```
Optional<String> first = Stream.of("x", "yy", "zzz", "wwww")
        .filter(s -> { System.out.println("check " + s); return s.length() > 1; })
```

```
        .findFirst();
// check x, check yy   — "zzz" and "wwww" are never examined
```

That is why `Stream.iterate(1, i -> i * 2).filter(i -> i > 1000).findFirst()` terminates on an infinite source, while `Stream.iterate(1, i -> i * 2).sorted().findFirst()` never does (sorted must consume everything first).

Spliterator and characteristics

`Spliterator` = *splittable iterator*. Two jobs: traverse (`tryAdvance` , `forEachRemaining`) and split (`trySplit` returns a spliterator covering a prefix, used by parallel streams). It reports **characteristics** that the pipeline uses to optimise:

Flag	Meaning	Optimisation it enables
`SIZED` / `SUBSIZED`	Exact size known (and after splits)	`toArray` allocates exactly once; `count()` can skip traversal
`ORDERED`	Encounter order is meaningful (`List` , arrays)	Must preserve order; `unordered()` drops it
`DISTINCT`	No duplicates (`Set`)	`distinct()` becomes a no-op
`SORTED`	Already sorted (`TreeSet`)	`sorted()` with natural order becomes a no-op
`NONNULL` , `IMMUTABLE` , `CONCURRENT`	Source guarantees	Fewer checks; safe concurrent modification

The famous consequence (Java 9+):

```
List.of(1, 2, 3).stream().peek(System.out::println).count();   // prints NOTHING, returns 3
List.of(1, 2, 3).stream().filter(x -> true).peek(System.out::println).count();  // prints 1 2 3
```

With a `SIZED` source and no stage that can change the count, `count()` returns the size directly. `peek` is a debugging aid, not a place for logic.

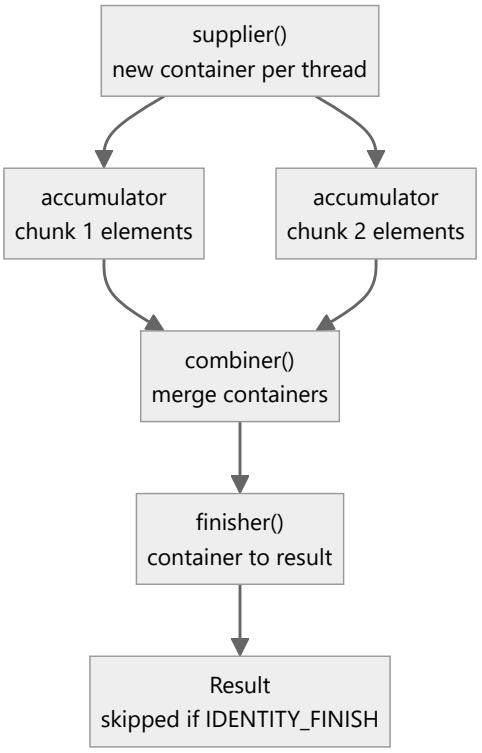
Streams are single-use and lazy

```
Stream<Run> s = runs.stream().filter(Run::failed);
s.count();
s.toList();          // IllegalStateException: stream has already been operated upon or closed
```

A stream also binds lazily to its source: for most JDK collections, the spliterator is created when the terminal operation starts, so modifications made *before* the terminal operation are visible, and structural modification *during* it throws `ConcurrentModificationException` (fail-fast spliterator) — or, worse, produces undefined results for non-interfering requirements.

6.5 Collectors in depth

`collect(Collector)` is a **mutable reduction**: create a container, fold elements into it, merge containers (in parallel), optionally transform at the end.



toList variants

Call	Mutable?	Nulls	Since
`collect(Collectors.toList())`	Unspecified, in practice `ArrayList` (mutable)	Allowed	8
`collect(Collectors.toUnmodifiableList())`	Unmodifiable	**NPE** on null	10
`stream.toList()`	Unmodifiable	Allowed	16

toMap: duplicates, merge, nulls, map type

```
Map<String, Run> byCode = runs.stream()
        .collect(Collectors.toMap(Run::code, r -> r));
// IllegalStateException: Duplicate key RUN-7 (attempted merging values ...) if two runs share a code

Map<String, Run> latest = runs.stream()
        .collect(Collectors.toMap(Run::code, Function.identity(),
                (a, b) -> a.startedAt().isAfter(b.startedAt()) ? a : b,   // merge function
                LinkedHashMap::new));                                      // map supplier keeps order
```

Trap: `toMap` throws `NullPointerException` if any **value** is null, because it uses `HashMap.merge`, which rejects null values — even though `HashMap` itself allows them. Either filter nulls, map them to a sentinel, or collect with `collect(HashMap::new, (m, r) -> m.put(k(r), v(r)), Map::putAll)`.

groupingBy and downstream collectors

`groupingBy(classifier)` = `groupingBy(classifier, HashMap::new, toList())`. The power is the **downstream collector** applied per group.

```
Map<BreakStatus, List<Break>> byStatus     = breaks.stream().collect(groupingBy(Break::status));
Map<BreakStatus, Long> countByStatus       = breaks.stream().collect(groupingBy(Break::status, counting()));
Map<String, Set<String>> accountsByOrg     = breaks.stream().collect(groupingBy(Break::orgCode,
mapping(Break::account, toSet())));
```

```
Map<BreakStatus, Optional<Break>> biggest  = breaks.stream().collect(groupingBy(Break::status,
maxBy(comparing(Break::amount))));
Map<BreakStatus, Break> biggestNoOptional  = breaks.stream().collect(groupingBy(Break::status,
        collectingAndThen(maxBy(comparing(Break::amount)), Optional::get)));
Map<String, Map<BreakStatus, Long>> nested = breaks.stream().collect(groupingBy(Break::orgCode,
groupingBy(Break::status, counting())));
TreeMap<LocalDate, List<Break>> sorted     = breaks.stream().collect(groupingBy(Break::valueDate, TreeMap::new,
toList()));
Map<BreakStatus, List<Break>> bigOnly      = breaks.stream().collect(groupingBy(Break::status,
        filtering(b -> b.amount().compareTo(LIMIT) > 0, toList())));   // Java 9: keeps empty groups
```

`filtering` inside `groupingBy` differs from `filter` before it: filtering first drops groups that end up empty; the `filtering` downstream keeps the key with an empty list. `flatMapping` (Java 9) is the flat equivalent of `mapping`.

`groupingBy` throws NPE when the classifier returns `null` ("element cannot be mapped to a null key"). `groupingByConcurrent` builds a `ConcurrentHashMap` and is the right choice only in unordered parallel streams.

partitioningBy, teeing, collectingAndThen

```
Map<Boolean, List<Break>> split = breaks.stream().collect(partitioningBy(b -> b.ageDays() > 30));
// ALWAYS has both keys true and false, even if one list is empty (groupingBy would omit the missing key)

record Totals(BigDecimal debit, BigDecimal credit) {}
Totals t = txns.stream().collect(teeing(                                  // Java 12: two collectors, one pass
        filtering(Txn::isDebit,  reducing(BigDecimal.ZERO, Txn::amount, BigDecimal::add)),
        filtering(Txn::isCredit, reducing(BigDecimal.ZERO, Txn::amount, BigDecimal::add)),
        Totals::new));

List<String> names = users.stream().map(User::name)
        .collect(collectingAndThen(toList(), Collections::unmodifiableList));
```

Other collectors you should name: `joining(", ", "[", "]")`, `summingInt/Long/Double`, `averagingDouble`, `summarizingLong` (count, sum, min, max, average in one pass), `minBy/maxBy`, `reducing`, `toCollection(TreeSet::new)`.

Writing a custom Collector

```
public static Collector<Txn, ?, Map<String, BigDecimal>> sumByCurrency() {
    return Collector.of(
        HashMap::new,                                                  // supplier
        (Map<String, BigDecimal> m, Txn t) -> m.merge(t.currency(), t.amount(), BigDecimal::add),  //
accumulator
        (m1, m2) -> { m2.forEach((k, v) -> m1.merge(k, v, BigDecimal::add)); return m1; },        // combiner
        Collector.Characteristics.UNORDERED);                          // identity finisher implied
}
```

Characteristics: `IDENTITY_FINISH` (finisher is a cast, skip it), `UNORDERED` (result does not depend on encounter order), `CONCURRENT` (one shared container may be accumulated from many threads — only honoured if also `UNORDERED` or the stream is unordered).

6.6 flatMap, mapMulti, reduce vs collect

`map` is one-to-one; `flatMap` is one-to-many: each element maps to a *stream* and the streams are concatenated.

```
List<Txn> all = statements.stream().flatMap(st -> st.lines().stream()).toList();
Optional<Account> acct = findUser(id).flatMap(User::primaryAccount);      // Optional.flatMap avoids
Optional<Optional<…>>
Stream.of("a,b", "c").flatMap(s -> Arrays.stream(s.split(","))).toList();  // [a, b, c]

// Java 16 mapMulti: push zero or more results without creating a stream per element
statements.stream().<Txn>mapMulti((st, sink) -> st.lines().forEach(sink)).toList();
```

`flatMap` was not lazy over the inner stream until JDK 10 (it pushed the whole inner stream even after `findFirst`); since then it respects short-circuiting.

`reduce` vs `collect`:

	`reduce`	`collect`
Kind	Immutable reduction: combine values into a new value	Mutable reduction: accumulate into a container
Good for	Sums, max, combining immutable values (`BigDecimal` , `String` concatenation of few elements)	Lists, maps, `StringBuilder` , anything with a container
Parallel rule	Identity must be a true identity; accumulator must be **associative** and stateless	Supplier must create a **fresh** container per call

```
int sum = IntStream.rangeClosed(1, 5).reduce(0, Integer::sum);           // 15
Optional<Integer> max = Stream.of(4, 9, 2).reduce(Integer::max);          // no identity -> Optional
int totalLength = words.stream().reduce(0, (acc, w) -> acc + w.length(), Integer::sum);  // 3-arg: identity, accumulator, combiner

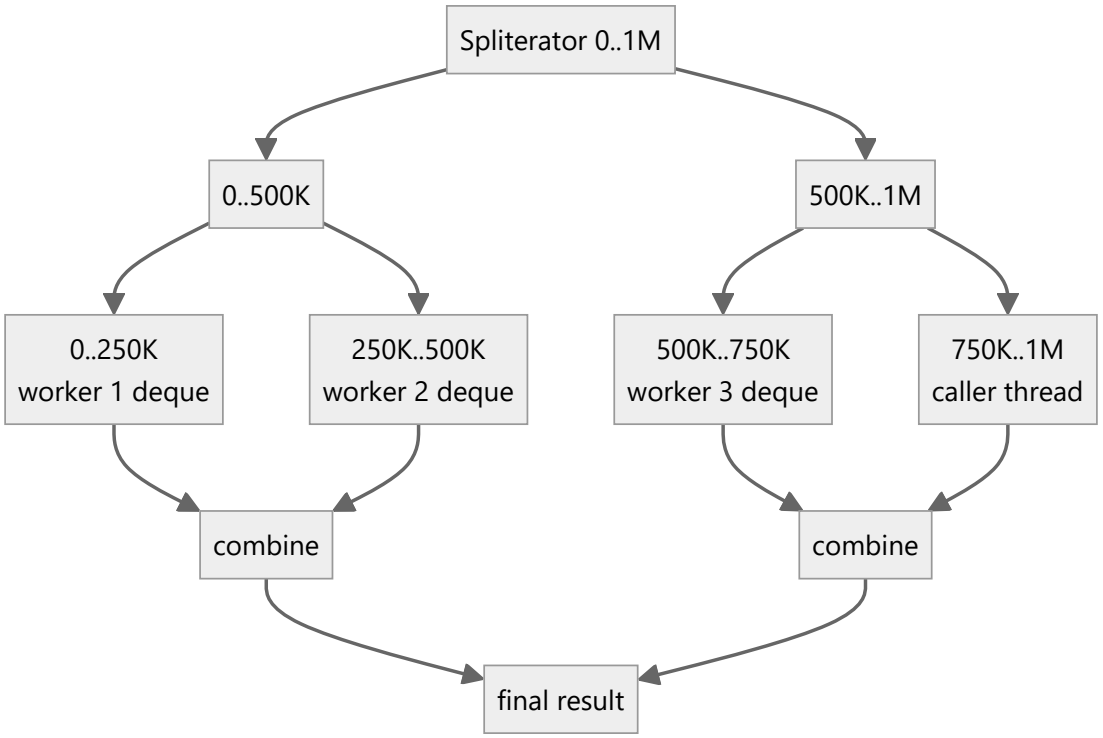
// Wrong: a mutable container passed as identity is SHARED across threads in parallel
List<String> bad = words.parallelStream().reduce(new ArrayList<>(), (l, w) -> { l.add(w); return l; }, (a, b) -> { a.addAll(b); return a; });
```

The broken last example corrupts the list in parallel (and duplicates elements) because every split uses the same `ArrayList` identity. Use `collect`.

> **Trap:** `reduce(0, (a, b) -> a - b)` or `reduce(1, ...)` as a sum identity gives different answers sequentially and in parallel, because subtraction is not associative and 1 is not the identity for `+`. Parallel reduction applies the identity once per chunk.

6.7 Parallel streams

`collection.parallelStream()` (or `.parallel()`) splits the source with `trySplit` , processes chunks as `ForkJoinTask` s on the **ForkJoinPool common pool**, and merges results. The common pool has `availableProcessors() - 1` worker threads (the calling thread also works, so all cores are used), configurable with `-Djava.util.concurrent.ForkJoinPool.common.parallelism=N` .



Each worker has its own deque; it pushes and pops its own subtasks LIFO and **steals** from the tail of another worker's deque when idle (work stealing), so uneven chunks balance out.

Splitting quality decides the speed-up

Source	Splits	Parallel suitability
`ArrayList`, arrays, `IntStream.range`	Exact halves in O(1), `SIZED` + `SUBSIZED`	Excellent
`HashSet`, `HashMap.keySet`	By bucket ranges, sizes estimated	Good
`TreeSet` / `TreeMap`	By subtree	Fair
`LinkedList`	Must walk the list	Poor
`Stream.iterate`, `BufferedReader.lines`, `Files.lines` (pre-JDK 9)	Sequential by nature, batches of growing size	Poor

When NOT to use parallel streams

1. **Inside a web server request.** The common pool is shared by every parallel stream and every `CompletableFuture.*Async` without an executor in the JVM. Ten concurrent requests each running a parallel stream compete for the same N-1 threads; you gain nothing and add contention.
2. **Blocking I/O in the lambda** (HTTP calls, JDBC). Workers block, the pool starves, unrelated code stalls. Use an explicit executor (or virtual threads).
3. **Small data or cheap operations.** The NQ rule of thumb: N elements × Q cost per element should exceed roughly 10,000 before parallelism pays for splitting, task creation and merging.
4. **Order-dependent operations.** `findFirst`, `limit`, `skip`, `sorted`, `forEachOrdered` and `distinct` on ordered streams must preserve encounter order and buffer a lot. Use `findAny`, call `.unordered()` when order does not matter.
5. **Side effects or shared mutable state.** `parallelStream().forEach(list::add)` on an `ArrayList` loses elements or throws `ArrayIndexOutOfBoundsException`.
6. **Contended merges.** `groupingBy` in parallel merges maps (expensive); `groupingByConcurrent` avoids it but loses order.

```
List<Integer> unsafe = new ArrayList<>();
IntStream.range(0, 10_000).parallel().forEach(unsafe::add);   // size < 10000, or an exception
List<Integer> safe = IntStream.range(0, 10_000).parallel().boxed().toList();  // ordered, correct
```

`forEach` on a parallel stream does **not** respect encounter order; `forEachOrdered` does, at the cost of buffering.

Running a parallel stream in a custom pool (`customPool.submit(() -> list.parallelStream()...).get()`) works because tasks forked from inside a `ForkJoinPool` run in *that* pool. It is an implementation behaviour, not a documented contract — prefer explicit executors for anything that blocks.

> **RefinePoint:** we never use parallel streams in request handling. The heavy matching of millions of rows does not happen in Java collections at all: `ryfn-engine` loads files into **DuckDB** in-process and `SqlCompiler` turns match rules into SQL, so DuckDB's own vectorised, multi-threaded executor does the work. Streams are used for the in-memory glue — grouping breaks, building summaries, mapping DTOs — where sequential streams are fast enough and predictable.

6.8 Primitive streams and boxing cost

`Stream<Integer>` stores `Integer` objects: 16 bytes each plus a 4–8 byte reference, a pointer chase per element, and allocation for values outside the -128..127 cache. `IntStream` , `LongStream` , `DoubleStream` work on primitives and add numeric terminals.

```
int total = runs.stream().mapToInt(Run::matchedCount).sum();         // no boxing
OptionalDouble avg = runs.stream().mapToLong(Run::durationMs).average();   // OptionalDouble, empty for no
elements
IntSummaryStatistics st = runs.stream().mapToInt(Run::breakCount).summaryStatistics();
List<Integer> boxed = IntStream.rangeClosed(1, 5).boxed().toList();    // back to objects
IntStream.range(0, 5);        // 0..4
IntStream.rangeClosed(0, 5);  // 0..5
"hello".chars();              // IntStream of UTF-16 code units; use codePoints() for emoji/supplementary chars
```

`Stream<Integer>.reduce(0, Integer::sum)` unboxes and re-boxes at every step. On a few million elements the primitive version is several times faster and allocates nothing. There is no `CharStream` , `ByteStream` or `FloatStream` .

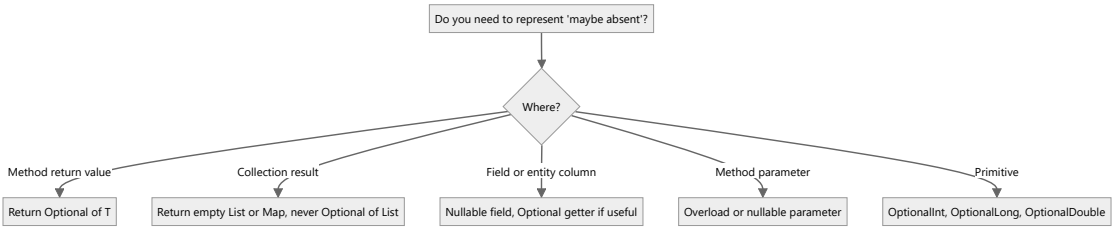
6.9 Stream vs loop performance

- For **small collections** (tens to hundreds of elements) a plain `for` loop is usually faster: no pipeline objects, no lambda dispatch, simpler for the JIT. The difference is microseconds and almost never matters.
- For **large data with stateless stages**, JIT-compiled streams come close to loops because the sink chain inlines; boxing is the real cost, not the stream framework.
- Streams win on **readability** of transformations (group, partition, aggregate) and on optional parallelism; loops win for complex control flow (early `return` from the method, `break` out of nested loops, checked exceptions, mutating several locals).
- Measure with **JMH**, not `System.currentTimeMillis()` in `main` — warm-up and dead-code elimination make naive benchmarks meaningless.

Senior answer: "I default to streams for transformations and loops for control flow, and I only optimise after a profiler shows the pipeline is hot."

6.10 Optional

`Optional<T>` is a container for zero or one value, designed (by its authors' own statement) as a **method return type** that forces the caller to think about absence.



Correct usage

```
public Optional<Workspace> findByCode(String code) { ... }

String plan = workspaces.findByCode(code)
        .filter(Workspace::active)
        .map(Workspace::plan)
        .map(Plan::name)
        .orElse("FREE");

Workspace ws = workspaces.findByCode(code)
        .orElseThrow(() -> new NotFoundException("Workspace " + code));     // Java 8 with supplier
Workspace ws2 = workspaces.findByCode(code).orElseThrow();                // Java 10: NoSuchElementException

opt.ifPresentOrElse(this::notify, () -> log.warn("none"));                 // Java 9
Optional<Run> any = primary().or(this::fallback);                         // Java 9: lazy alternative Optional
long present = ids.stream().map(repo::findById).flatMap(Optional::stream).count();   // Java 9
if (opt.isEmpty()) { ... }                                                 // Java 11
```

orElse vs orElseGet

```
String v = Optional.of("cached").orElse(loadFromDb());        // loadFromDb() ALWAYS runs — argument evaluated first
String w = Optional.of("cached").orElseGet(() -> loadFromDb()); // loadFromDb() runs only when empty
```

`orElse(x)` is fine for constants and already-computed values. For anything with a cost or side effect (DB call, creating an entity, logging) use `orElseGet`. This is the single most-asked Optional question, and it causes real bugs: an `orElse(repository.save(new Default()))` inserts a row on every call.

Incorrect usage

Anti-pattern	Why	Better
`opt.get()` without check	Same NPE risk with a different exception	`orElseThrow()` (explicit intent)
`if (opt.isPresent()) { x = opt.get(); }`	Null-check with more ceremony	`map` / `ifPresent` / `orElse`
`Optional` field in an entity/DTO	Not `Serializable`, Jackson/JPA friction, extra allocation	Nullable field
`Optional` parameter	Callers must wrap; three states (null, empty, present)	Overloads
`Optional<List<T>>`	Empty list already means "none"	Return `List.of()`
`Optional.of(maybeNull)`	NPE when null	`Optional.ofNullable`
Returning `null` from an `Optional` method	Defeats the purpose	Always `Optional.empty()`
`Optional` for identity-sensitive ops (`==` , synchronized)	It is a value-based class	Compare with `equals`

Spring Data repositories return `Optional<T>` from `findById`; Spring MVC accepts `Optional` request params; Jackson needs the `jdk8` module (included by Spring Boot) to serialise it — another reason to keep it off DTOs.

6.11 Checked exceptions in lambdas

The `java.util.function` interfaces declare no checked exceptions, so this fails to compile:

```
List<String> contents = paths.stream().map(Files::readString).toList();   // unhandled IOException
```

Options, in order of preference:

```
// 1. Wrap into the JDK's unchecked equivalent, inside a small helper method
private static String read(Path p) {
    try { return Files.readString(p); }
    catch (IOException e) { throw new UncheckedIOException(e); }
}
paths.stream().map(Loader::read).toList();

// 2. A throwing functional interface plus an adapter
@FunctionalInterface
interface ThrowingFunction<T, R> { R apply(T t) throws Exception; }
static <T, R> Function<T, R> unchecked(ThrowingFunction<T, R> f) {
    return t -> {
        try { return f.apply(t); }
        catch (RuntimeException e) { throw e; }
        catch (Exception e) { throw new CompletionException(e); }
    };
}
paths.stream().map(unchecked(Files::readString)).toList();

// 3. Return a result type instead of throwing (keeps going on errors)
sealed interface Parsed permits Ok, Failed {}
record Ok(Txn txn) implements Parsed {}
record Failed(int line, String error) implements Parsed {}
```

"Sneaky throw" (using generic type erasure to rethrow a checked exception without declaring it) works but hides exceptions from the compiler and from `catch` clauses callers would write; avoid it in application code. When a loop needs checked exceptions *and* early exit, a plain `for` loop is often the clearest answer.

Remember resource-backed streams: `Files.lines`, `Files.list`, `Files.walk` hold an open file handle and must be closed — `try (Stream<String> lines = Files.lines(path)) { ... }`. A stream's `onClose` handlers run on `close()`, not after the terminal operation.

6.12 RefinePoint patterns with streams

Grouping breaks for the dashboard tiles:

```
record Totals(long count, BigDecimal amount) {}
public record BreakSummary(BreakStatus status, long count, BigDecimal total) {}

public List<BreakSummary> summarise(List<Break> breaks) {
    Map<BreakStatus, Totals> byStatus = breaks.stream()
        .collect(groupingBy(Break::status,
                 () -> new EnumMap<>(BreakStatus.class),                // iterates in enum declaration order
                 teeing(counting(),
                        reducing(BigDecimal.ZERO, Break::amount, BigDecimal::add),
                        Totals::new)));                                 // one pass: count and sum together
    return byStatus.entrySet().stream()
        .map(e -> new BreakSummary(e.getKey(), e.getValue().count(), e.getValue().amount()))
        .toList();
}
```

Summing money correctly:

```
BigDecimal total = lines.stream().map(Txn::amount).reduce(BigDecimal.ZERO, BigDecimal::add);

Map<String, BigDecimal> byCurrency = lines.stream()
        .collect(groupingBy(Txn::currency, TreeMap::new,
                 reducing(BigDecimal.ZERO, Txn::amount, BigDecimal::add)));
```

RefinePoint: money is always `BigDecimal` and always summed with `reduce(BigDecimal.ZERO, BigDecimal::add)`, for example the `total(items)` helper in the month-end certification code (`CertificationService`) and the report builder's SUM aggregate. `summingDouble(t -> t.amount().doubleValue())` is the tempting shortcut and it is banned: `double` cannot represent 0.10 exactly, so errors accumulate over a month of lines. In a reconciliation product, "off by 3 cents against the bank total" is a visible defect that destroys trust.

RefinePoint: the real pipelines are short and readable. `RecordService` groups result items per run with `groupingBy(Item::runId, LinkedHashMap::new, toList())`, so runs stay in the order they arrived, and `DataQualityService` indexes error records by row number with `toMap(ErrorRecord::getRowNumber, r -> r, (a, b) -> a)`, where the explicit merge function documents that the first record for a row wins. `MatchStrategy` is a plain enum (`EXACT`, `NUMERIC_TOLERANCE`, `JARO_WINKLER` ...), which the engine's configuration selects, so no stream is needed there.

6.13 Interview questions and answers

Q1. What is a functional interface? Is `Comparator` one even though it declares two methods?

An interface with exactly one abstract method, which lambdas and method references can implement. `Comparator` declares `compare` and `equals`, but `equals` re-declares a public `Object` method and does not count, so `Comparator` is functional. Default and static methods never count. `@FunctionalInterface` is optional; it makes the compiler reject a second abstract method.

Q2. What is the type of a lambda expression?

It has no type of its own; its type is the **target type** inferred from the context (assignment, method argument, return, cast). The same lambda `s -> s.isEmpty()` can be a `Predicate<String>` or a `Function<String, Boolean>`. That is why `var f = s -> s.isEmpty();` does not compile — there is no target type to infer from.

Q3. How is a lambda compiled and executed? Does it create an inner class?

No class file is generated. `javac` moves the body into a private synthetic method (`lambda$main$0`) and puts an `invokedynamic` instruction where the lambda is evaluated, with `LambdaMetafactory.metafactory` as bootstrap. On the first execution the JVM runs the bootstrap, which spins a hidden class implementing the interface and returns a call site; later executions reuse the linked call site. Non-capturing lambdas yield a cached singleton, capturing lambdas allocate a small object holding the captured values.

Q4. Why did Java choose `invokedynamic` instead of anonymous inner classes for lambdas?

Four reasons: no class-file explosion (each anonymous class is a `.class` file to load and verify), no forced allocation for non-capturing lambdas, the JVM can change the translation strategy in a later release without recompilation (the bytecode only says "give me a Function"), and the JIT optimises linked `invokedynamic` call sites as well as ordinary calls. Lambdas also do not capture `this` unless they use it, so they do not accidentally retain the outer object the way inner classes do.

Q5. Why must captured local variables be effectively final?

Because the lambda captures the *value*, copied into the lambda object, while the local variable lives in a stack frame that may disappear before the lambda runs (possibly on another thread). Allowing reassignment would create two copies that silently diverge, or require moving locals to the heap with synchronisation. Fields are not restricted because the lambda reads them through the captured `this` reference. Workarounds like a one-element array or `AtomicInteger` compile but are usually a sign the code should use a reduction.

Q6. What does `this` refer to inside a lambda vs an anonymous class?

In a lambda, `this` is the enclosing instance (lexical scoping); in an anonymous class, `this` is the anonymous-class instance. Likewise a lambda cannot declare a parameter with the same name as an enclosing local variable, while an anonymous class can shadow it.

Q7. Name the four kinds of method references with examples.

Static (`Integer::parseInt`), bound instance on a specific object (`System.out::println` , `cache::get`), unbound instance on an arbitrary object of a type where the first argument becomes the receiver (`String::length` , `BigDecimal::add`), and constructor (`ArrayList::new` , `int[]::new`).

Q8. Is `obj::method` exactly equivalent to `x -> obj.method(x)` ?

No. In the method reference, `obj` is evaluated once when the reference is created and null-checked immediately; in the lambda, `obj` is evaluated on every call. So `String s = null; Supplier<Integer> r = s::length;` throws NPE at creation, the lambda only when invoked. Also `repo.load()::process` calls `load()` once, while `() -> repo.load().process()` calls it each time.

Q9. Why does `Function<Integer, String> f = Integer::toString;` fail to compile?

It is ambiguous: it could refer to the static `Integer.toString(int)` (with unboxing) or the instance method `Integer.toString()` on the argument. Use `String::valueOf` or `i -> i.toString()` .

Q10. What are intermediate and terminal operations? What is laziness?

Intermediate operations (`filter` , `map` , `sorted`) return a new stream and only record the step. Terminal operations (`collect` , `forEach` , `count` , `findFirst`) trigger execution by building the sink chain and pulling the source. Without a terminal operation nothing runs:

```
Stream.of(1, 2, 3).peek(System.out::println).map(x -> x * 2);   // prints nothing
```

Laziness enables fusion (one pass), short-circuiting on infinite streams and avoiding work for elements nobody consumes.

Q11. Explain the internal structure of a stream pipeline.

Each stage is an `AbstractPipeline` node linked to its upstream; the head holds the source `Spliterator` . The terminal operation calls `wrapSink` , which wraps each stage's operation around the downstream `Sink` , then calls `spliterator.forEachRemaining(sink)` (or a `tryAdvance` loop when short-circuiting). Elements are pushed one by one through all stateless stages; stateful stages like `sorted` buffer elements and release them in `end()` .

Q12. Predict the output.

```
Stream.of("one", "two", "three", "four")
      .filter(s -> { System.out.println("F:" + s); return s.length() > 3; })
```

```
        .map(s -> { System.out.println("M:" + s); return s.toUpperCase(); })
        .findFirst();
```

`F:one, F:two, F:three, M:three`. Execution is vertical (element by element) and `findFirst` stops after the first element that reaches it, so `four` is never touched.

Q13. Predict the output.

```
List<Integer> list = Stream.of(5, 3, 1).peek(System.out::print).sorted().peek(System.out::print).toList();
```

`531135`. The first `peek` sees each element as it enters `sorted`, which buffers all three before emitting `1, 3, 5` to the second `peek`.

Q14. Why does `List.of(1,2,3).stream().peek(System.out::println).count()` print nothing?

Since Java 9, `count()` checks whether the source is `SIZED` and no stage can change the number of elements; if so it returns the size without traversing. `peek` does not change the count, so it is skipped. Add a `filter` and it prints. Lesson: never put required logic in `peek`.

Q15. Can you reuse a stream?

No. A stream can be consumed once; a second terminal (or intermediate) operation throws `IllegalStateException: stream has already been operated upon or closed`. If you need to run several pipelines, keep a `Supplier<Stream<T>>` (`() -> list.stream()`) or collect once and reuse the collection.

Q16. Stateless vs stateful intermediate operations — why does it matter?

Stateless ops (`filter`, `map`, `flatMap`, `peek`) handle each element independently and fuse perfectly, including in parallel. Stateful ops (`sorted`, `distinct`, `limit`, `skip`) must see other elements; `sorted` is a full barrier that buffers the whole stream, `distinct` keeps a set, and `limit/skip` in ordered parallel streams must coordinate chunks. They decide memory use, whether infinite streams can terminate, and how well parallel streams scale.

Q17. What is a Spliterator? What are its characteristics used for?

A splittable iterator: it traverses (`tryAdvance`, `forEachRemaining`) and splits itself for parallelism (`trySplit`). Characteristics (`SIZED`, `SUBSIZED`, `ORDERED`, `SORTED`, `DISTINCT`, `NONNULL`, `IMMUTABLE`, `CONCURRENT`) let the framework skip work: exact-size arrays for `toArray`, no-op `distinct()` on a `Set`, no-op `sorted()` on a `TreeSet`, shortcut `count()`. Custom data sources (e.g., a paged API reader) can expose a stream through `StreamSupport.stream(spliterator, false)`.

Q18. What does short-circuiting mean? Give examples.

An operation that can complete without processing all elements: `findFirst`, `findAny`, `anyMatch`, `allMatch`, `noneMatch` (terminal), `limit`, `takeWhile` (intermediate). `allMatch` stops at the first `false`, `anyMatch` at the first `true`. On an empty stream `allMatch` returns `true` and `anyMatch` returns `false` (vacuous truth).

Q19. Will this terminate?

```
Stream.iterate(1, i -> i + 1).filter(i -> i % 2 == 0).limit(3).forEach(System.out::println);   // A
Stream.iterate(1, i -> i + 1).sorted().limit(3).forEach(System.out::println);                  // B
IntStream.iterate(0, i -> (i + 1) % 2).distinct().limit(10).forEach(System.out::println);       // C
```

A prints 2, 4, 6 and stops. B never terminates (`sorted` must consume the infinite stream; eventually `OutOfMemoryError`). C prints 0 and 1 and then hangs: `distinct` never finds a third distinct value, so `limit(10)` is never satisfied. Use

`Stream.iterate(seed, hasNext, next)` (Java 9) or `takeWhile` to bound infinite streams.

Q20. `map` vs `flatMap`?

`map` transforms each element into exactly one element (`Stream<List<T>>` stays nested). `flatMap` transforms each element into a stream and concatenates them (`Stream<T>`). Same idea for `Optional.flatMap`, which avoids `Optional<Optional<T>>` when the mapper itself returns an `Optional`.

Q21. What is `mapMulti` and when would you use it over `flatMap`?

Java 16. The mapper receives the element and a `Consumer` and pushes zero or more outputs. It avoids creating a `Stream` per element, which helps when most elements map to zero or one results, and it makes type-filtering easy: `objects.stream().<Txn>mapMulti((o, sink) -> { if (o instanceof Txn t) sink.accept(t); })`.

Q22. `reduce` vs `collect`?

`reduce` is an immutable reduction: combine values into a new value (sum, max, `BigDecimal` totals). `collect` is a mutable reduction: accumulate into a container (lists, maps, `StringBuilder`). Using `reduce` to build a list either copies the list at every step ($O(n^2)$) or, with a shared mutable identity, breaks in parallel.

Q23. What are the three overloads of `reduce`?

`reduce(BinaryOperator)` returns `Optional<T>` (no identity, empty stream → empty). `reduce(identity, BinaryOperator)` returns `T`. `reduce(identity, BiFunction accumulator, BinaryOperator combiner)` lets the result type differ from the element type; the combiner merges partial results in parallel. Requirements: identity must satisfy `combiner(identity, x) == x`, operations must be associative and stateless.

Q24. What is wrong here?

```
int sum = List.of(1, 2, 3, 4).parallelStream().reduce(10, Integer::sum);
```

Sequentially it returns 20; in parallel it typically returns something like 50, because 10 is not an identity for addition and is applied once per chunk. Add the offset outside: `10 + list.stream().mapToInt(i -> i).sum()`.

Q25. `Collectors.toList()` vs `Stream.toList()` vs `toUnmodifiableList()`?

`Collectors.toList()` returns an unspecified list, in practice a mutable `ArrayList`, nulls allowed. `toUnmodifiableList()` (Java 10) is unmodifiable and throws NPE on null elements. `Stream.toList()` (Java 16) is unmodifiable and allows nulls. Switching `collect(toList())` to `.toList()` in old code breaks callers that later call `add` on the result — the most common surprise when modernising.

Q26. What happens with duplicate keys in `Collectors.toMap`? With null values?

Duplicate keys throw `IllegalStateException: Duplicate key …` unless you pass a merge function `(a, b) -> …`. Null values throw `NullPointerException` because the implementation uses `Map.merge`. The four-argument form also takes a map supplier (`LinkedHashMap::new`, `TreeMap::new`, `() -> new EnumMap<>(X.class)`).

Q27. `groupingBy` vs `partitioningBy`?

`groupingBy` groups by any key and contains only keys that occurred. `partitioningBy` takes a `Predicate`, the map always has exactly two keys (`true` and `false`), even when one side is empty, and its implementation is a specialised two-slot map. Both accept downstream collectors.

Q28. What is a downstream collector? Give five examples.

The collector applied to each group's elements inside `groupingBy` / `partitioningBy`: `counting()`, `summingLong(...)`, `mapping(f, toSet())`, `maxBy(comparator)`, `reducing(identity, mapper, op)`, `filtering(pred, toList())`, `flatMapping(...)`, `collectingAndThen(...)`, `averagingDouble(...)`, `joining(", ")`, or another `groupingBy` for nested maps.

Q29. `filter` before `groupingBy` vs `filtering` as downstream — any difference?

Yes. `filter` first removes elements, so a group whose elements were all removed disappears from the map. `filtering` keeps every key and gives it an empty list. If the UI must show every status even with zero items, use `filtering` (or start from an `EnumMap` pre-filled with zeros).

Q30. What is `Collectors.teeing`?

Java 12. Sends each element to two collectors and merges both results with a `BiFunction`, all in one pass: average = `teeing(summingDouble(x), counting(), (sum, n) -> sum / n)`, or min and max together, or debit and credit totals.

Q31. What does `collectingAndThen` do?

Applies a finishing function to a collector's result: make a list unmodifiable, unwrap the `Optional` from `maxBy` inside a `groupingBy` (safe because a group is never empty), or convert a list to a custom type.

Q32. How do you write a custom Collector?

Implement `Collector<T, A, R>` or use `Collector.of(supplier, accumulator, combiner, [finisher], characteristics…)`. The supplier must return a fresh container each time; the combiner must merge two partial containers correctly (it is only called in parallel, so bugs there hide until someone adds `.parallel()`); characteristics `IDENTITY_FINISH`, `UNORDERED`, `CONCURRENT` tell the framework what it may optimise.

Q33. How does a parallel stream work internally?

The source spliterator is split recursively via `trySplit` into chunks, each processed as a `ForkJoinTask` on the common `ForkJoinPool` (parallelism = cores − 1, plus the calling thread). Idle workers steal tasks from the tail of busy workers' deques. Partial results are combined up the task tree (combiner for collect/reduce). Ordered terminals must also stitch results in encounter order.

Q34. When should you not use parallel streams?

Small data or cheap per-element work, sources that split badly (`LinkedList`, `Stream.iterate`, I/O-backed streams), blocking I/O in the lambdas, code running in request threads of a server (shared common pool), order-sensitive operations (`findFirst`, `limit`, `sorted`), and any lambda with side effects on shared state. Measure before and after; most business code gets no benefit.

Q35. Why is blocking I/O in a parallel stream dangerous in a Spring Boot app?

All parallel streams and all `CompletableFuture.supplyAsync` calls without an explicit executor share the **one** common pool, sized to CPU count. A parallel stream that makes HTTP calls parks those few workers on I/O; every other part of the application that relies on the common pool stalls. Use a dedicated executor (or virtual threads in Java 21) for I/O fan-out.

Q36. Predict the result.

```
List<Integer> out = new ArrayList<>();
IntStream.range(0, 1000).parallel().forEach(out::add);
System.out.println(out.size());
```

Unpredictable: a number below 1000, or an `ArrayIndexOutOfBoundsException` when two threads resize the array concurrently. `ArrayList` is not thread-safe. Correct: `IntStream.range(0, 1000).parallel().boxed().toList()`.

Q37. `findFirst` vs `findAny`? `forEach` vs `forEachOrdered`?

`findFirst` returns the first element in encounter order (expensive to guarantee in parallel); `findAny` may return any element (cheaper in parallel; sequentially it usually returns the first, but do not rely on it). `forEach` in parallel processes elements in arbitrary order on many threads; `forEachOrdered` respects encounter order, losing much of the parallel benefit.

Q38. Is a parallel stream's result order guaranteed?

For an ordered source, operations like `collect(toList())`, `toList()`, `toArray`, `reduce` and `findFirst` preserve encounter order even in parallel; the framework merges partial results in order. Execution order (which element is processed when, by which thread) is not deterministic, and `forEach` / `findAny` / `groupingByConcurrent` give up ordering.

Q39. Why use `IntStream` instead of `Stream<Integer>`?

To avoid boxing: each `Integer` is a heap object with a header and a pointer indirection. Primitive streams also give numeric terminals (`sum`, `average`, `summaryStatistics`) and `range/rangeClosed`. Convert with `mapToInt`, `mapToObj`, `boxed()`, `asLongStream()`.

Q40. Is a stream faster than a for loop?

Usually not for small collections — a loop has less overhead. For large data, sequential streams with primitive specialisations come close to loops after JIT inlining. Streams are chosen for clarity of transformation logic and easy (but rarely useful) parallelism. A senior answer mentions JMH, boxing as the real cost, and that I/O dominates in business code anyway.

Q41. Why does `Optional` exist and where should it be used?

To make "might be absent" explicit in method **return types** so callers must handle it, replacing `null` returns. Not designed for fields, parameters, collections or serialisation. Repository finders, cache lookups, "find first matching rule" are good uses.

Q42. `orElse` vs `orElseGet` — predict the side effect.

```
Optional.of("x").orElse(create());       // create() runs
Optional.of("x").orElseGet(this::create); // create() does not run
```

`orElse` takes a value, so its argument is evaluated before the call, even when the Optional is full. `orElseGet` takes a `Supplier` and calls it only when empty. Use `orElseGet` for anything expensive or with side effects.

Q43. `Optional.of` vs `Optional.ofNullable` vs `Optional.empty`?

`of(x)` throws NPE when `x` is null — use it when null would be a bug. `ofNullable(x)` returns empty for null. `empty()` is the singleton empty instance. `orElseThrow()` (Java 10) is the readable replacement for `get()`.

Q44. Why shouldn't you use `Optional` as a field or method parameter?

As a field: `Optional` is not `Serializable`, JPA cannot map it, Jackson needs a module, and it costs an extra object per instance. As a parameter: the caller must wrap values, and the parameter now has three states (null, empty, present), which is worse than an overload or a nullable parameter. Use a nullable field with an `Optional`-returning getter if you want the API benefit.

Q45. What does `Optional.map` return when the mapper returns null?

`Optional.empty()`. `map` wraps the result with `ofNullable`, so a getter returning null safely ends the chain: `user.map(User::address).map(Address::city).orElse("unknown")`.

Q46. How do you handle checked exceptions inside a stream lambda?

Wrap them: a helper method that catches `IOException` and throws `UncheckedIOException`, or an adapter `unchecked(ThrowingFunction)` that converts any checked exception to a runtime one. If the pipeline must continue past errors, map each element to a result type (`Ok` / `Failed` records in a sealed interface) and partition at the end. Sneaky throws hide the exception from the compiler and should be avoided in application code. For complex error handling a loop is often clearer.

Q47. Do you need to close a stream?

Only resource-backed streams: `Files.lines`, `Files.list`, `Files.walk`, `BufferedReader.lines` over a file, `JdbcTemplate.queryForStream`, Spring Data `Stream<T>` repository methods (which also need an open transaction). Use try-with-resources. Collection-backed streams hold nothing to release.

Q48. Write: find the second highest salary.

```
Optional<BigDecimal> second = employees.stream()
        .map(Employee::salary)
        .distinct()
        .sorted(Comparator.reverseOrder())
        .skip(1)
        .findFirst();
```

`distinct()` handles ties (two people earning the top salary). Without it, `skip(1)` returns the top salary again. For the employee(s) with the second highest salary, group by salary into a `TreeMap` and take the second key's list. An O(n) alternative without sorting tracks the top two values in a single loop.

Q49. Write: group employees by department, count them, and find the highest-paid per department.

```
Map<String, Long> count = employees.stream().collect(groupingBy(Employee::dept, counting()));

Map<String, Optional<Employee>> top = employees.stream()
        .collect(groupingBy(Employee::dept, maxBy(comparing(Employee::salary))));

Map<String, String> topName = employees.stream()
        .collect(groupingBy(Employee::dept,
                 collectingAndThen(maxBy(comparing(Employee::salary)), o -> o.get().name())));

Map<String, Double> avg = employees.stream().collect(groupingBy(Employee::dept, averagingDouble(e ->
e.salary().doubleValue())));
```

Q50. Write: frequency of each character in a string, in order of first appearance.

```
Map<Character, Long> freq = "programming".chars()
        .mapToObj(c -> (char) c)
        .collect(groupingBy(Function.identity(), LinkedHashMap::new, counting()));
// {p=1, r=2, o=1, g=2, a=1, m=2, i=1, n=1}
```

`chars()` gives an `IntStream` of UTF-16 code units; for text with emoji use `codePoints()` and `Character.toString(int)`.

Q51. Write: first non-repeating character.

```
Optional<Character> first = input.chars()
        .mapToObj(c -> (char) c)
        .collect(groupingBy(Function.identity(), LinkedHashMap::new, counting()))
        .entrySet().stream()
        .filter(e -> e.getValue() == 1)
        .map(Map.Entry::getKey)
        .findFirst();
// "swiss" -> 'w'
```

The `LinkedHashMap` supplier is the whole trick: a plain `HashMap` loses insertion order, so "first" becomes arbitrary. For the first *repeating* character, use a `HashSet` and `filter(c -> !seen.add(c))` — a stateful lambda, acceptable only in a sequential stream.

Q52. Write: find duplicates in a list of integers.

```
Set<Integer> dups = nums.stream()
        .collect(groupingBy(Function.identity(), counting()))
        .entrySet().stream()
        .filter(e -> e.getValue() > 1)
        .map(Map.Entry::getKey)
        .collect(toSet());

// or, sequential only:
Set<Integer> seen = new HashSet<>();
Set<Integer> dups2 = nums.stream().filter(n -> !seen.add(n)).collect(toSet());
```

Q53. Write: join names of active users, sorted, comma-separated; and the longest word in a sentence.

```
String csv = users.stream().filter(User::active).map(User::name).sorted().collect(joining(", "));

String longest = Arrays.stream(sentence.split("\\s+"))
        .max(Comparator.comparingInt(String::length))
        .orElse("");
```

Q54. Write: flatten a list of lists and sum all numbers; and convert a list to a map of id → object, keeping the last on duplicates.

```
int sum = listOfLists.stream().flatMap(List::stream).mapToInt(Integer::intValue).sum();

Map<Long, Order> byId = orders.stream()
        .collect(toMap(Order::id, Function.identity(), (first, second) -> second));
```

Q55. Write: partition numbers into even and odd; sum squares of odd numbers.

```
Map<Boolean, List<Integer>> parts = nums.stream().collect(partitioningBy(n -> n % 2 == 0));
```

```
int sumSquaresOdd = nums.stream().filter(n -> n % 2 != 0).mapToInt(n -> n * n).sum();
```

Watch negative numbers: `-3 % 2` is `-1`, so test `n % 2 != 0`, not `n % 2 == 1`.

Q56. Write: top 3 most frequent words in a text.

```
List<String> top3 = Arrays.stream(text.toLowerCase().split("\\W+"))
        .filter(w -> !w.isBlank())
        .collect(groupingBy(Function.identity(), counting()))
        .entrySet().stream()
        .sorted(Map.Entry.<String, Long>comparingByValue().reversed()
                .thenComparing(Map.Entry.comparingByKey()))
        .limit(3)
        .map(Map.Entry::getKey)
        .toList();
```

The explicit type witness `Map.Entry.<String, Long>comparingByValue()` is needed because `.reversed()` breaks the target-type inference chain — a detail interviewers like to probe.

Q57. Write: check whether a string is a palindrome, and reverse each word in a sentence, using streams.

```
boolean palindrome = IntStream.range(0, s.length() / 2)
        .allMatch(i -> s.charAt(i) == s.charAt(s.length() - 1 - i));

String reversedWords = Arrays.stream(sentence.split(" "))
        .map(w -> new StringBuilder(w).reverse().toString())
        .collect(joining(" "));
```

Q58. Write: sort a list of employees by department, then salary descending, then name.

```
List<Employee> sorted = employees.stream()
        .sorted(Comparator.comparing(Employee::dept)
                .thenComparing(Employee::salary, Comparator.reverseOrder())
                .thenComparing(Employee::name))
        .toList();
```

`comparing(Employee::salary).reversed().thenComparing(...)` is a common bug when `reversed()` is applied to the whole chain after more comparators were added — it reverses everything before it.

Q59. Write: sum transaction amounts per currency with `BigDecimal`, and explain why not `double`.

```
Map<String, BigDecimal> totals = txns.stream()
        .collect(groupingBy(Txn::currency, TreeMap::new,
                reducing(BigDecimal.ZERO, Txn::amount, BigDecimal::add)));
```

`double` is binary floating point; values like 0.1 have no exact representation, and errors accumulate over thousands of additions. In finance a 0.01 discrepancy is a defect. `BigDecimal` is exact for decimal values; just remember `equals` compares scale (`2.0` ≠ `2.00`), so compare amounts with `compareTo`.

Q60. How did you use streams in your project?

In RefinePoint we use streams for in-memory shaping, not for bulk data: grouping result items per run with `groupingBy(Item::runId, LinkedHashMap::new, toList())`, totalling amounts with `reduce(BigDecimal.ZERO, BigDecimal::add)` in the certification and report code, indexing records with `toMap` and an explicit merge function, and mapping entities to record DTOs. There is no `parallelStream` anywhere in the services. Bulk matching of millions of rows

goes to DuckDB via SQL compiled from match rules, because no Java stream beats a vectorised columnar engine on that workload.

Q61. What is wrong with this code from a code review?

```
List<String> result = new ArrayList<>();
orders.stream().filter(Order::paid).forEach(o -> result.add(o.id()));
```

It uses a stream for iteration with side effects into an external mutable list — correct only sequentially, and it reads like a loop wearing a costume. Write `List<String> result = orders.stream().filter(Order::paid).map(Order::id).toList();`. Side effects belong in `forEach` only at the very end of a pipeline (logging, sending), never as a way of collecting.

Q62. Can a lambda throw an exception? What happens to an exception inside a parallel stream?

A lambda may throw unchecked exceptions freely and checked exceptions only if the functional interface declares them. In a parallel stream an exception in any worker is propagated to the caller of the terminal operation (wrapped/re-thrown by the ForkJoin framework); other tasks may still be running briefly, and there is no guarantee which element's exception you see if several fail.

Q63. What is the difference between `Stream.of(array)` and `Arrays.stream(array)` for a primitive array?

`Stream.of(intArray)` produces a `Stream<int[]>` with **one** element (the array itself), because `of(T... values)` cannot take primitives as `T`. `Arrays.stream(intArray)` produces an `IntStream` of the elements. Classic output-prediction trap: `Stream.of(new int[]{1,2,3}).count()` is `1`.

Q64. Is a lambda serializable? Can a lambda be a Spring bean?

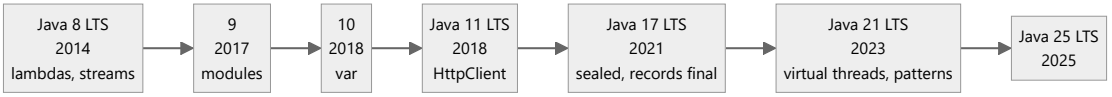
Only if its target type is serializable, e.g., an intersection cast `(Runnable & Serializable) () -> …`; then the compiler generates a `$deserializeLambda$` method and serialises a `SerializedLambda`. It is fragile across recompilations, so avoid it. A lambda can absolutely be a Spring bean: `@Bean Function<String, String> normaliser() { return String::trim; }` — Spring Cloud Function is built on that.

7. Java 9 to 21: Every Feature an Interviewer Asks About

"Which Java version do you use, and what did you actually use from it?" is now a standard opener. A candidate who says "Java 17" and then cannot explain a sealed interface or a record's compact constructor loses credibility in the first five minutes. This chapter walks the releases from 9 to 21 (with a short look beyond), explains the features that changed how we write code — records, sealed types, pattern matching — at the level of *why they exist and how they compile*, and ends with the migration story every senior engineer should be able to tell: Java 8/11 + Spring Boot 2 to Java 21 + Spring Boot 3.

7.1 The release model and the LTS line

Since Java 9 (2017) a new feature release ships every **six months** (March and September). Every few releases is a **Long-Term Support** (LTS) release that vendors (Oracle, Eclipse Adoptium/Temurin, Amazon Corretto, Azul, Microsoft, Red Hat) patch for years. Since Java 17 the LTS cadence is every two years.



Features move through stages: **incubator** (a separate `jdk.incubator.*` module, API may change completely), **preview** (in the language/JVM but needs `--enable-preview` at compile *and* run time, may change or be withdrawn), then **final** (standard). Interviewers like asking "is X final in 21?" — the table below is the cheat sheet.

Version	Type	Features you must know
9 (2017)		JPMS modules, JShell, private interface methods, `List.of/Set.of/Map.of`, `Stream.takeWhile/dropWhile/ofNullable`, `iterate` with predicate, `Optional.or/stream/ifPresentOrElse`, compact strings, G1 default GC, indy string concatenation, `ProcessHandle`, `Flow` (reactive streams), try-with-resources on effectively final variables, `CompletableFuture.orTimeout`
10 (2018)		`var` for local variables, `List.copyOf`, `Collectors.toUnmodifiableList`, `Optional.orElseThrow()`, parallel full GC in G1, application class-data sharing, better container awareness
11 (2018)	**LTS**	Standard `HttpClient`, `String.isBlank/strip/lines/repeat`, `Files.readString/writeString`, `var` in lambda parameters, `java Hello.java` single-file launch, `Predicate.not`, `Optional.isEmpty`, Epsilon GC, ZGC (experimental), Flight Recorder open-sourced, **Java EE and CORBA modules removed**
12		Switch expressions (preview), `Collectors.teeing`, `String.indent/transform`, Shenandoah (experimental)
13		Text blocks (preview), `yield`
14		**Switch expressions final**, **helpful NullPointerExceptions**, records (preview), `instanceof` patterns (preview), CMS GC removed
15		**Text blocks final**, sealed classes (preview), hidden classes, ZGC and Shenandoah production-ready, **Nashorn removed**, biased locking disabled
16		**Records final**, **`instanceof` pattern matching final**, `Stream.toList()`, `mapMulti`, strong encapsulation by default, `jpackage`, Unix-domain sockets
17 (2021)	**LTS**	**Sealed classes final**, strong encapsulation of JDK internals (`--illegal-access` gone), pattern matching for `switch` (preview), `RandomGenerator` API, SecurityManager deprecated for removal, Applet API deprecated for removal, RMI activation removed
18–20		UTF-8 by default (18), simple web server `jwebserver` (18), `finalize` deprecated for removal (18), virtual threads / record patterns / structured concurrency previews (19–20)
21 (2023)	**LTS**	**Virtual threads**, **pattern matching for `switch`**, **record patterns**, **sequenced collections**, **generational ZGC**, Key Encapsulation Mechanism API; previews: string templates, unnamed patterns and variables, unnamed classes and instance main, structured concurrency, scoped values

For awareness beyond 21: Java 22 finalised unnamed variables `_` and the Foreign Function & Memory API; Java 23 *withdrew* string templates; Java 24 made `synchronized` no longer pin virtual threads and finalised stream gatherers; Java 25 (LTS, September 2025) finalised scoped values, flexible constructor bodies (statements before `super(...)`), compact source files with instance `main` methods, and module import declarations.

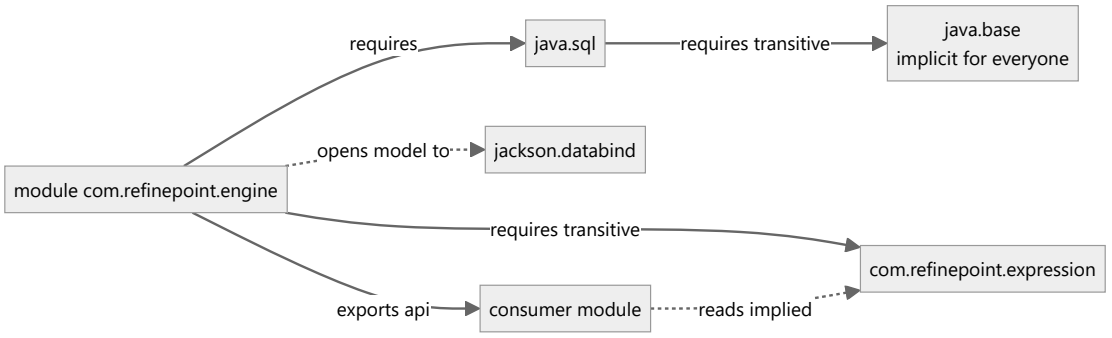
> **Interview tip:** when asked "what's new in Java 21", lead with the *story*, not a list: "21 completes the data-oriented programming features that started in 14 — records, sealed types and pattern matching now work together with exhaustive switches — and it brings virtual threads, which change how we write blocking I/O code." Then give examples.

7.2 Java 9: modules (JPMS) and the small things

The module system

Before 9, the classpath was one flat list of JARs: any public class was visible to everyone, two JARs with the same package silently merged (or shadowed each other — "JAR hell"), and missing dependencies surfaced as `NoClassDefFoundError` at runtime. JPMS (Project Jigsaw) adds a **module**: a named set of packages with an explicit descriptor.

```
// module-info.java at the root of the source tree (illustrative: RefinePoint itself runs on the classpath)
module com.refinepoint.engine {
    requires java.sql;                          // read dependency
    requires transitive com.refinepoint.expression;  // our consumers also read it
    exports com.refinepoint.engine.api;         // only this package is accessible to other modules
    exports com.refinepoint.engine.spi to com.refinepoint.plugins;   // qualified export
    opens com.refinepoint.engine.model to com.fasterxml.jackson.databind;  // deep reflection at runtime only
    uses com.refinepoint.plugin.DataConnector;             // ServiceLoader consumer
    provides com.refinepoint.plugin.DataConnector with com.refinepoint.engine.CsvConnector;
}
```



Key ideas:

- **Strong encapsulation:** `public` is no longer enough — a public class in a non-exported package is inaccessible from other modules, even with reflection (unless `opens`).
- **Reliable configuration:** the module graph is resolved at startup; a missing module or two modules exporting the same package (**split package**) fails fast.
- `exports` gives compile-time and runtime access to public types; `opens` gives runtime *deep reflection* (private members) — what Jackson, Hibernate and Spring need. `open module` opens every package.
- Code on the classpath lives in the **unnamed module** (reads everything, exports everything). A plain JAR placed on the module path becomes an **automatic module** (name from the `Automatic-Module-Name` manifest entry or the file name), exports all packages and reads all modules — the bridge for migration.
- `jlink` builds a custom runtime image containing only the modules you use (a 40–60 MB JRE instead of 300 MB); `jdeps` analyses dependencies, including use of JDK internals.

Do Spring Boot applications use modules? Almost never. Spring relies on classpath scanning, reflection and runtime proxies; frameworks and most libraries run happily in the unnamed module. The JDK itself is modular, which is the part that

matters to everyone: it is why `sun.misc.*` / `com.sun.*` internals became inaccessible and why Java EE modules could be removed in 11.

The smaller Java 9 features

```
// Collection factories — immutable, null-hostile, randomised iteration order for Set/Map
List<String> roles = List.of("ADMIN", "APPROVER");
Map<String, Integer> limits = Map.of("FREE", 3, "PRO", 50);          // up to 10 pairs
Map<String, Integer> big = Map.ofEntries(Map.entry("A", 1), Map.entry("B", 2));
Set.of("a", "a");      // IllegalArgumentException: duplicate element
List.of("a", null);    // NullPointerException
roles.contains(null);  // NullPointerException too!

// Private interface methods — share code between default methods
interface Exporter {
    default byte[] csv(List<Row> rows)  { return encode(rows, ','); }
    default byte[] tsv(List<Row> rows)  { return encode(rows, '\t'); }
    private byte[] encode(List<Row> rows, char sep) { ... }
}

// try-with-resources on an effectively final variable
BufferedReader reader = Files.newBufferedReader(path);
try (reader) { ... }

// Diamond with anonymous classes
Comparator<Run> c = new Comparator<>() { public int compare(Run a, Run b) { ... } };
```

JShell is the REPL (`jshell`), handy for trying an API (`"a,b".split(",").length`) or a regex during an interview. **Compact strings** (JEP 254) store Latin-1 strings as one byte per char, roughly halving `String` memory in typical apps — internal, no API change.

7.3 Java 10: `var`

`var` is **local variable type inference**: the compiler infers the static type from the initializer. It is still static typing — the type is fixed at compile time and appears in the bytecode exactly as if you wrote it.

```
var runs = new ArrayList<ReconRun>();          // ArrayList<ReconRun>, not List<ReconRun>
for (var entry : totals.entrySet()) { ... }    // Map.Entry<String, BigDecimal>
try (var in = Files.newInputStream(p)) { ... }
var counter = new Object() { int hits; };      // non-denotable anonymous type: counter.hits works
```

Allowed	Not allowed
Local variables **with an initializer**	Fields, method parameters, return types
Enhanced `for` and classic `for` index	`var x;` (no initializer)
try-with-resources resources	`var x = null;` (no type to infer)
Lambda parameters (Java 11), to add annotations: `(@NonNull var s) -> …`	`var a = {1, 2};` (array initializer needs a type)
	`var f = s -> s.length();` (lambda needs a target type)
	`var` mixed with explicit types in one lambda's params

Traps:

```
var list = new ArrayList<>();      // ArrayList<Object> — diamond with no target infers Object
var n = 10;  n = 10L;              // compile error: n is int
var id = 1_000_000_000_0L;         // long — the literal decides
var ids = List.of(1, 2L);          // List<Number & Comparable<...>> — an intersection type
```

`var` is a **reserved type name**, not a keyword: `int var = 5;` still compiles, but a class named `var` does not. Use it where the type is obvious from the right-hand side (`new` , factory with a clear name); avoid it when the type carries meaning the reader needs (`var result = service.process(x);`).

7.4 Java 11: HttpClient and String methods

HttpClient

`java.net.http.HttpClient` replaced `HttpURLConnection` (incubated in 9, standard in 11). It supports HTTP/1.1 and HTTP/2, synchronous and asynchronous calls, WebSocket, and is immutable and thread-safe — create one and reuse it.

```
HttpClient client = HttpClient.newBuilder()
        .version(HttpClient.Version.HTTP_2)
        .connectTimeout(Duration.ofSeconds(5))
        .executor(Executors.newVirtualThreadPerTaskExecutor())      // Java 21
        .build();

HttpRequest req = HttpRequest.newBuilder(URI.create("https://api.example.com/rates?ccy=EUR"))
        .timeout(Duration.ofSeconds(10))                            // per-request (response) timeout
        .header("Accept", "application/json")
        .GET()
        .build();

HttpResponse<String> res = client.send(req, HttpResponse.BodyHandlers.ofString());          // blocking
CompletableFuture<String> body = client.sendAsync(req, HttpResponse.BodyHandlers.ofString())
        .thenApply(HttpResponse::body);                                                      // async
```

In Spring Boot 3 you rarely call it directly, but Spring 6.1's `RestClient` can use it underneath (`JdkClientHttpRequestFactory`), which removes the need for Apache HttpClient in many services. Note it does not throw on 4xx/5xx — check `statusCode()` yourself.

New String methods

Method	Since	Note
`isBlank()`	11	true for empty or only whitespace (`Character.isWhitespace`)
`strip()` , `stripLeading()` , `stripTrailing()`	11	Unicode-aware whitespace; `trim()` only removes chars ≤ U+0020
`lines()`	11	Lazy `Stream<String>` split on `\n` , `\r` , `\r\n`
`repeat(n)`	11	`"-".repeat(20)`
`indent(n)` , `transform(f)`	12	`transform` applies any function: `s.transform(String::strip)`
`stripIndent()` , `translateEscapes()` , `formatted(args)`	15	`"%s-%d".formatted(code, n)`
`splitWithDelimiters` , `indexOf(ch, from, to)`	21	Also `StringBuilder.repeat` in 21

```
"  hello  ".trim();    // unchanged: EM SPACE (U+2003) is above U+0020
"  hello  ".strip();   // "hello"
"   ".isEmpty();               // false
"   ".isBlank();               // true
```

Java 11 also removed the Java EE modules (`java.xml.bind` JAXB, `java.xml.ws` JAX-WS, `java.activation` , `java.corba` , `java.transaction`). Upgrading from 8 to 11 classically fails with `ClassNotFoundException: javax.xml.bind.JAXBException` — fix by adding the libraries as normal dependencies (today the `jakarta.xml.bind` artifacts).

7.5 Switch expressions (14) and text blocks (15)

Switch expressions

```
// Old statement: fall-through, break noise, no value
String label;
switch (status) {
    case OPEN: case REOPENED: label = "Needs action"; break;
    case RESOLVED: label = "Done"; break;
    default: label = "Unknown";
}

// Switch expression: arrow labels, no fall-through, yields a value, exhaustive
String label = switch (status) {
    case OPEN, REOPENED -> "Needs action";
    case RESOLVED -> "Done";
    case IN_REVIEW -> {
        log.debug("review pending");
        yield "Waiting for approver";      // yield returns a value from a block
    }
};
```

Rules: a switch **expression** must be exhaustive — for an enum, cover every constant or add `default`. If you cover every constant without `default`, the compiler inserts a hidden default that throws (`MatchException` in Java 21; `IncompatibleClassChangeError` before) when a new constant appears at runtime from a recompiled enum. That is the desired behaviour: add a constant, and every switch without `default` fails to *compile* on the next build, showing you every place to update. Arrow-form `case` never falls through; `yield` is a contextual keyword used only inside switch expressions. Arrow labels can also be used in switch *statements*.

Text blocks

```
String sql = """
        SELECT r.id, r.status, count(b.id) AS breaks
        FROM recon_run r
        LEFT JOIN recon_break b ON b.run_id = r.id
        WHERE r.org_id = ?
        GROUP BY r.id, r.status
        """;
```

How the compiler processes a text block: (1) line terminators normalised to `\n`; (2) **incidental indentation removed** — the minimum indentation across all non-blank lines *and the closing delimiter* is stripped (so moving the closing `"""` left adds indentation to the content); (3) trailing spaces on each line removed; (4) escape sequences interpreted. The opening `"""` must be followed by a line break. New escapes: `\<newline>` joins lines (no `\n`), `\s` is a space that survives trailing-space stripping. A text block is an ordinary `String` constant, interned like any literal. Combine with `formatted()` for parameters — never build SQL with values via formatting; use bind parameters.

7.6 Records (16)

A **record** is a transparent, shallowly immutable carrier for a fixed set of values. The compiler generates the private final fields, a canonical constructor, accessors named after the components (`amount()`, not `getAmount()`), and `equals`, `hashCode`, `toString` based on all components.

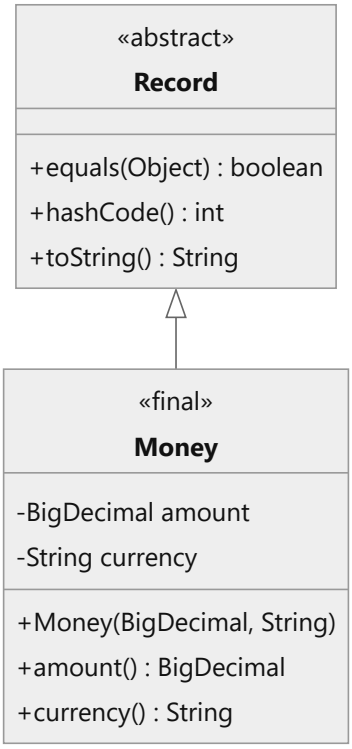
```
public record Money(BigDecimal amount, String currency) {

    public Money {                                      // compact canonical constructor
        Objects.requireNonNull(amount, "amount");
        if (currency == null || currency.length() != 3) throw new IllegalArgumentException("ISO currency
required");
        amount = amount.setScale(2, RoundingMode.HALF_EVEN);  // reassign the PARAMETER; fields are assigned
```

```
after the body
        currency = currency.toUpperCase();
    }

    public Money(String amount, String currency) {      // extra constructor must delegate to the canonical one
        this(new BigDecimal(amount), currency);
    }

    public static Money zero(String ccy) { return new Money(BigDecimal.ZERO, ccy); }   // static factory
    public Money plus(Money o) {                                                         // instance methods
are fine
        if (!currency.equals(o.currency)) throw new IllegalArgumentException("currency mismatch");
        return new Money(amount.add(o.amount), currency);
    }
}
```



What the compiler generates and enforces:

- The class is implicitly `final` and extends `java.lang.Record` — no `extends` allowed, but `implements` is.
- Each component becomes a `private final` field; you **cannot declare other instance fields** (static fields are fine).
- The **canonical constructor** takes all components in order. The **compact** form omits the parameter list and the field assignments; the body runs first and assigns the (possibly reassigned) parameters to fields at the end. You cannot assign `this.amount` in a compact constructor.
- Accessors can be overridden (e.g., to return a defensive copy) but must be `public` with the exact return type.
- `equals/hashCode/toString` are generated through `invokedynamic` with `ObjectMethods` as bootstrap — the same trick as lambdas.
- Records can be nested (implicitly `static`), local (declared inside a method — great for intermediate stream results), generic, and can implement interfaces including sealed ones.
- Serialization of a record always goes through the canonical constructor, so validation cannot be bypassed by a crafted byte stream — a security improvement over normal classes.

Limits: shallow immutability only — a `List` component can still be mutated unless you copy it (`items = List.copyOf(items)`); arrays as components have identity-based `equals` ; no inheritance; no lazy or derived fields (compute in a method or in the constructor and make it a component).

Records as DTOs, with Jackson and JPA

```
public record CreateReconRequest(
        @NotBlank String name,
        @NotNull UUID leftSourceId,
        @NotNull UUID rightSourceId,
        @Size(max = 20) List<@Valid MatchRuleDto> rules) {}

@PostMapping("/orgs/{orgCode}/recons")
public ApiResponse<ReconDto> create(@PathVariable String orgCode, @Valid @RequestBody CreateReconRequest req) {
... }
```

- **Jackson** 2.12+ deserialises records through the canonical constructor using component names — no `@JsonCreator` needed; Spring Boot 3 ships a compatible version. Bean Validation annotations on components work with `@Valid`.
- **JPA entities cannot be records**: JPA needs a no-arg constructor, mutable state for dirty checking, and non-final classes for lazy-loading proxies. Records are excellent for **projections**: `SELECT new com.x.RunSummary(r.id, r.status, r.startedAt) FROM ReconRun r`, Spring Data class-based projections (`List<RunSummary> findByOrgId(UUID orgId)`), and `JdbcTemplate` row mappers (`(rs, i) -> new RunSummary(...)`). Hibernate 6.2+ also supports records as `@Embeddable` value objects.
- Spring Boot `@ConfigurationProperties` binds to records via constructor binding (no `@ConstructorBinding` needed since Boot 3 when there is a single constructor).

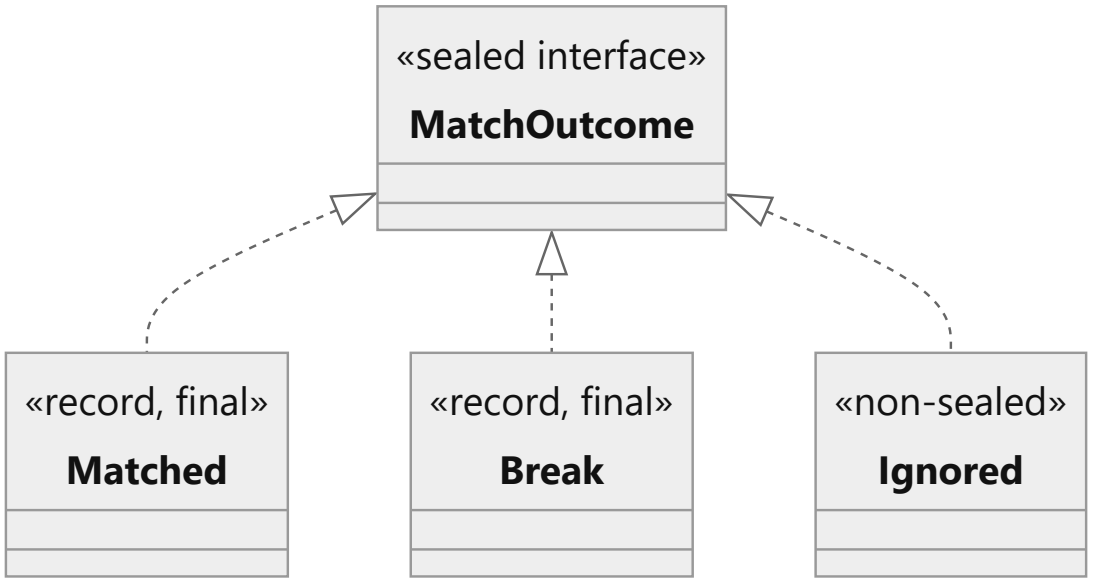
> **RefinePoint:** the AI service models its whole provider contract with records — `LlmRequest`, `LlmResponse`, `Message`, `Attachment` — so `AnthropicClient` and `OpenAiCompatibleClient` exchange immutable values and the structured JSON returned by the model is mapped straight onto records by Jackson. Entities stay classes extending `BaseOrgEntity`; everything crossing an API or service boundary is a record.

7.7 Sealed classes (17)

A **sealed** class or interface restricts which classes may extend or implement it.

```
public sealed interface MatchOutcome permits Matched, Break, Ignored {}

public record Matched(UUID leftId, UUID rightId, String rule) implements MatchOutcome {}
public record Break(UUID id, BreakReason reason, BigDecimal difference) implements MatchOutcome {}
public non-sealed class Ignored implements MatchOutcome { ... }       // reopened for extension
```



Rules:

- Every permitted subclass must be `final`, `sealed` (with its own `permits`), or `non-sealed`. Records and enums are implicitly final.
- Permitted subclasses must be in the **same module**, or in the **same package** if the code is in the unnamed module (normal classpath apps).
- `permits` can be omitted when all subclasses are declared in the same source file.
- `Class.isSealed()` and `getPermittedSubclasses()` expose the hierarchy reflectively; the JVM enforces it at class load (`PermittedSubclasses` attribute), so it is not merely a compiler check.

Why: sealed types make a hierarchy a *closed set of alternatives* (an algebraic sum type). The compiler then knows every case, so a `switch` over a sealed type can be **exhaustive without `default`**, and adding a new permitted subtype breaks compilation in every switch that does not handle it. That is safer than a `default` branch that silently swallows the new case.

7.8 Pattern matching: instanceof, switch, record patterns

Pattern matching for instanceof (16)

```
// Before
if (obj instanceof Break) { Break b = (Break) obj; total = total.add(b.difference()); }

// After: test, cast and bind in one step
if (obj instanceof Break b && b.difference().signum() != 0) { total = total.add(b.difference()); }

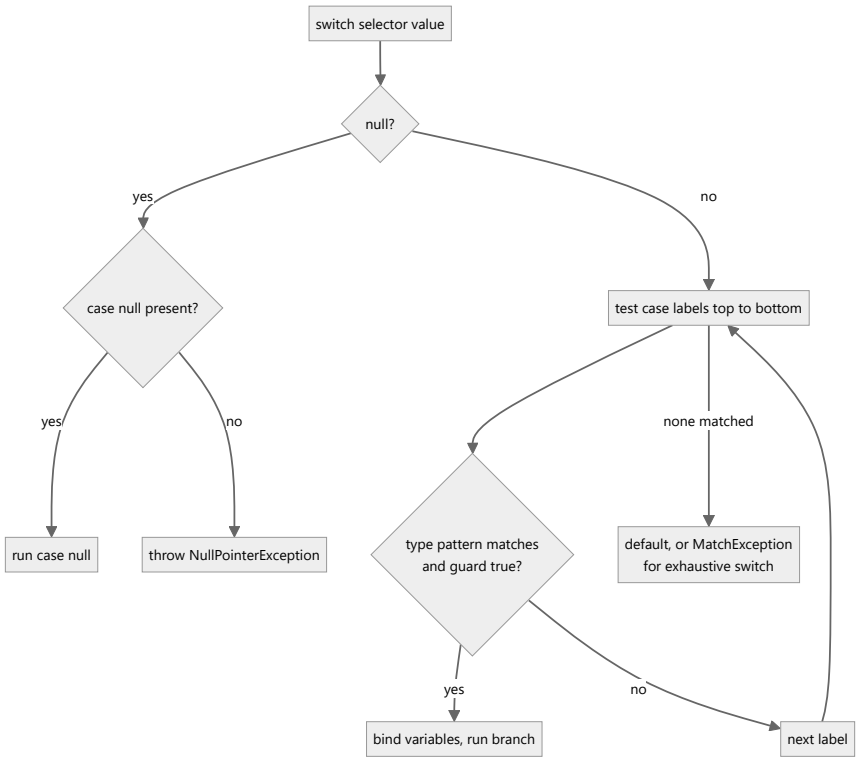
// Flow scoping: the binding is in scope where the compiler knows the match succeeded
if (!(obj instanceof Break b)) return;
log.info("break {}", b.id());          // b in scope here

// equals() becomes one line
@Override public boolean equals(Object o) {
    return o instanceof Money m && amount.compareTo(m.amount) == 0 && currency.equals(m.currency);
}
```

`if (obj instanceof Break b || b.id() != null)` does not compile — with `||` the binding is not definitely matched on the right side.

Pattern matching for switch (21)

```
static String describe(Object o) {
    return switch (o) {
        case null -> "null";                                  // explicit null handling (otherwise NPE)
        case Integer i when i > 1_000 -> "large int " + i;    // guard
        case Integer i -> "int " + i;
        case String s when s.isBlank() -> "blank string";
        case String s -> "string of " + s.length();
        case int[] arr -> "int array of " + arr.length;
        default -> "other " + o.getClass().getSimpleName();
    };
}
```



Rules interviewers test:

1. **Dominance:** a label must not be unreachable because an earlier one already matches everything it matches. `case CharSequence cs -> …; case String s -> …;` is a compile error. Guarded patterns go before the unguarded pattern of the same type.
2. **Exhaustiveness:** pattern switches (expression or statement) must be exhaustive — via `default`, a total pattern (`case Object o`), or covering all permitted subtypes of a sealed type.
3. **null:** old switches throw NPE on null; a pattern switch does too unless it has `case null` (which can combine: `case null, default -> …`).
4. **Guards** use `when` (the `&&` guard syntax of the previews was dropped).
5. Selector can be any reference type; `case` constants of enums can be qualified (`case Status.OPEN`) and mixed with patterns in Java 21.

Record patterns and nested deconstruction (21)

A record pattern matches a record *and* extracts its components in one step, and it nests:

```
sealed interface Shape permits Circle, Rect {}
record Point(double x, double y) {}
record Circle(Point centre, double r) implements Shape {}
record Rect(Point topLeft, Point bottomRight) implements Shape {}

static double area(Shape s) {
    return switch (s) {                                          // exhaustive: no default needed
        case Circle(Point c, double r) -> Math.PI * r * r;
        case Rect(Point(var x1, var y1), Point(var x2, var y2)) -> Math.abs(x2 - x1) * Math.abs(y2 - y1);
    };
}

if (obj instanceof Circle(Point(var x, var y), var r) && r > 0) { ... }   // works with instanceof too
```

Record patterns call the accessor methods; `var` infers each component's type; generic records infer type arguments (`case Box(String s)` against `Box<String>`). In Java 21, `_` as an unnamed pattern was preview (`case Circle(_, var r)`), final in Java 22.

Data-oriented programming

The three features together give a style Brian Goetz calls **data-oriented programming**: model data as immutable records, model alternatives as sealed interfaces, and write operations as exhaustive pattern switches *outside* the data types.

```
public sealed interface MatchOutcome permits Matched, Break, Ignored {}
public record Matched(UUID leftId, UUID rightId, String rule) implements MatchOutcome {}
public record Break(UUID id, BreakReason reason, BigDecimal difference) implements MatchOutcome {}
public record Ignored(UUID id, String why) implements MatchOutcome {}

String auditLine(MatchOutcome o) {
    return switch (o) {
        case Matched(var l, var r, var rule)            -> "matched %s with %s by %s".formatted(l, r, rule);
        case Break(var id, var reason, var diff) when diff.signum() == 0 -> "break %s: %s".formatted(id,
reason);
        case Break(var id, var reason, var diff)        -> "break %s: %s, diff %s".formatted(id, reason, diff);
        case Ignored(var id, var why)                   -> "ignored %s: %s".formatted(id, why);
    };
}
```

Compare with the classic OOP approach (an abstract `describe()` method on each type). OOP makes adding new *types* cheap; data-oriented makes adding new *operations* cheap and keeps the data types free of presentation or persistence logic. With a closed, stable set of outcomes and many operations (audit, export, UI, metrics), the sealed-switch style wins — the expression problem from chapter 1.

> **RefinePoint:** the expression language `ryfn-expression` is a natural fit: the AST `Node` types (literals, field references, binary operators, function calls) are a closed set, and two very different operations walk the same tree — the evaluator and `SqlCompiler`, which turns a rule into DuckDB SQL. An exhaustive switch over a sealed node hierarchy means adding a node type is a compile error in both walkers until both handle it, instead of a runtime "unsupported node" in production.

7.9 Helpful NullPointerExceptions (14)

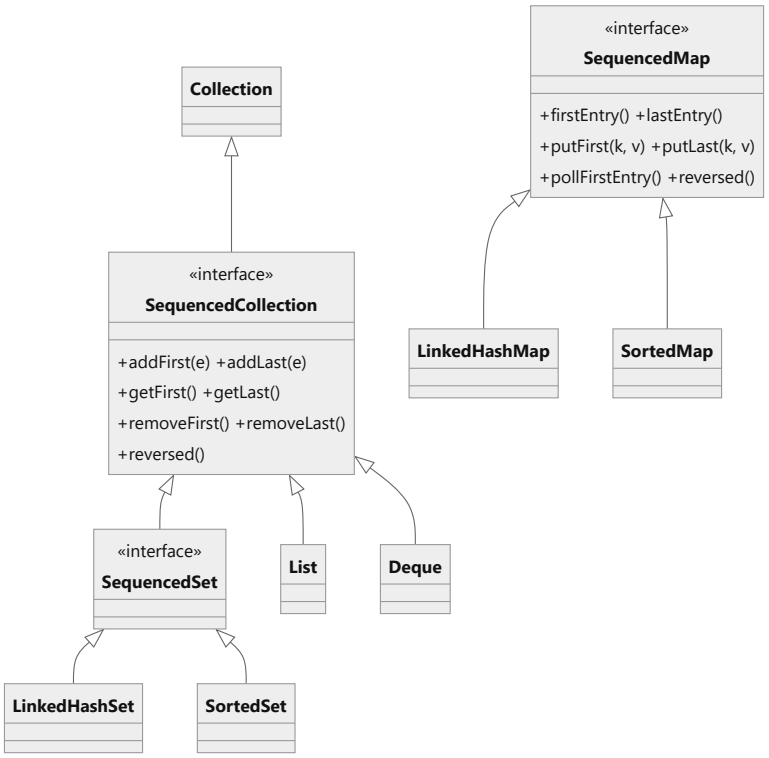
```
Before:  java.lang.NullPointerException
             at com.x.InvoiceService.total(InvoiceService.java:42)

After:   java.lang.NullPointerException: Cannot invoke "java.math.BigDecimal.add(java.math.BigDecimal)"
             because the return value of "com.x.Line.amount()" is null
```

JEP 358: the JVM analyses the bytecode at the failing instruction and describes *which* expression was null — the array, the field, the method's return value. On by default since Java 15 (`-XX:+ShowCodeDetailsInExceptionMessages`). Local variable names appear only when the class was compiled with `-g` (otherwise `"<local4>"`); Maven and Gradle compile with debug info by default. The message is computed lazily, only when `getMessage()` is called, so there is no cost for NPEs that are never inspected.

7.10 Sequenced collections (21)

Before 21, "get the first/last element" had a different API for every type: `list.get(0)`, `deque.getFirst()`, `sortedSet.first()`, `linkedHashSet.iterator().next()` and no direct way at all for the last element of a `LinkedHashSet`. JEP 431 adds three interfaces into the existing hierarchy.




```
List<Run> runs = repo.recent();
Run newest = runs.getFirst();                // instead of runs.get(0)
Run oldest = runs.getLast();                 // instead of runs.get(runs.size() - 1)
for (Run r : runs.reversed()) { ... }        // a reversed VIEW, no copy

LinkedHashMap<String, Instant> seen = new LinkedHashMap<>();
Map.Entry<String, Instant> eldest = seen.firstEntry();
seen.putLast("RUN-9", Instant.now());        // moves to the end if present
```

`reversed()` returns a live view; writes through it affect the original. On unmodifiable lists `addFirst` throws `UnsupportedOperationException`; on a `SortedSet`, `addFirst/addLast` throw because position is determined by the comparator. `getFirst()` on an empty list throws `NoSuchElementException` (not `IndexOutOfBoundsException`).

7.11 Virtual threads, generational ZGC and other runtime changes

Virtual threads (JEP 444, final in 21) are lightweight threads scheduled by the JVM on a small pool of carrier platform threads. Blocking a virtual thread on I/O unmounts it from its carrier instead of blocking an OS thread, so thread-per-request code scales to hundreds of thousands of concurrent requests. In Spring Boot 3.2+, `spring.threads.virtual.enabled=true` runs Tomcat request handling, `@Async` and scheduling on virtual threads. In Java 21, a virtual thread blocking inside `synchronized` **pins** its carrier (fixed in Java 24). Chapter 10 covers the internals.

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    List<Future<Rate>> rates = currencies.stream().map(c -> executor.submit(() -> fxClient.rate(c))).toList();
}
Thread.ofVirtual().name("ingest-", 0).start(task);
```

Generational ZGC (JEP 439): ZGC (sub-millisecond pauses regardless of heap size) gains young and old generations, collecting short-lived objects more often and cheaply. In Java 21 enable with `-XX:+UseZGC -XX:+ZGenerational`; it became the default ZGC mode in 23 and the only mode in 24. G1 remains the default collector.

Strong encapsulation of JDK internals (JEP 403, Java 17): internal packages (`sun.*`, `jdk.internal.*`, private members of `java.*` classes) are closed to reflection by default and the `--illegal-access` escape hatch is gone. Java 9–15 merely *warned* ("An illegal reflective access operation has occurred"); 16 denied by default with a flag to relax; 17 removed the flag. Old libraries (Lombok < 1.18.22, old Mockito/ByteBuddy, old Groovy, some serialization libraries) fail with

`InaccessibleObjectException`. Fixes: upgrade the library (the real fix), or `--add-opens java.base/java.lang=ALL-UNNAMED` as a temporary bridge. `sun.misc.Unsafe` remained accessible through the `jdk.unsupported` module.

Do not confuse this with the **Key Encapsulation Mechanism API** (JEP 452, Java 21): a *cryptography* API (`javax.crypto.KEM`) for key-encapsulation schemes such as RSA-KEM and ECIES-style hybrid encryption, and a building block for post-quantum algorithms. Same word, unrelated feature — interviewers occasionally mix them up, and a calm clarification scores points.

Other runtime items worth naming: **UTF-8 by default** (Java 18: `new FileReader(f)` no longer depends on the OS locale, a source of mojibake on Windows servers), **container awareness** (heap sized from cgroup memory limits; use `-XX:MaxRAMPercentage=75` in Docker), **CDS/AppCDS** for faster startup, **JFR** as a free low-overhead profiler (`jcmd <pid> JFR.start`).

7.12 Removed and deprecated

Item	Status	Replacement
Java EE modules (JAXB, JAX-WS, JAF, CORBA, JTA)	Removed in 11	Jakarta artifacts as normal dependencies
Nashorn JavaScript engine	Deprecated 11, removed 15	GraalJS, or standalone Nashorn library
Applets, `appletviewer`	Deprecated 9, `appletviewer` removed 11, Applet API deprecated for removal 17	None — browsers dropped plugins
SecurityManager	Deprecated for removal 17, permanently disabled in 24	Containers, OS-level sandboxing, module encapsulation
`Object.finalize()`	Deprecated 9, deprecated for removal 18	try-with-resources, `java.lang.ref.Cleaner`
CMS garbage collector	Removed 14	G1, ZGC
Biased locking	Disabled/deprecated 15, removed 18	Nothing needed: modern code uses `java.util.concurrent` locks
RMI activation, Pack200, experimental AOT/Graal JIT	Removed 14–17	—
`Thread.stop/suspend/resume`	Throw `UnsupportedOperationException` (20+)	Interruption
`--illegal-access`	Removed 17	Upgrade libraries, `--add-opens`
String templates (preview in 21–22)	Withdrawn in 23	`formatted()`, `MessageFormat`

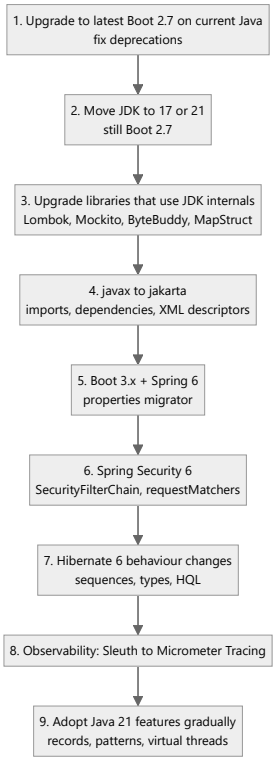
7.13 Preview features in Java 21

Use them only with `--enable-preview` on both `javac` and `java`, never in production code you must support across JDK upgrades. Know what they are:

- **String templates** (JEP 430): `STR."Hello \{name}"` — template processors with embedded expressions. Preview in 21 and 22, then **withdrawn** for redesign. Good interview point: "previews can disappear".
- **Unnamed patterns and variables** (JEP 443): `_` for unused variables and pattern components: `catch (NumberFormatException _)`, `case Circle(_, var r)`, `for (var _ : list)`. Final in 22.
- **Unnamed classes and instance main methods** (JEP 445): `void main() { System.out.println("hi"); }` in a file with no class declaration — aimed at teaching. Finalised as compact source files in 25.
- **Structured concurrency** (JEP 453): `StructuredTaskScope` treats a group of subtasks as one unit — if one fails, siblings are cancelled; the scope cannot outlive its block. Still evolving after 21.
- **Scoped values** (JEP 446): immutable, bounded-lifetime alternative to `ThreadLocal`, cheap with millions of virtual threads: `ScopedValue.where(ORG, org).run(() -> handle())`. Final in 25.

7.14 Migrating from Java 8/11 to 17/21 and Spring Boot 3

Spring Boot 3 requires **Java 17+** and **Jakarta EE 9+** (the `jakarta.*` namespace). These are the steps of a real migration, in the order that keeps the build green:



What breaks and how to fix it:

1. `javax` → `jakarta.` `javax.persistence`, `javax.servlet`, `javax.validation`, `javax.annotation` (`@PostConstruct`), `javax.transaction` all become `jakarta.*`. `javax.sql`, `javax.crypto`, `javax.net` stay — they are part of the JDK, not Java EE. Every library compiled against `javax.servlet` (old filters, old Swagger/springfox) must be replaced by a Jakarta version (springdoc-openapi 2.x). OpenRewrite's `UpgradeSpringBoot_3_x` recipe automates most of it.
2. **Spring Security 6:** `WebSecurityConfigurerAdapter` removed — declare a `SecurityFilterChain` bean; `antMatchers` → `requestMatchers`; `authorizeRequests` → `authorizeHttpRequests`; lambda DSL required for most config.
3. **Spring MVC 6:** trailing-slash matching is off (`/api/runs/` no longer matches `/api/runs`); `PathPatternParser` is the default matcher.
4. **Hibernate 6:** different default sequence naming (one shared `hibernate_sequence` becomes a per-entity `<entity>_SEQ`), stricter HQL, new type system (`@Type` changes), `ID` generation changes — test persistence thoroughly.
5. **Properties:** add `spring-boot-properties-migrator` temporarily; it logs renamed keys at startup (`spring.redis.*` → `spring.data.redis.*`).
6. **Tracing:** Spring Cloud Sleuth does not support Boot 3; move to Micrometer Tracing with the OpenTelemetry or Brave bridge.
7. **JDK-level:** removed JAXB/JAX-WS (add dependencies), strong encapsulation errors (upgrade libraries), changed default charset in 18 (explicit charsets everywhere), `SecurityManager` uses removed.
8. **Build:** Maven compiler `<release>21</release>`, Surefire/Failsafe recent versions, Docker base image to a 21 JRE, `-XX:MaxRAMPercentage` instead of fixed `-Xmx` in containers.

Interview tip: tell the migration as a story with numbers and order. A template, to fill with your own numbers from a migration you actually did: "We had N services on Boot 2.7 and Java 11. We first moved every service to Java 17 on 2.7, which surfaced library upgrades (Lombok, Mockito) without mixing in Spring changes. Then one pilot service went to Boot 3 using OpenRewrite for javax→jakarta; the painful part was Spring Security 6 configuration and Hibernate 6 sequence naming. With a checklist from the pilot, the rest took a couple of sprints. Afterwards we adopted records for DTOs and enabled virtual threads on I/O-heavy services." The *order* (JDK first, then framework) is what shows seniority: it isolates failures.

RefinePoint: RefinePoint started on Java 21 and Spring Boot 3.4.3, so all 13 services and `ryfn-platform` were written in the `jakarta.*` world from day one. The shared library's auto-configurations (`PlatformSecurityAutoConfiguration` , `PlatformWebAutoConfiguration`) use Boot 3's `AutoConfiguration.imports` file rather than the old `spring.factories` key — another Boot 2 → 3 change worth mentioning when asked about auto-configuration.

7.15 Interview questions and answers

Q1. Which Java versions are LTS, and why does it matter?

8, 11, 17, 21 and 25. LTS releases get years of vendor patches, so enterprises and frameworks target them; the releases between are supported only until the next one ships, six months later. Spring Boot 3 requires 17 as a minimum, which pushed most of the industry off 8 and 11. Non-LTS releases are fine if you can upgrade every six months, and most teams cannot commit to that.

Q2. What is the difference between incubator, preview and final?

Incubator is an API in a `jdk.incubator.*` module that may change completely. Preview is a fully specified language or JVM feature that needs `--enable-preview` at compile and run time and may still change or be withdrawn. Final is standard and permanent. Class files compiled with preview features are tied to that exact JDK version. String templates are the proof that previews can vanish: preview in 21 and 22, withdrawn in 23.

Q3. What problem does JPMS solve?

It replaces the flat classpath with explicit modules that declare what they need and what they expose. That gives **reliable configuration** (a missing dependency or split package fails at startup, not with `NoClassDefFoundError` at runtime) and **strong encapsulation** (`public` no longer means visible to everyone). It also let the JDK itself be split up, which is why `jlink` can build small runtimes and why internals became inaccessible.

Q4. `requires` vs `requires transitive` vs `requires static`?

`requires` reads a module at compile and run time. `requires transitive` also makes the dependency readable by anyone who requires you, used when your API exposes its types. `requires static` is compile-time only, optional at runtime, the module equivalent of Maven's `provided` scope (annotation libraries, for example).

Q5. `exports` vs `opens`?

`exports` gives compile-time and runtime access to the package's public types. `opens` gives no compile-time access, but allows deep reflection at runtime, including private members. Frameworks need `opens` : Jackson on DTO fields, Hibernate on entity fields. `exports p to m` and `opens p to m` limit it to named modules. `open module` opens everything.

Q6. What are the unnamed module and automatic modules?

Everything on the classpath lands in the **unnamed module**, which reads all modules and exports all its packages, but named modules cannot require it. A plain JAR on the module path becomes an **automatic module**: it gets a name from `Automatic-Module-Name` or the file name, reads everything and exports everything. It is the migration bridge between the classpath and full modularity.

Q7. What is a split package and why is it a problem?

The same package present in two modules. JPMS forbids it, because a package must belong to exactly one module for readability to work, and startup fails. The classpath allowed it silently, so old libraries sometimes break when moved to the module path (the JSR-305 `javax.annotation` split is the classic). It stays invisible while everything runs in the unnamed module.

Q8. Do you use modules in your Spring Boot services?

No, and that is the normal answer. Spring relies on classpath scanning, reflection and runtime proxies, which would need `opens` almost everywhere, and many libraries do not ship proper descriptors. The benefit is mostly for libraries and small runtimes. RefinePoint has no `module-info.java`; the 13 services run on the classpath. For plug-ins we use `ServiceLoader` with `META-INF/services` (`Plugins.load(DataConnector.class)`), which predates modules and works without them. We still benefit indirectly, since the JDK itself is modular.

Q9. What are the properties of `List.of`, `Set.of` and `Map.of`?

They are unmodifiable (mutators throw `UnsupportedOperationException`), reject nulls with NPE (even `contains(null)` throws on `List.of`), and `Set.of` / `Map.of` throw `IllegalArgumentException` on duplicates. Set and map iteration order is deliberately randomised per JVM run with a salt, so tests that depend on it fail early. `Arrays.asList` differs: it is a fixed-size view over the array, `set` works, nulls are allowed.

Q10. `List.of` vs `List.copyOf` vs `Collections.unmodifiableList`?

`List.of` builds a new unmodifiable list from its arguments. `List.copyOf` makes an independent unmodifiable copy, and returns the same instance if the input is already an immutable list. `Collections.unmodifiableList` is a read-only **view**: changes to the underlying list still show through. For defensive copies in record constructors use `List.copyOf`.

Q11. Why were private methods added to interfaces?

So that default methods can share code without exposing a helper as part of the public contract. Before Java 9 a helper had to be another default method (visible and overridable) or live in a separate utility class. Private static interface methods are allowed too. Interface methods can never be `protected`.

Q12. What is JShell used for?

It is the JDK's REPL, for quick experiments without a class or a `main`. Useful for checking API behaviour, regexes and date arithmetic. It keeps state between snippets and supports `/imports`, `/vars` and `/save`. It is a learning and exploration tool, not a production one.

Q13. Is `var` dynamic typing?

No. The type is inferred at compile time and is fixed; the bytecode is identical to an explicit declaration. `var x = 10; x = "s";` is a compile error. It reduces noise on the left side without changing the type system, and it is limited to local variables so that APIs always have explicit types.

Q14. Where can't you use `var`?

Fields, method parameters, return types, catch parameters, variables without an initializer, `null` initializers, array initializers (`var a = {1,2}`), and lambdas or method references without a target type. One lambda cannot mix `var` and explicit parameter types. `var` is allowed in `for`, try-with-resources and, since 11, lambda parameters.

Q15. Predict the type.

```
var a = new ArrayList<>();          // ArrayList<Object>
var b = List.of(1, 2.0);            // List<Number & Comparable<...>>
var c = new int[]{1, 2};            // int[]
var d = 'x' + 1;                    // int
var e = new Object() { int n = 5; };
System.out.println(e.n);            // 5, the anonymous type is usable through var
```

The diamond with `var` is the classic trap: with no target type, inference falls back to `Object`.

Q16. Can a variable be named `var`?

Yes. `var var = 5;` compiles. `var` is a **reserved type name**, not a keyword, chosen so that existing code using it as an identifier keeps compiling. Only a class or type parameter named `var` is forbidden. `record`, `sealed`, `permits`, `yield` and `when` are contextual in a similar way.

Q17. When would you avoid `var`?

When the type is not obvious from the right side and it matters to the reader: `var result = service.process(x);`. Also with numeric literals where the width matters, and when you want a variable typed as an interface (`var list = new ArrayList<Run>()` is an `ArrayList`, not a `List`). Good uses: long generic types, `new` expressions, loops over `entrySet()`.

Q18. How does the Java 11 `HttpClient` compare with `RestTemplate` or `RestClient`?

`HttpClient` is the low-level JDK client: HTTP/2, async with `CompletableFuture`, WebSocket, no dependency. `RestClient` (Spring 6.1) and the older `RestTemplate` sit a level higher, adding message converters, error handling, interceptors and observability, and can use `HttpClient` underneath via `JdkClientHttpRequestFactory`. In a Spring service I use `RestClient`. Note that `HttpClient` does not throw on 4xx/5xx, its connect timeout and request timeout are separate settings, and it should be reused, because every instance has its own connection pool and selector thread.

Q19. `trim()` vs `strip()`? `isEmpty()` vs `isBlank()`?

`trim()` removes characters with code ≤ U+0020; `strip()` removes Unicode whitespace as defined by `Character.isWhitespace`, including an em space. Non-breaking space U+00A0 is removed by neither, because `isWhitespace` excludes it, which is a real headache with bank files pasted from Excel. `isEmpty()` means length 0; `isBlank()` means empty or all whitespace.

Q20. What does `"a\nb\r\nc".lines()` return?

A lazy `Stream<String>` of `a`, `b`, `c`. It handles all three line terminators and it is more efficient than `split("\\R")` because it does not build an array first. No trailing empty string is produced for a final terminator.

Q21. Switch statement vs switch expression?

A statement performs actions and may fall through; an expression produces a value, must be exhaustive and cannot fall through when using arrows. `yield` returns a value from a block. Arrow syntax can be used in statements too, and colon syntax in expressions (with `yield`), though mixing the two forms in one switch is a compile error.

Q22. Predict the output.

```
int day = 2;
String r = switch (day) {
    case 1, 7 -> "weekend";
    case 2 -> { String s = "mon"; yield s.toUpperCase(); }
    default -> "weekday";
};
System.out.println(r);
```

`MON`. There is no fall-through, and `return` is not allowed inside a switch expression block. It would have to be `yield`.

Q23. What happens if I add a constant to an enum used in a switch expression with no `default`?

At the next compile, every such switch fails, which is the point: the compiler lists every place to update. If an old class file runs against the new enum without recompiling, the compiler-generated default throws `MatchException` in Java 21 (`IncompatibleClassChangeError` on earlier releases). A `default` branch would silently swallow the new constant, so for closed sets I leave it out.

Q24. How is indentation handled in a text block?

The compiler strips incidental whitespace: the smallest indentation across all non-blank lines plus the line of the closing delimiter. Move the closing `"""` left and content gains indentation. Trailing whitespace is stripped, line endings become `\n`. Use `\` at line end to join lines and `\s` to preserve a trailing space.

Q25. Predict the output.

```
String a = """
    hello
      world
    """;
String b = """
    hello""";
```

`a` is `"hello\n world\n"`: the minimum indent is 4, the second line keeps its extra 2, and there is a trailing newline because the closing delimiter is on its own line. `b` is `"hello"` with no newline. `"""hello"""` on one line does not compile, since the opening delimiter must be followed by a line break.

Q26. What is a record? What does the compiler generate?

A final, shallowly immutable data carrier declared by its components. The compiler generates private final fields, a canonical constructor, accessors named after the components, and `equals`, `hashCode` and `toString` over all components, through `invokedynamic` and `ObjectMethods`. It extends `java.lang.Record`.

Q27. Canonical vs compact constructor?

A canonical constructor has the full parameter list and must assign every field. A compact constructor omits the parameter list and the assignments: the body validates or reassigns the parameters, and the fields are assigned automatically at the end. Assigning `this.x` inside a compact constructor is a compile error. Any other constructor must delegate with `this(...)`. The canonical constructor cannot be less accessible than the record.

Q28. What can't a record do?

It cannot extend a class (it can implement interfaces), be extended, declare instance fields beyond its components, have non-final component fields, or have native methods. It can have static fields and methods, instance methods, nested types,

annotations and generics.

Q29. Are records immutable?

Only shallowly. The fields are final, but a mutable component can still be changed:

```
record Team(List<String> names) {}
var list = new ArrayList<>(List.of("a"));
var t = new Team(list);
list.add("b");                // t.names() now has 2 elements
```

Fix with `names = List.copyOf(names)` in a compact constructor. Arrays are worse: `equals` compares the array reference, so two records with equal array contents are not equal.

Q30. Can a record be a JPA entity? How do you use records with JPA?

No. JPA needs a no-arg constructor, non-final fields for dirty checking and non-final classes for lazy-loading proxies. Records fit well as read models: JPQL constructor expressions, Spring Data projections and `JdbcTemplate` row mappers. Hibernate 6.2+ also maps a record as an `@Embeddable` value object such as `Money`. Reading into a record projection also avoids the persistence-context cost of loading entities for read-only screens.

Q31. How do records work with Jackson and Bean Validation?

Jackson 2.12+ deserialises through the canonical constructor and serialises through the accessors, and needs the component names, which records always retain. Validation annotations on components (`@NotBlank String name`) apply with `@Valid` on the controller parameter. `@JsonProperty` and `@JsonIgnore` work on components. No Lombok needed, which removes a build-time dependency for DTOs.

Q32. Records vs Lombok `@Data` / `@Value` ?

Records are a language feature: no annotation processor, enforced immutability, canonical-constructor serialisation and pattern-matching support. `@Data` gives mutable JavaBeans with getters and setters, still needed for JPA entities and some frameworks. `@Value` is the closest, but it cannot be deconstructed in a pattern and it depends on Lombok. Records use `name()` rather than `getName()`, so old code expecting JavaBean getters (BeanUtils, some JSP or template engines) may need adapting.

Q33. How do you get a "with" style update on a record?

Java 21 has no built-in support, so write it by hand: `public Money withAmount(BigDecimal a) { return new Money(a, currency); }`. Each call goes through the canonical constructor, so validation always re-runs. Derived record creation (`with` expressions) has been proposed for a future release but is not in 21.

Q34. What is a sealed class and why would you use one?

A class or interface that restricts its direct subtypes to a declared list. It models a closed set of alternatives, keeps the hierarchy under the author's control, and lets the compiler check exhaustive switches with no `default`, so adding a subtype produces compile errors at every switch that must handle it.

Q35. What must a permitted subclass declare?

Exactly one of `final`, `sealed` or `non-sealed`. Records and enums are implicitly final. It must also directly extend the sealed type and be in the same module, or the same package if unnamed. `non-sealed` reopens that branch to any subclass, so exhaustiveness still holds at the permitted level: the switch covers the `non-sealed` type and all its unknown subclasses with one label.

Q36. Sealed classes vs enums vs final classes?

An enum is a fixed set of **instances**, each a singleton with the same shape. A sealed type is a fixed set of **types**, each with its own fields and any number of instances. `final` closes a class to all subclasses, while sealed allows a specific list. Use an enum for statuses, and sealed plus records for outcomes that carry different data.

Q37. What is flow scoping in `instanceof` patterns?

A binding variable is in scope wherever the compiler can prove the match succeeded. `if (o instanceof String s && s.isEmpty())` is valid, while `|| s.isEmpty()` is not. After `if (!(o instanceof String s)) return;`, `s` is in scope for the rest of the method. A binding can shadow a field, which is a subtle source of confusion.

Q38. Which of these compile?

```
if (o instanceof String s && s.length() > 1) {}   // A
if (o instanceof String s || s.length() > 1) {}   // B
String t = "x";
if (t instanceof String u) {}                     // C
Integer i = 1;
if (i instanceof String v) {}                     // D
```

A compiles. B does not, since `s` is not definitely matched on the right of `||`. C compiles from Java 21 (a total pattern in `instanceof` was an error before). D does not, since `Integer` can never be a `String` and the compiler rejects inconvertible types.

Q39. What are the rules of dominance in a pattern switch?

A label is a compile error if an earlier label matches everything it would match. Subtypes must come before supertypes, and a guarded pattern must come before the unguarded pattern of the same type. Constant labels must precede the pattern of their type (`case "x"` before `case String s`). `default` should come last; a pattern label after `default` is a compile error.

Q40. Predict the output or error.

```
static String f(Object o) {
    return switch (o) {
        case CharSequence c -> "cs";
        case String s -> "s";
        default -> "other";
    };
}
```

Compile error: `case String s` is dominated by `case CharSequence c`. Swap them and `f("a")` returns `s`.

Q41. How does a pattern switch handle null?

It throws `NullPointerException` unless a `case null` label is present, keeping old behaviour for existing code. `case null -> …` handles it explicitly and `case null, default -> …` merges it with the default. `default` alone does not match null. Note that `case null` is the only place a null reaches a switch: a total pattern such as `case Object o` does not match it without that label.

Q42. How does a pattern switch compile?

To an `invokedynamic` call with `SwitchBootstraps.typeSwitch` as bootstrap, which returns the index of the first matching label given a start index. The switch then jumps on that index, and when a guard fails it restarts from the next label.

Compared with an `if` / `instanceof` chain, the checks stay in one place and the JVM may optimise them, but performance is rarely the reason to use it. Exhaustiveness and readability are.

Q43. What are record patterns and how do they nest?

A record pattern tests the type and deconstructs its components into bindings, recursively: `case Rect(Point(var x1, var y1), Point p2)`. It calls the accessors, so an accessor that throws produces `MatchException`. A nested pattern against a null component does not match: `Rect(Point(var x, var y), var p)` fails when the first point is null, while `var p` itself accepts null.

Q44. What makes a switch exhaustive over a sealed type with record patterns?

The compiler computes coverage across the nested components. Given `sealed interface S permits A, B`, `record P(S l, S r)`, covering `P(A, A)`, `P(A, B)`, `P(B, A)` and `P(B, B)` is exhaustive with no `default`; drop one and it is an error. Guarded labels do not count toward exhaustiveness, because the compiler cannot reason about a boolean expression.

Q45. What is data-oriented programming?

Modelling the domain as immutable data (records), alternatives as sealed types, and behaviour as functions using exhaustive pattern switches, kept separate from the data. It suits data pipelines, parsers, message handling and API boundaries. Use OOP polymorphism when new *types* are added often and behaviour belongs with state. Use data-oriented style when the set of types is stable and new *operations* are frequent.

Q46. Give a real example of sealed types and pattern matching from your work.

RefinePoint's rule language: `Node` is a `sealed interface` whose variants are nested records (`Field`, `Column`, `Text`, `Num`, `Bool`, `Call`, `Arith`, `Negate` and others), produced by the parser. The evaluator, the SQL compiler that targets DuckDB and the `RuleWriter` that renders a rule back to English are separate operations over the same tree. New node types are rare and new operations do come up, which is the textbook case for this style.

Q47. What are helpful NullPointerExceptions?

From Java 14, on by default from 15, the NPE message says which expression was null: `Cannot invoke "String.length()" because the return value of "User.name()" is null`. The JVM derives it by analysing bytecode when the message is requested, so it has no cost until read. Chained calls such as `a.b().c().d()` no longer need to be split across lines just to diagnose production NPEs. Local variable names appear only with `-g`.

Q48. What are sequenced collections?

Java 21 interfaces `SequencedCollection`, `SequencedSet` and `SequencedMap`, added above `List`, `Deque`, `LinkedHashSet`, `SortedSet`, `LinkedHashMap` and `SortedMap`, providing `getFirst`, `getLast`, `addFirst`, `addLast`, `removeFirst`, `removeLast` and `reversed`. They give a uniform API for "has a defined encounter order". `HashSet` and `HashMap` do not implement them.

Q49. Is `list.reversed()` a copy?

No, it is a live view. Changes to either side are visible in the other, and it costs nothing to create. To get an independent copy use `new ArrayList<>(list.reversed())`. On an unmodifiable list the view is unmodifiable too. A class implementing both `List` and `Deque` (`LinkedList`) resolves `reversed()` to a type that is both.

Q50. What changed for virtual threads in Java 21 and what is the main gotcha?

They became final (JEP 444): cheap JVM-scheduled threads for blocking code, created with `Executors.newVirtualThreadPerTaskExecutor()` or `Thread.ofVirtual()`, enabled in Spring Boot 3.2+ with `spring.threads.virtual.enabled=true`. Main gotcha in 21: blocking inside `synchronized` or a native frame **pins** the carrier thread (the `synchronized` case was fixed in 24). Do not pool them, and limit downstream concurrency with a semaphore, since the database pool is still finite. Details in chapter 10.

Q51. What is generational ZGC and when would you turn it on?

ZGC split into young and old generations so most garbage is collected cheaply, giving lower allocation-stall risk and better throughput with the same very short pauses. On 21: `-XX:+UseZGC -XX:+ZGenerational`. Consider it for large heaps or strict latency targets. G1 remains the sensible default for typical services. Chapter 12 goes deeper.

Q52. What did JDK 17 change about access to internals?

JEP 403 made strong encapsulation permanent: `--illegal-access` is gone, and reflective access to non-public members of JDK classes throws `InaccessibleObjectException` unless you pass `--add-opens`. The journey: warnings from 9, deny by default in 16, no global switch in 17. Upgrade the library rather than collecting `--add-opens` flags. Each flag is a documented debt.

Q53. What is the Key Encapsulation Mechanism API?

JEP 452 in Java 21: `javax.crypto.KEM`, an API for schemes where the sender derives a shared secret and an encapsulated form of it using the receiver's public key. It is the basis for hybrid public-key encryption and post-quantum algorithms. It has nothing to do with the *strong encapsulation* of JDK internals, and it is worth clarifying which one an interviewer means.

Q54. What was removed or deprecated, and what replaced it?

Java EE modules and CORBA removed in 11 (add Jakarta dependencies). Nashorn removed in 15 (GraalJS). CMS removed in 14 (G1, ZGC). SecurityManager deprecated for removal in 17 (OS and container isolation). `finalize` deprecated for removal in 18 (`Cleaner`, try-with-resources). Applet API deprecated for removal in 17 (nothing). Biased locking removed in 18. `Thread.stop` throws from 20 (interruption).

Q55. Why was biased locking removed?

It optimised uncontended `synchronized` for old APIs like `Vector` and `Hashtable`, which modern code rarely uses, while revocation needed costly safepoints and the code added heavy complexity to the JVM. JEP 374 disabled and deprecated it in 15, and the code was removed in 18. Thin locks are fast enough for today's workloads. Chapter 8 covers the mark word.

Q56. Which Java 21 features were still preview?

String templates, unnamed patterns and variables, unnamed classes with instance `main`, structured concurrency and scoped values. Later, unnamed variables were final in 22, string templates were withdrawn in 23, and scoped values and compact source files were final in 25. Do not depend on any preview API in production code on 21.

Q57. What is `StructuredTaskScope`?

A preview API on 21 that treats concurrent subtasks as one unit within a lexical block: fork subtasks, join, and handle results together. `ShutdownOnFailure` cancels siblings when one fails, `ShutdownOnSuccess` when one succeeds. No subtask outlives the scope, which removes thread leaks and gives clear thread-dump hierarchy. The API was reshaped in later previews, so treat it as direction rather than a stable contract.

Q58. Scoped values vs `ThreadLocal`?

A `ThreadLocal` is mutable, lives until removed, and each thread (and each inherited copy) carries its own map, which is costly with millions of virtual threads and leaks in pools. A `ScopedValue` is immutable within a bounded `where(...).run(...)` block, cleaned up automatically, and shared cheaply with child tasks under structured concurrency. Preview on 21, final in 25.

Q59. What is new in the JDK's default charset behaviour?

From Java 18 (JEP 400) the default charset is UTF-8 on all platforms, regardless of OS locale. Before that, `new FileReader(f)` and `getBytes()` used the platform charset, so a Windows server with cp1252 corrupted non-ASCII data that worked on Linux. Migration check: code that relied on the old default (or files actually written in cp1252) now behaves differently. Use explicit charsets, and `-Dfile.encoding=COMPAT` as a temporary bridge.

Q60. How do you approach a migration from Java 8 to 21?

Run `jdeps --jdk-internals` for internal API use. Upgrade build tools and libraries first, then the JDK to 11 or directly to 21 with `--release` left at the old value, fix removed-module dependencies (JAXB) and `InaccessibleObjectException`s, run everything including integration tests and GC-log comparison, deploy a canary, and only then raise `--release` and adopt new syntax. Review GC flags, as many old ones are removed and make the JVM refuse to start.

Q61. What exactly changes in `javax` → `jakarta`?

Only the Java EE namespaces moved, because Oracle's transfer to Eclipse did not include the `javax` trademark: `persistence`, `servlet`, `validation`, `annotation`, `transaction`, `mail`, `jms`, `websocket` and so on. JDK packages like `javax.sql`, `javax.crypto`, `javax.net` and `javax.naming` stay. No behaviour changes, but binary compatibility breaks, so every library compiled against `javax.servlet` needs a Jakarta build. OpenRewrite handles the source changes; the hard part is third-party dependencies.

Q62. What else breaks going from Spring Boot 2 to 3?

Spring Security 6 (`SecurityFilterChain`, `requestMatchers`, no `WebSecurityConfigurerAdapter`), trailing slash no longer matched, `AutoConfiguration.imports` replacing `spring.factories` for auto-configuration, renamed properties (`spring.redis` → `spring.data.redis`), Hibernate 6 semantics, Sleuth replaced by Micrometer Tracing, and `HttpStatus` usage changed to `HttpStatusCode` in some APIs. `spring-boot-properties-migrator` flags properties at startup.

Q63. Why move the JDK first and the framework second?

Each step then has one cause for each failure, which makes breakage easy to diagnose and revert. Boot 2.7 runs on 17, so the JDK upgrade can ship on its own and get production mileage. Doing both together means an `InaccessibleObjectException`, a `jakarta` import failure and a Hibernate query change all land in one big branch. Small, independently deployable steps are the senior answer.

Q64. How would you test a JDK upgrade beyond the unit tests?

Integration tests with real dependencies (Testcontainers), a load test comparing throughput, p99 latency and GC logs against the old version, a startup log check for warnings, and a staged rollout (one instance or a canary) with JFR recordings to compare allocation and lock behaviour. Check container memory too: heap ergonomics and native memory use can shift between releases.

Q65. Which Java 21 features do you use in your project?

RefinePoint runs on Java 21: records for DTOs and for the `LlmClient` contract (`LlmRequest`, `LlmResponse`, `Message`, `Attachment`), a sealed `Node` hierarchy for the rule language, `instanceof` patterns throughout (`OrgContextService` uses

`attrs instanceof ServletRequestAttributes servlet`), switch expressions and text blocks for SQL in `JdbcTemplate` work queues such as `RefreshWorker.claim()`. Being honest about what you did **not** use (JPMS, preview features) is as credible as listing what you did.